BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÁO CÁO THỰC TẬP CƠ SỞ

NGHIÊN CỨU, PHÁT TRIỂN HỆ THỐNG HỌC MÁY TỰ ĐỘNG DỰA TRÊN AIRFLOW

Người hướng dẫn:

TS. Lê Đức Thuận - Khoa CNTT, Học viện Kỹ thuật mật mã

Sinh viên thực hiện:

Lê Thành An - AT210501

Lê Viết Phương Anh - AT210502

Lê Việt Hoàng - AT210521

Mã nhóm: 63

Hà Nội - 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÁO CÁO THỰC TẬP CƠ SỞ

NGHIÊN CỨU, PHÁT TRIỂN HỆ THỐNG HỌC MÁY TỰ ĐỘNG DỰA TRÊN AIRFLOW

Người hướng dẫn:

TS. Lê Đức Thuận - Khoa CNTT, Học viện Kỹ thuật mật mã

Sinh viên thực hiện:

Lê Thành An - AT210501

Lê Viết Phương Anh - AT210502

Lê Việt Hoàng - AT210521

Mã nhóm: 63

Hà Nội - 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Hà Nội, ngày ..... tháng ..... năm 2026

GIÁO VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Nhóm chúng em trân trọng cảm ơn thầy TS. Lê Đức Thuận, Khoa Công nghệ thông tin, Học viện Kỹ thuật mật mã đã hướng dẫn và định hướng nội dung đề tài "Nghiên cứu, phát triển hệ thống học máy tự động dựa trên Airflow". Những góp ý của thầy giúp nhóm xác định phạm vi nghiên cứu, tổ chức quy trình thực hiện và hoàn thiện báo cáo.

Nhóm chúng em xin cảm ơn các thầy đã trang bị kiến thức nền tảng để nhóm vận dụng vào quá trình xây dựng, triển khai và đánh giá hệ thống. Báo cáo trình bày kết quả đã được ghi nhận trong hồ sơ dự án, đồng thời nêu rõ các hạn chế và nội dung cần tiếp tục kiểm chứng.

Hà Nội, ngày ..... tháng ..... năm 2026

Nhóm sinh viên thực hiện

LỜI CAM ĐOAN

Nhóm sinh viên gồm Lê Thành An (AT210501), Lê Viết Phương Anh (AT210502) và Lê Việt Hoàng (AT210521) thuộc Nhóm 63 thực hiện đề tài "Nghiên cứu, phát triển hệ thống học máy tự động dựa trên Airflow" dưới sự hướng dẫn của TS. Lê Đức Thuận. Nhóm chịu trách nhiệm về nội dung trình bày, cách sử dụng tài liệu tham khảo và việc diễn giải các kết quả thực nghiệm trong báo cáo.

Các kiến thức tham khảo được dẫn nguồn trong nội dung và danh mục tài liệu tham khảo. Kết quả mô hình được trình bày cùng phạm vi dữ liệu và phương pháp đánh giá. Hình ảnh minh họa thể hiện các chức năng đã xây dựng trong hai dự án; kết quả trên một tệp minh họa không được dùng thay cho đánh giá độ chính xác trên tập kiểm thử độc lập.

Hà Nội, ngày ..... tháng ..... năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT.....i
DANH MỤC HÌNH.....ii
DANH MỤC BẢNG.....iii
LỜI MỞ ĐẦU.....1
CHƯƠNG 1 CƠ SỞ LÝ THUYẾT.....3
1.1 Học tự động và tự động hóa trong học máy.....3
1.1.1 Khái niệm và phạm vi tự động hóa.....3
1.1.2 Vòng đời dữ liệu và mô hình.....3
1.1.3 Tái lập và kiểm soát thí nghiệm.....4
1.2 Tìm hiểu về Airflow.....5
1.2.1 Kiến trúc và vai trò của Airflow.....5
1.2.2 DAG run và thời điểm dữ liệu.....6
1.2.3 Operator TaskFlow và truyền dữ liệu.....7
1.2.4 Executor và lựa chọn triển khai.....8
1.3 Một số bài toán cần quản lý tự động được đặt ra.....9
1.3.1 Bài toán phân tích cổ phiếu.....9
1.3.2 Bài toán mã độc và an toàn thông tin.....10
1.3.3 Dữ liệu và ranh giới thông tin.....11
1.4 Các mô hình học máy tích hợp trong hệ thống.....12
1.4.1 Random Forest.....12
1.4.2 XGBoost.....12
1.4.3 Mạng MLP.....13
1.4.4 Mạng LSTM.....14
1.4.5 FinBERT và kết hợp tín hiệu.....15
1.4.6 Chỉ số đánh giá và cách đọc.....16
1.4.7 Logistic Regression và các mô hình đối chứng.....17
1.4.8 Hiệu chỉnh xác suất và bất định đánh giá.....17
1.5 Mô tả yêu cầu của hệ thống.....18
1.5.1 Yêu cầu chức năng.....18
1.5.2 Yêu cầu phi chức năng.....19
1.5.3 Tiêu chí nghiệm thu và phạm vi chứng minh.....20
1.6 Kết luận chương 1.....20
CHƯƠNG 2 XÂY DỰNG CÁC MODULE CHO HỆ THỐNG.....21
2.1 Module thu thập dữ liệu.....21

2.1.1 Kiến trúc module và luồng dữ liệu.....21
2.1.2 Thu thập giá cổ phiếu.....21
2.1.3 Thu thập thông tin thị trường và đặc trưng nền.....23
2.1.4 Thu thập và lưu quan sát tin tức.....24
2.1.5 Danh mục dữ liệu an toàn thông tin.....25
2.1.6 Hợp đồng đầu vào và kiểm tra nguồn.....27
2.2 Module xử lý và đồng bộ dữ liệu.....28
2.2.1 Làm sạch và kiểm định OHLCV.....28
2.2.2 Xây dựng đặc trưng cổ phiếu.....29
2.2.3 Tạo nhãn và chính sách lợi suất.....30
2.2.4 Dựng chuỗi và chia tập theo thời gian.....31
2.2.5 Biểu diễn văn bản trong dữ liệu malware.....32
2.2.6 Danh mục nhóm và ngăn rò rỉ malware.....32
2.2.7 Lưu trữ dữ liệu và phiên bản bất biến.....33
2.2.8 Đồng bộ lần chạy và khôi phục.....34
2.3 Module xây dựng và quản lý mô hình.....35
2.3.1 Huấn luyện XGBoost cho stock.....35
2.3.2 Huấn luyện LSTM cho stock.....36
2.3.3 Suy luận FinBERT và điều kiện ghép tin.....37
2.3.4 Huấn luyện Random Forest đa dữ liệu.....38
2.3.5 Huấn luyện XGBoost đa dữ liệu.....39
2.3.6 Huấn luyện MLP đa dữ liệu.....40
2.3.7 Bộ mô hình phục vụ scanner đa miền (PE, PDF, Script, Office).....41
2.3.8 Đăng ký và lựa chọn phiên bản mô hình.....42
2.4 Module đánh giá mô hình.....43
2.4.1 Đánh giá mô hình cổ phiếu.....43
2.4.2 Tổng hợp và đánh giá malware.....44
2.4.3 Dashboard API và tính nhất quán đầu ra.....45
2.5 Kết luận chương 2.....46
CHƯƠNG 3 TRIỂN KHAI VÀ THỰC NGHIỆM HỆ THỐNG.....47
3.1 Thiết lập môi trường.....47
3.1.1 Thành phần và cấu hình triển khai.....47
3.1.2 Cổng dịch vụ và tổ chức dữ liệu.....48
3.1.3 Trình tự khởi động và kiểm tra.....48
3.2 Triển khai các Module.....49

3.2.1 Triển khai các module stock..... 49
3.2.2 Triển khai các module malware..... 50
3.2.3 Triển khai scanner và các giới hạn suy luận..... 51
3.3 Thiết kế DAG trong Airflow..... 52
3.3.1 Thiết kế DAG điều phối stock..... 52
3.3.2 Thiết kế DAG điều phối malware..... 53
3.3.3 DAG đánh giá và điều kiện công bố..... 54
3.4 Kết quả và đánh giá thực nghiệm..... 55
3.4.1 Phạm vi và phương pháp đối chiếu..... 55
3.4.2 Kết quả kiểm định chéo K-fold của mô hình XGBoost trên 100 mã cổ phiếu..... 56
3.4.3 Kết quả đánh giá Holdout và Champion Gate trên 100 mã cổ phiếu..... 57
3.4.4 Theo dõi thí nghiệm và công bố stock..... 59
3.4.5 So sánh ba thuật toán malware trên đa tập dữ liệu an ninh mạng..... 62
3.4.6 Metric bộ model scanner đã lưu..... 69
3.4.7 Minh họa chức năng quét tệp..... 70
3.4.8 Thiết kế thực nghiệm và quy mô dữ liệu..... 72
3.4.9 Kết quả nhánh dự báo lợi suất T+5..... 73
3.4.10 Kiểm thử phần mềm và kiểm chứng điều phối..... 74
3.4.11 Đánh giá tổng hợp và hướng hoàn thiện..... 76
KẾT LUẬN..... 78
TÀI LIỆU THAM KHẢO..... 79

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Diễn giải	Ý nghĩa trong đề tài
DAG	Directed Acyclic Graph	Đồ thị công việc không có chu trình
ML	Machine Learning	Học máy
DL	Deep Learning	Học sâu
MLOps	Machine Learning Operations	Quản lý vòng đời hệ thống học máy
ETL	Extract Transform Load	Thu thập, biến đổi và lưu dữ liệu
OHLCV	Open High Low Close Volume	Giá mở, cao, thấp, đóng và khối lượng
PE	Portable Executable	Định dạng tệp thực thi Windows
RF	Random Forest	Rừng ngẫu nhiên
MLP	Multi Layer Perceptron	Mạng nơ ron truyền thẳng nhiều lớp
LSTM	Long Short Term Memory	Mạng hồi quy có trạng thái nhớ
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
TF IDF	Term Frequency Inverse Document Frequency	Biểu diễn văn bản theo trọng số từ
ROC AUC	Area Under ROC Curve	Chỉ số phân biệt thứ hạng dự đoán
FPR	False Positive Rate	Tỷ lệ báo động giả
TP FP TN FN	Các ô ma trận nhầm lẫn	Đúng sai theo lớp dương và âm
API	Application Programming Interface	Giao diện trao đổi giữa các dịch vụ
RSS	Really Simple Syndication	Nguồn phân phối tin tức
UTC	Coordinated Universal Time	Mốc thời gian chuẩn lưu trong hệ thống
S3	Simple Storage Service API	Giao thức lưu trữ đối tượng
SOC	Security Operations Center	Ngữ cảnh sử dụng bảng điều khiển an ninh
T+1, T+5	Phiên giao dịch	Chỉ các phiên giao dịch tiếp theo
Artifact	Sản phẩm được lưu	Có thể gồm dữ liệu, trọng số, bộ tiền xử lý, cấu hình và kết quả đánh giá.

DANH MỤC HÌNH

Hình 1.1 Vòng đời xử lý và các điểm kiểm soát sản phẩm học máy..... 4
Hình 1.2 Thành phần điều phối và kho sản phẩm của hệ thống.....6
Hình 1.3 Ranh giới học lựa chọn và kiểm thử theo thời gian..... 11
Hình 2.1 Điều kiện để một phiên bản tin tham gia dự đoán lịch sử.....25
Hình 2.2 Các cột đặc trưng của bộ dữ liệu PE.....27
Hình 2.3 Chuỗi lịch sử và nhãn của phiên kế tiếp..... 32
Hình 3.1 Bảng đặc trưng đầu vào của dự án stock..... 50
Hình 3.2 Giao diện bảng tổng hợp cổ phiếu và trạng thái các nguồn..... 50
Hình 3.3 Danh sách năm DAG malware trong Airflow..... 51
Hình 3.4 Danh sách chín DAG trong dự án stock..... 53
Hình 3.5 Đồ thị mười task điều phối của master stock..... 53
Hình 3.6 Master malware với ba nhánh huấn luyện song song.....54
Hình 3.7 Bốn task của DAG XGBoost malware..... 54
Hình 3.8 Bốn bước của DAG đánh giá và công bố malware..... 55
Hình 3.9 Ba nhánh đánh giá mô hình stock và bước tổng hợp..... 55
Hình 3.10 Kiểm định dữ liệu và kết quả holdout trên dashboard stock.....58
Hình 3.11 Thông báo đánh giá XGBoost LSTM và trạng thái Ensemble..... 59
Hình 3.12 Các run LSTM được theo dõi trong MLflow..... 60
Hình 3.13 Các run XGBoost được theo dõi trong MLflow.....60
Hình 3.14 Công bố dự báo stock và trạng thái safe mode qua Telegram..... 61
Hình 3.15 Ví dụ phân tích cổ phiếu UBER theo yêu cầu.....61
Hình 3.16 Dashboard malware ở phiên bản lịch sử (không đại diện báo cáo 23 bộ dữ liệu)..... 67
Hình 3.17 Accuracy, F1, ROC-AUC và FPR được lưu trên 23 bộ dữ liệu malware..68
Hình 3.18 F1 và FPR trung bình theo tám nhóm dữ liệu an ninh mạng..... 69
Hình 3.19 Kết quả quét 7z.exe ở phiên bản Scanner lịch sử..... 71
Hình 3.20 Kết quả quét fake_trojan_test.exe ở phiên bản Scanner lịch sử.....71
Hình 3.21 Master stock hoàn tất ngày 15 tháng 10 năm 2026..... 74
Hình 3.22 Master malware hoàn tất ngày 13 tháng 10 năm 2026.....75
Hình 3.23 Tổng kết 220 ca kiểm thử stock.....76
Hình 3.24 Kết quả 12 ca kiểm thử malware..... 76

DANH MỤC BẢNG

Bảng 1.1 Thông tin tối thiểu để đối chiếu một thí nghiệm.....5
Bảng 1.2 Các thời điểm cần phân biệt trong bài toán stock.....7
Bảng 1.3 Thành phần điều phối và trách nhiệm kiểm soát.....8
Bảng 1.4 So sánh cấu hình thực thi hiện có.....9
Bảng 1.5 Phân biệt mục tiêu của các nhánh stock.....10
Bảng 1.6 Các chỉ số dùng trong phân tích kết quả.....16
Bảng 1.7 Vai trò của các mô hình đối chứng.....17
Bảng 1.8 Đặc tả chức năng của hệ thống.....19
Bảng 2.1 Hai luồng xử lý trong dự án malware.....21
Bảng 2.2 Danh mục 100 mã cổ phiếu theo cấu hình ngày 24/10/2026.....22
Bảng 2.3 Dữ liệu bối cảnh và điểm kiểm soát thời gian.....24
Bảng 2.4 Danh mục 23 tập dữ liệu an ninh mạng trong hệ thống.....26
Bảng 2.5 Hợp đồng dữ liệu cần giữ.....28
Bảng 2.6 Các điểm kiểm định dữ liệu giá.....29
Bảng 2.7 Các họ đặc trưng stock và điều kiện tạo.....30
Bảng 2.8 Quy tắc tạo nhãn trong bài toán stock.....31
Bảng 2.9 Điểm kiểm soát rò rỉ trong mã malware.....33
Bảng 2.10 Lưu trữ sản phẩm theo hai dự án.....34
Bảng 2.11 Một số tình huống đồng bộ.....35
Bảng 2.12 Chuỗi xử lý huấn luyện XGBoost stock.....36
Bảng 2.13 Cấu hình và điểm kiểm soát LSTM stock.....37
Bảng 2.14 Metadata cần giữ cho suy luận cảm xúc.....38
Bảng 2.15 Cấu hình Random Forest benchmark và scanner.....39
Bảng 2.16 Một số tham số XGBoost benchmark.....40
Bảng 2.17 Hai loại mạng MLP trong dự án.....41
Bảng 2.18 Thành phần đóng gói scanner đa miền hiện tại.....42
Bảng 2.19 Phân biệt các loại đánh giá stock.....44
Bảng 2.20 Nguồn kết quả malware được phân biệt trong báo cáo.....45
Bảng 2.21 Thông tin nối sản phẩm với giao diện.....46
Bảng 3.1 Thành phần môi trường hai dự án.....47
Bảng 3.2 Các cổng dịch vụ trong cấu hình mặc định.....48
Bảng 3.3 Trình tự kiểm tra trước thực nghiệm.....49
Bảng 3.4 Đầu ra của từng module malware.....51
Bảng 3.5 Phân biệt điểm mô hình và điểm rủi ro scanner.....52

Bảng 3.6 Nguồn và phạm vi các bảng kết quả.....56
Bảng 3.7 Kết quả kiểm định chéo K-fold của mô hình XGBoost trên 100 mã cổ phiếu.....57
Bảng 3.8 Đánh giá hiệu năng Holdout của các mô hình stock trên 12.500 mẫu kiểm định..... 58
Bảng 3.9 Kết quả thực nghiệm chi tiết của ba thuật toán trên 23 tập dữ liệu an ninh mạng.....62
Bảng 3.10 Thống kê hiệu năng trung bình toàn diện và xếp hạng ba thuật toán trên 23 tập dữ liệu..... 67
Bảng 3.11 Metric Scanner PE trong metadata đa miền ngày 24/10/2026..... 70
Bảng 3.12 Quy mô bốn bộ dữ liệu huấn luyện Scanner đa miền..... 72
Bảng 3.13 Các mốc chia tập của cấu hình stock tham chiếu.....72
Bảng 3.14 Sai số lợi suất T+5 trong hồ sơ ngày 08 tháng 10.....73
Bảng 3.15 Kết quả các lần chạy master trên ảnh bổ sung..... 74
Bảng 3.16 Kết quả kiểm thử phần mềm của hai dự án.....76
Bảng 3.17 Các hướng hoàn thiện ưu tiên.....77

LỜI MỞ ĐẦU

Lý do chọn đề tài

Một hệ thống học máy cần nhiều bước liên tiếp: thu thập dữ liệu, làm sạch, xây dựng đặc trưng, huấn luyện, đánh giá và công bố kết quả. Nếu các bước được chạy thủ công, dữ liệu và mô hình dễ bị nhầm phiên bản; việc lặp lại thí nghiệm hoặc tìm nguyên nhân lỗi mất nhiều thời gian. Apache Airflow cung cấp mô hình DAG để xác định quan hệ phụ thuộc và theo dõi trạng thái từng công việc [1], [2].

Mục tiêu nghiên cứu

Mục tiêu là xây dựng và phân tích bốn nhóm module: thu thập dữ liệu, xử lý và đồng bộ, xây dựng và quản lý mô hình, đánh giá mô hình. Airflow điều phối các module; Docker Compose, PostgreSQL và MinIO hỗ trợ môi trường và lưu trữ. Kết quả được trình bày trên giao diện và các báo cáo có thể đối chiếu.

Stock đối chiếu XGBoost, LSTM và Logistic Regression cho hướng giá T+1 trong thí nghiệm lịch sử; FinBERT bổ sung cảm xúc tin tức. Malware đối chiếu Random Forest, XGBoost và MLP trên 23 bộ dữ liệu, đồng thời phục vụ Scanner cho PE, PDF, Script và Office. Mỗi bảng so sánh được gắn với mục tiêu và phạm vi đánh giá riêng.

Đối tượng và phạm vi

Đối tượng nghiên cứu là mã nguồn, cấu hình, dữ liệu và kết quả của hai dự án airflow-stock và air_flow (malware). Báo cáo cập nhật cấu hình và kết quả thực nghiệm mới nhất: stock vận hành toàn diện trên danh mục 100 mã cổ phiếu lớn nhất S&P theo 11 nhóm ngành GICS với lịch sử thu thập 5 năm (đạt ~126.000 bản ghi nến OHLCV, 85 đặc trưng cho XGBoost, tensor 3D 30 phiên cho LSTM, và 2.488 bản tin RSS cho FinBERT); malware sở hữu 23 bộ dữ liệu benchmark và 4 nhóm mô hình Scanner đa miền. Các kết quả thực nghiệm và đánh giá chất lượng mô hình trong báo cáo phản ánh đầy đủ trên tập kiểm định độc lập của hệ thống 100 mã.

Phương pháp và bố cục

Phương pháp gồm phân tích mã, kiểm tra lược đồ dữ liệu, đọc sản phẩm CSV/JSON, đối chiếu ảnh với cấu trúc DAG và tham khảo tài liệu chính thức. Kết quả lịch sử và kết quả trên ảnh ngày 13/10 được ghi tách theo nguồn. Thí nghiệm quét một tệp minh họa không thay thế đánh giá trên tập dữ liệu có nhãn độc lập.

Báo cáo gồm ba chương. Chương 1 trình bày cơ sở lý thuyết và yêu cầu hệ thống. Chương 2 mô tả bốn nhóm module của hai bài toán. Chương 3 trình bày môi trường,

triển khai module, thiết kế DAG và kết quả thực nghiệm. Kết luận tổng hợp đóng góp, hạn chế và hướng phát triển.

CHƯƠNG 1 CƠ SỞ LÝ THUYẾT

1.1 Học tự động và tự động hóa trong học máy

1.1.1 Khái niệm và phạm vi tự động hóa

Học máy xây dựng hàm dự đoán từ dữ liệu thay vì mô tả toàn bộ quy tắc bằng chương trình. Với tập quan sát gồm đặc trưng x và nhãn y, quá trình huấn luyện tìm tham số giúp giảm sai lệch giữa đầu ra mô hình và nhãn trên dữ liệu được phép sử dụng. Mô hình sau huấn luyện được áp dụng cho quan sát mới. Giá trị của một hệ thống học máy phụ thuộc cả chất lượng hàm dự đoán lẫn cách dữ liệu đến được hàm đó.

Trong đề tài, học tự động được hiểu theo hai lớp. Lớp thuật toán tự cập nhật tham số từ dữ liệu, chẳng hạn trọng số mạng nơ ron hoặc cấu trúc cây quyết định. Lớp quy trình tự thực hiện các bước thu thập, chuẩn hóa, huấn luyện, đánh giá và lưu kết quả theo quan hệ phụ thuộc. Apache Airflow đảm nhiệm lớp quy trình. Nó không tự quyết định nhãn nào đúng, thuật toán nào phù hợp hay kết quả nào đủ ý nghĩa thống kê để triển khai.

Tự động hóa mang lại lợi ích rõ nhất khi một thao tác được lặp lại nhiều lần và có điều kiện đầu vào xác định. Ví dụ, dữ liệu cổ phiếu chỉ nên được công bố sau khi kiểm tra đủ mã và đúng phiên. Báo cáo malware chỉ nên được tổng hợp từ những lần huấn luyện được nhận diện rõ. Nếu tự động chạy trên dữ liệu sai hoặc ghi đè nguồn kiểm thử, hệ thống có thể tạo nhiều kết quả thiếu tin cậy hơn trong thời gian ngắn hơn.

Dự án stock có cơ chế tìm siêu tham số bằng Optuna; malware hiện sử dụng cấu hình định sẵn ở các DAG. AutoML đầy đủ còn bao gồm tìm kiếm đặc trưng, lựa chọn thuật toán và chiến lược triển khai. Vì vậy đề tài được mô tả là tự động hóa quy trình học máy có thành phần tối ưu ở stock. Việc lựa chọn phải dùng train và validation; test phục vụ đánh giá cuối [3].

Vì vậy, mục tiêu của đề tài là xây dựng quy trình có thể kiểm tra và chạy lại, đồng thời đánh giá nghiêm túc giới hạn của mô hình. Tự động hóa được coi là phương tiện làm rõ tính nhất quán của thí nghiệm, không phải cơ sở để mặc nhiên khẳng định độ chính xác cao.

1.1.2 Vòng đời dữ liệu và mô hình

Hai dự án có thể được nhìn theo cùng một vòng đời: nhận dữ liệu, kiểm tra, tạo đặc trưng, huấn luyện, đánh giá, lưu phiên bản và cung cấp đầu ra. Mỗi giai đoạn cần một sản phẩm xác định để bước sau sử dụng. Một tệp CSV không chỉ cần tồn tại; nó còn cần lược đồ, thời điểm nguồn, quy tắc nhãn và trạng thái kiểm định. Tương tự,

trọng số chưa đủ để tái lập suy luận nếu thiếu thứ tự đặc trưng hoặc bộ chuẩn hóa tương ứng.

Trong bài toán stock, dữ liệu thay đổi theo phiên và nhãn T+1 chỉ có sau phiên tiếp theo. Điều này tạo hai luồng khác nhau: phát hành dự đoán hiện tại và đánh giá dự đoán đã trưởng thành. Trong malware, một tập CSV có thể tương đối ổn định nhưng ý nghĩa lớp, phương pháp thu mẫu và cách trích đặc trưng khác nhau giữa các nguồn. Vòng đời phải giữ được những khác biệt đó thay vì chỉ dùng một bộ tham số chung.

Một điểm kiểm soát cần thiết là ranh giới công bố. Nếu trọng số đã tải lên nhưng scaler chưa có, mô hình chưa nên xuất hiện như phiên bản hoàn chỉnh. Nếu một task thử lại sau lỗi mạng, nó cần biết sản phẩm nào đã hoàn tất và sản phẩm nào còn dang dở. Cách tổ chức này giúp truy vấn kết quả theo lần chạy, đồng thời hỗ trợ hoàn nguyên về phiên bản trước khi ứng viên mới không đáp ứng yêu cầu.



Hình 1.1 Vòng đời xử lý và các điểm kiểm soát sản phẩm học máy

1.1.3 Tái lập và kiểm soát thí nghiệm

Khả năng tái lập yêu cầu xác định được dữ liệu, mã nguồn, cấu hình và môi trường của một lần chạy. Tên tệp giống nhau không chứng minh nội dung giống nhau. Dự án stock ghi nhận một đối tượng MinIO cùng tên đã thay nội dung giữa các thí nghiệm; vì vậy các lượt so sánh sau dùng bản sao đóng băng và SHA 256. Trường hợp này cho thấy quản lý phiên bản dữ liệu là yêu cầu thực tế, không chỉ là lựa chọn trình bày báo cáo.

Tính nhất quán còn phụ thuộc cách tổ chức đánh giá. Nếu cùng một cửa sổ test được xem nhiều lần để thay đặc trưng, số cây hay ngưỡng, cửa sổ đó đã tham gia quá trình nghiên cứu. Một lần chạy sau dù có chia train và test đúng vẫn không trở thành kiểm chứng tương lai độc lập. Báo cáo vì vậy phân biệt kiểm định lịch sử mang tính khám phá với dự đoán được lưu trước khi nhãn xuất hiện.

Kiểm thử phần mềm và đánh giá mô hình trả lời hai câu hỏi khác nhau. Ca kiểm thử có thể xác nhận việc thay giá tương lai không làm đổi đặc trưng quá khứ, hoặc việc thiếu trọng số được báo là không khả dụng. Các ca đó không chứng minh AUC sẽ cao trên thị trường tương lai. Ngược lại, một AUC cao trong CSV không chứng minh API dùng đúng scaler hoặc DAG truyền đúng phiên bản.

Tự động hóa cần giữ dấu vết cả thành công và thất bại. Stock có snapshot, model card và tracking; malware hiện lưu CSV report và metadata scanner. Muốn mở rộng khả năng truy nguyên cần bổ sung split manifest, hash model và dự đoán từng mẫu. Tracking hỗ trợ tra cứu, nhưng không thay thế kiểm soát tập đánh giá [4].

Bảng 1.1 Thông tin tối thiểu để đối chiếu một thí nghiệm

Thành phần	Thông tin cần lưu	Lợi ích kiểm chứng
Dữ liệu	Khóa đối tượng, hash, khoảng thời gian	Xác định đúng tập đầu vào
Mô hình	Cấu hình, seed, checkpoint, ngưỡng	Phân biệt các ứng viên
Đánh giá	Nhãn và xác suất từng mẫu	Tính lại các chỉ số
Vận hành	Mã lần chạy và thời điểm thực	Ghép đúng các sản phẩm

1.2 Tìm hiểu về Airflow

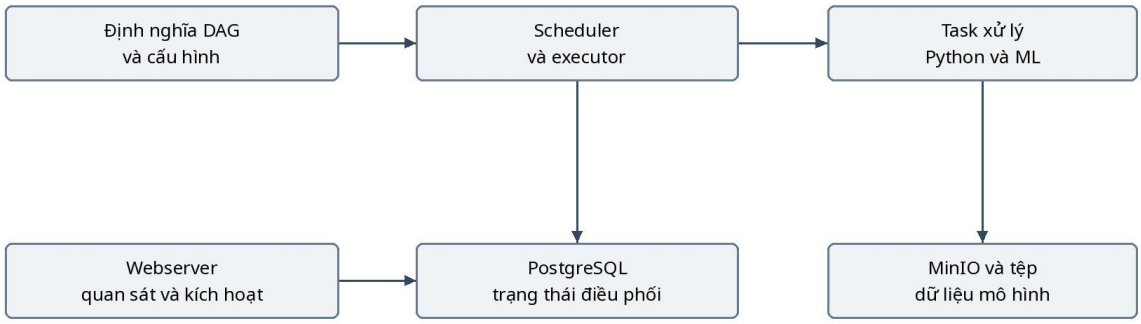
1.2.1 Kiến trúc và vai trò của Airflow

Airflow mô tả quy trình bằng DAG, trong đó các nút là task và các cạnh biểu diễn quan hệ phụ thuộc. Scheduler đọc định nghĩa, xác định công việc đủ điều kiện và chuyển cho executor. Webserver hỗ trợ quan sát trạng thái, xem log và kích hoạt quy trình. Metadata database lưu thông tin điều phối. Những vai trò này tạo nền tảng cho việc quản lý chuỗi tác vụ của cả hai dự án [1].

Đối với đề tài, có thể phân biệt dữ liệu điều khiển và dữ liệu nghiệp vụ. PostgreSQL của Airflow lưu trạng thái task, lần chạy và cấu hình liên quan; giá cổ phiếu, đặc trưng và trọng số được lưu thành tệp hoặc đối tượng MinIO. Redis trong cấu hình malware làm broker cho Celery, không đóng vai trò kho dữ liệu huấn luyện. Việc phân biệt giúp tránh suy luận sai rằng chỉ cần sao lưu metadata database là đủ khôi phục toàn bộ kết quả nghiên cứu.

Airflow không thực hiện thay công việc chuyên môn của từng task. Chương trình bên trong task vẫn phải đọc dữ liệu, xử lý lỗi và lưu kết quả đúng cách. Khi task kết thúc thành công do nuốt ngoại lệ, scheduler chỉ thấy trạng thái thành công; nó không

biết một bộ dữ liệu đã bị bỏ qua. Bởi vậy, tiêu chí thành công của task cần bám vào sản phẩm đầu ra, số lượng tập được xử lý và các điều kiện kiểm định.



Hình 1.2 Thành phần điều phối và kho sản phẩm của hệ thống

1.2.2 DAG run và thời điểm dữ liệu

Một định nghĩa DAG có thể tạo nhiều lần chạy. Mỗi lần chạy chứa các task instance với trạng thái riêng. Tên DAG xác định loại quy trình, còn mã lần chạy xác định một lần thực hiện cụ thể. Dự án stock truyền pipeline_run_id để liên kết các DAG con và hạn chế ghép nhầm dự đoán cùng ngày nhưng khác lượt thực thi. Cách phân biệt này cần được áp dụng nhất quán đến báo cáo và giao diện.

Airflow phân biệt thời gian logic, khoảng dữ liệu và thời điểm task thực sự chạy. Khoảng dữ liệu mô tả phần dữ liệu mà quy trình chịu trách nhiệm xử lý; thời gian thực cho biết khi nào quan sát, huấn luyện hoặc phát hành xảy ra [5]. Với cổ phiếu, việc nhầm các khái niệm này có thể khiến dự đoán bị gắn thời điểm sớm hơn khả năng quan sát thực tế. Với tin tức, thời điểm xuất bản còn khác thời điểm hệ thống thu được bài viết.

Lịch stock khai báo chuỗi cron 0 22 * * 1-5. Trong cách diễn giải UTC của dự án, đó là 22 giờ từ thứ Hai đến thứ Sáu, tương ứng 05 giờ sáng ngày kế tiếp tại Việt Nam. Lịch được đặt sau thời điểm đóng cửa thông thường của thị trường Mỹ; tuy nhiên cron không tự nhận biết ngày nghỉ thị trường. Dữ liệu phiên nhận được vẫn phải được kiểm định. Không thể dùng việc lịch chạy đúng giờ để thay thế kiểm tra tính mới của giá.

Các DAG con được master kích hoạt để tổ chức toàn tuyến. Master malware hiện không đặt lịch tự động; master stock có lịch ngày làm việc theo UTC. catchup=False giới hạn chạy bù tự động. Xử lý dữ liệu cũ cần backfill với nguồn và nhãn được kiểm soát. Gắn ngày cũ cho một lần chạy hiện tại không biến dữ liệu vừa thu thành thông tin đã có trong quá khứ.

Bảng 1.2 Các thời điểm cần phân biệt trong bài toán stock

Mốc	Ví dụ ý nghĩa	Quy tắc sử dụng
Phiên tham chiếu	Phiên giá đóng cuối cùng	Làm cơ sở tạo dự đoán
issued_ at	Thời điểm phát hành thực	Không được lùi về quá khứ
Ngày kết quả	Phiên T+1 hoặc T+5	Ghép từ phiên quan sát
observed_ at	Lần đầu thấy nội dung tin	Điều kiện ghép lịch sử

1.2.3 Operator TaskFlow và truyền dữ liệu

PythonOperator phù hợp khi một hàm Python đã có cần được đưa vào DAG. TaskFlow dùng decorator để biến hàm thành task và biểu diễn quan hệ truyền kết quả. TriggerDagRunOperator kích hoạt một DAG khác, thuận tiện cho mô hình điều phối nhiều nhánh. Trong stock, master sử dụng operator kích hoạt các bước thu thập, huấn luyện và đánh giá; trong malware, các DAG thuật toán dùng TaskFlow cho chuỗi extract, transform, train và save.

XCom thích hợp với thông điệp nhỏ như đường dẫn đối tượng hoặc kết quả kiểm tra, không nên thay kho lưu bảng dữ liệu lớn [6]. Dự án malware trả về danh sách đường dẫn CSV đã xử lý và danh sách bản ghi chỉ số. Điều đó giảm lượng dữ liệu phải trao đổi qua metadata database. Tuy nhiên, đường dẫn cục bộ chỉ sử dụng được nếu task sau nhìn thấy cùng hệ thống tệp; đây là điều kiện đặc biệt quan trọng với CeleryExecutor.

Trong cấu hình hiện tại, các dịch vụ Airflow malware cùng gắn dags vào /opt/airflow/dags. Tệp xử lý và report vì vậy có thể được dùng giữa các container trên cùng máy. Nếu worker chuyển sang máy khác, đường dẫn này không tự đồng bộ. Hướng mở rộng là lưu sản phẩm vào object storage và truyền bucket, key cùng hash bằng XCom.

Không phải mọi quan hệ truyền biến đều biểu diễn đủ điều kiện nghiệp vụ. Một danh sách rỗng hoặc file report cũ có thể vẫn đi qua các task nếu mã không kiểm tra tính mới. Các DAG malware hiện bắt một số lỗi theo dataset rồi tiếp tục; cần đọc log và kiểm tra số tập đã xử lý. Bước fan-in chỉ đảm bảo quan hệ chờ giữa các DAG, còn tính đủ và cùng phiên bản của sản phẩm cần điều kiện bổ sung.

Bảng 1.3 Thành phần điều phối và trách nhiệm kiểm soát

Cơ chế	Sử dụng trong dự án	Điểm kiểm soát
PythonOperator	Các task stock	Hàm phải xác nhận đầu ra
TaskFlow	DAG thuật toán malware	Kiểm tra danh sách sản phẩm
TriggerDagRunOperator	Hai master DAG	Đợi đúng lần chạy con
XCom	Metadata và đường dẫn	Giữ kích thước nhỏ
MinIO	Dataset, model, report	Phiên bản và quyền truy cập

1.2.4 Executor và lựa chọn triển khai

Executor quyết định cách task được thực thi. LocalExecutor chạy các tiến trình task trên máy của scheduler, phù hợp môi trường triển khai một máy với yêu cầu vận hành đơn giản. CeleryExecutor phân phối task qua broker đến worker và cần cấu hình kho kết quả [7]. Hai dự án lựa chọn khác nhau nên không thể mô tả một kiến trúc executor chung cho toàn bộ đề tài.

Stock cấu hình LocalExecutor và giới hạn tài nguyên cho các dịch vụ. Master nối tuần tự nhánh XGBoost, LSTM và FinBERT để giảm mức sử dụng bộ nhớ đỉnh. Cấu trúc này tăng thời gian hoàn tất so với chạy song song trong trường hợp đủ tài nguyên, nhưng giúp hạn chế cạnh tranh RAM giữa PyTorch, XGBoost và Transformers. Hiệu quả thực tế phải được đo bằng log thời gian và tài nguyên, chưa thể suy ra chỉ từ sơ đồ.

Malware dùng CeleryExecutor, Redis và PostgreSQL; master hiện tạo ba nhánh huấn luyện song song. Các trigger chờ DAG con bằng cơ chế deferrable, giúp giải phóng worker trong thời gian đợi. Mỗi nhánh xử lý lần lượt danh mục CSV. Khả năng phân phối của executor và mức song song trong DAG là hai lựa chọn riêng; cần kiểm tra cả số slot và số luồng nội bộ của thuật toán.

Việc chọn executor cần xem xét đồng thời độ độc lập task, vị trí dữ liệu, số worker và cơ chế retry. Đối với quy mô học phần, cấu hình Docker một máy thuận tiện minh họa. Khi mở rộng, cần đo thời gian chờ, thời gian xử lý, mức RAM và khả năng phục hồi khi worker dừng. Không có số đo hiện có đủ để kết luận Celery nhanh hơn LocalExecutor trên hai tải công việc khác nhau.

Bảng 1.4 So sánh cấu hình thực thi hiện có

Tiêu chí	Stock	Malware
Executor	LocalExecutor	CeleryExecutor
Broker	Không dùng Redis	Redis 7.2
Điều phối model	Tuần tự	Song song ba nhánh
Chờ DAG con	Đợi hoàn tất	Deferrable trigger
Lưu trung gian	MinIO/snapshot	CSV cục bộ và MinIO

1.3 Một số bài toán cần quản lý tự động được đặt ra

1.3.1 Bài toán phân tích cổ phiếu

Dự án stock mang tên "Quantum Multi-Modal Stock Forecasting & MLOps Platform", tổ chức dữ liệu cho danh mục 100 mã cổ phiếu có vốn hóa thị trường lớn nhất (Top 100 S&P Market Cap) chia theo 11 nhóm ngành GICS trong config_shared.py. Hệ thống vận hành tự động theo lịch Cron 0 22 * * 1-5 (22:00 UTC / 05:00 sáng giờ Việt Nam từ Thứ Hai đến Thứ Sáu) ngay sau khi phiên giao dịch NYSE/NASDAQ đóng cửa. Mục tiêu phân loại chính là hướng giá T+1 và giá mục tiêu T+5; ba luồng dữ liệu độc lập phục vụ ba mô hình: bảng đặc trưng 2D kích thước (N, 85) gồm 85 chỉ số kỹ thuật và vĩ mô nạp cho XGBoost; tensor 3D kích thước (N, 30, 25) gồm cửa sổ lookback 30 phiên và 25 chỉ số nến/động lượng nạp cho PyTorch LSTM; và luồng văn bản tin tức tài chính RSS trong 24h–48h gần nhất nạp cho ProsusAI FinBERT. Toàn bộ các kết quả thực nghiệm được kiểm định trên tập test độc lập 12.500 mẫu của 100 mã cổ phiếu.

Đối với mục tiêu T+1, nhãn dương được tạo khi giá đóng ở phiên tiếp theo lớn hơn giá đóng tham chiếu, theo chính sách giá của thí nghiệm. Quan sát cuối chưa có phiên kế tiếp phải giữ nhãn thiếu. Nếu ép nhãn thiếu thành 0, hệ thống sẽ bổ sung những mẫu giảm giá giả và làm sai cả huấn luyện lẫn đánh giá. Với lợi suất T+5, cần thống nhất lợi suất thông thường và lợi suất log trước khi quy đổi ra giá mục tiêu.

Dữ liệu cổ phiếu có cấu trúc phụ thuộc theo thời gian và theo thị trường. Các mã cùng ngày chịu ảnh hưởng bởi một số sự kiện chung; nhiều chỉ báo còn được tính từ các cửa sổ chồng lấn. Bởi vậy, chia ngẫu nhiên hàng dữ liệu dễ tạo sự trộn giữa quá khứ và tương lai. Đề tài dùng chia theo ngày chung giữa các mã và kiểm soát nhãn vượt ranh giới, đồng thời đánh giá bằng các cửa sổ walk-forward đã lưu.

Ba thuật toán phục vụ so sánh cùng mục tiêu trong báo cáo là XGBoost, LSTM và Logistic Regression. FinBERT đóng vai trò trích xuất cảm xúc văn bản tin tức theo công thức Score = (1 + Pos − Neg) / 2 thuộc khoảng [0, 1]. DAG

dag_ensemble_master kết hợp ba nguồn theo tỷ lệ Master Ensemble: Xác suất Ensemble = 35% × P_XGBoost + 35% × P_LSTM + 30% × Score_FinBERT. Toàn hệ thống áp dụng ba ngưỡng khuyến nghị thống nhất: MUA khi xác suất ≥ 55,0% (Bullish), BÁN khi xác suất ≤ 45,0% (Bearish), và ĐỨNG NGOÀI trong vùng 45,0% – 55,0% (Sideway Filter) để lọc nhiễu thị trường đi ngang. Trước khi đưa vào phục vụ sản xuất, mô hình bắt buộc phải vượt qua MLOps Champion Gate (Accuracy ≥ 48%, AUC ≥ 0,45) trên tập test độc lập 126 phiên (~6 tháng); nếu không đạt tiêu chuẩn, hệ thống tự động kích hoạt Safe Mode đưa tín hiệu về trạng thái đứng ngoài an toàn để bảo vệ nhà đầu tư.

Bảng 1.5 Phân biệt mục tiêu của các nhánh stock

Đầu ra	Đối tượng dự đoán	Cách đánh giá phù hợp
XGBoost LSTM Logistic	Hướng giá T+1	ACC, balanced ACC, AUC, log loss
FinBERT	Cảm xúc nội dung tin	Nhãn cảm xúc hoặc độ phủ tin
XGBoost return head	Lợi suất T+5	Sai số lợi suất cùng chân trời
Ensemble cố định	Điểm tổng hợp	Đánh giá riêng dự đoán đã phát hành

1.3.2 Bài toán mã độc và an toàn thông tin

Dự án malware gồm hai đường xử lý: benchmark ba thuật toán Random Forest, XGBoost và MLP trên 23 bộ dữ liệu an ninh mạng; Scanner tiếp nhận tệp và định tuyến sang bộ mô hình PE, PDF, Script hoặc Office. Bản cập nhật bổ sung các trường trích xuất còn thiếu của schema PE 77 chiều và ba bộ trích xuất đặc trưng chuyên biệt. Mô hình Scanner được huấn luyện từ bốn bộ dữ liệu tương ứng bốn loại tệp, không phải toàn bộ 23 bộ benchmark. Điểm benchmark không tự chứng minh độ chính xác quét tệp thực tế.

Các tập an toàn thông tin có thể thuộc nhiều miền như PE, URL, lưu lượng mạng hoặc văn bản tấn công. Mỗi miền có cách định nghĩa nhãn và đặc trưng khác nhau. Việc nhận diện tự động cột nhãn giúp xử lý nhiều lược đồ, nhưng thứ tự mã hóa nhãn không bảo đảm lớp số 1 luôn là mã độc. Khi đọc FPR và ROC-AUC cần biết lớp dương và loại bài toán trước khi so sánh.

Mã hiện tại trích đặc trưng PE bằng pefile nếu thư viện có sẵn, có nhánh phân tích thủ công và điền trường thiếu bằng median trong metadata. Bộ scanner kết hợp xác suất với các quy tắc về API, entropy, bộ cài và dấu hiệu chữ ký. Vì vậy điểm rủi ro

cuối cùng không đồng nhất với xác suất thô của một mô hình học máy. Đây là một lựa chọn triển khai cần được đánh giá riêng.

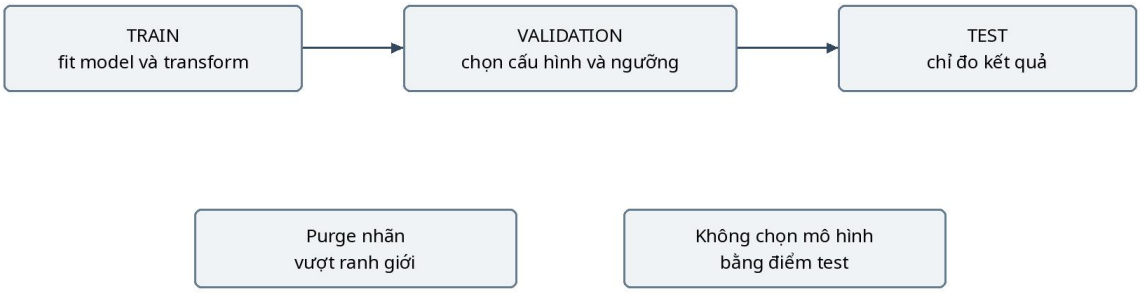
Phân tích tĩnh không cần thực thi tệp, thuận tiện cho môi trường học tập. Tuy nhiên không thấy API nguy hiểm chưa đủ để xác nhận an toàn, và entropy cao có thể do dữ liệu nén hợp lệ. Một chức năng hiển thị được kết quả quét chỉ chứng minh luồng giao diện và suy luận; muốn đo khả năng phát hiện cần tập tệp có nhãn độc lập, bao gồm cả mẫu lành tính và mẫu độc hại.

1.3.3 Dữ liệu và ranh giới thông tin

Một đặc trưng hợp lệ phải có khả năng quan sát tại thời điểm dự đoán. Đối với cổ phiếu, rolling mean phải kết thúc ở phiên tham chiếu, không được sử dụng giá của phiên cần dự đoán. Đối với tin tức, ngày xuất bản chưa đủ chứng minh hệ thống đã thu và phân tích tin trước lúc phát hành. Đối với malware, tên họ tấn công hoặc cột tổng hợp từ nhãn có thể làm lộ đáp án dù không trùng tên cột mục tiêu.

Ranh giới train, validation và test cũng áp dụng cho các phép biến đổi học từ dữ liệu. Bộ chuẩn hóa, từ điển TF IDF, phép chọn đặc trưng và thống kê điền thiếu đều có thể tiếp nhận thông tin từ test nếu được fit trước chia tập. Tài liệu scikit learn nhấn mạnh tách dữ liệu và học phép biến đổi chỉ từ phần train [8]. Trong phân tích dự án, quy tắc này được dùng để nhận diện những chỗ đã kiểm soát tốt và những chỗ còn cần sửa.

Ngoài rò rỉ trực tiếp, còn có thiên lệch do thiết kế nghiên cứu. Danh mục cổ phiếu cố định có thể không đại diện cho toàn bộ thị trường; các bộ malware tải sẵn có thể có phân phối khác lưu lượng thực tế. Số điểm dữ liệu lớn không tự giải quyết khác biệt này. Cần diễn giải kết quả trong đúng miền, đúng nguồn và đúng thời điểm thay vì gộp mọi chỉ số thành một mức chất lượng chung cho hệ thống.



Hình 1.3 Ranh giới học lựa chọn và kiểm thử theo thời gian

1.4 Các mô hình học máy tích hợp trong hệ thống

1.4.1 Random Forest

Random Forest kết hợp nhiều cây quyết định được học với sự ngẫu nhiên ở mẫu và tập đặc trưng xét tại các nút. Trong phân loại, các cây cùng đóng góp vào quyết định hoặc xác suất dự đoán. Sự đa dạng giữa cây giúp giảm phụ thuộc vào một cây đơn lẻ; tuy nhiên hiệu quả vẫn phụ thuộc dữ liệu và cách chọn độ phức tạp [9]. Báo cáo sử dụng thuật toán này làm một nhánh chính của malware.

Trong mã benchmark malware hiện tại, RF dùng 120 cây với độ sâu thích ứng từ 15 đến 30, min_samples_leaf=3 và min_samples_split=6. Scanner dùng một RF riêng với 100 cây. Các thông số cụ thể được phân tích trong Chương 2. Cùng họ thuật toán không đồng nghĩa cùng cấu hình hoặc cùng tập test.

Cây quyết định không đòi hỏi chuẩn hóa tuyến tính mọi cột như MLP, nhưng đầu vào vẫn cần lược đồ ổn định. Các DAG hiện mã hóa cột rời rạc bằng One-Hot hoặc factorize và loại định danh theo tên. Những ánh xạ cần được lưu để dùng lại khi suy luận. Quy tắc theo tên phải được rà soát theo miền, vì một dấu vết nguồn thu thập chưa bị nhận diện có thể làm test thuận lợi.

Khả năng giải thích RF có thể khai thác bằng phân tích đóng góp hoặc độ quan trọng đặc trưng, song báo cáo hiện có chưa lưu đầy đủ những thông tin này cho từng tập. Không nên dùng điểm trung bình cao để suy ra mô hình không overfit. Thí nghiệm cần thêm phân chia theo họ, nguồn hoặc thời gian, đặc biệt khi cùng một họ phần mềm tạo nhiều biến thể gần giống nhau.

Trong hệ thống tự động, RF phù hợp làm một ứng viên đối chứng vì huấn luyện tương đối trực tiếp và đầu ra xác suất dễ tích hợp. Vai trò vận hành của nó cần được quyết định bằng chỉ số theo lớp, mức báo động giả và chi phí suy luận đã đo. Số liệu hiện có chưa đủ để khẳng định RF là mô hình triển khai tốt nhất cho mọi loại đầu vào.

1.4.2 XGBoost

XGBoost là phương pháp boosting cây, xây mô hình bằng cách bổ sung các cây nhằm cải thiện mục tiêu tối ưu có điều chuẩn. Bài báo gốc trình bày cơ chế cây tăng cường có khả năng mở rộng [10]. Trong đề tài, cùng họ thuật toán này được sử dụng ở cả stock và malware, nhưng nhãn, đặc trưng, cách tìm cấu hình và thiết kế đánh giá khác nhau. Việc dùng cùng tên thuật toán không làm hai bài toán trở nên tương đương.

Hàm mục tiêu tổng quát của XGBoost tại bước lặp thứ t được tối ưu hóa theo biểu thức:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (1.1)$$

trong đó l là hàm mất mát khả vi đo lường sai lệch giữa nhãn thực tế y_i và dự báo tích lũy $\hat{y}_i^{(t-1)}$, còn $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum w_j^2$ là thành phần điều chuẩn kiểm soát độ phức tạp của mô hình cây thông qua số lượng lá T và trọng số lá w.

Ở stock, XGBoost nhận dữ liệu bảng gồm các biến kỹ thuật và bối cảnh thị trường. Chương trình huấn luyện chọn đặc trưng theo chính sách nhân quả, tìm một số siêu tham số qua Optuna và dùng đánh giá theo thời gian. Có nhánh phân loại hướng T+1 và nhánh hồi quy lợi suất T+5. Hai head phải được quản lý cùng chính sách nhãn và quy tắc suy luận tương ứng, tránh hiểu đầu ra phân loại là dự báo mức giá cụ thể.

Ở benchmark malware, cấu hình có 120 vòng tăng cường, độ sâu được đặt bằng 5 và tham số learning_rate có giá trị 0,08. Phiên bản hiện tại lấy mẫu tối đa 15.000 hàng ở transform. Việc so sánh công bằng cần thống nhất mẫu và lưu split manifest; seed cố định chưa đủ chứng minh mọi nhánh học cùng cohort.

Điều chuẩn và early stopping giúp kiểm soát độ phức tạp nhưng không thay thế việc tách dữ liệu đúng. Một mô hình nông vẫn có thể khai thác cột rò rỉ; một mô hình nhiều cây vẫn có thể đạt gần ngẫu nhiên khi tín hiệu yếu. Trong thí nghiệm stock, các biến thể đơn giản hơn được giữ như ứng viên nghiên cứu vì độ chính xác và AUC còn phụ thuộc mạnh giai đoạn.

Khi đóng gói, cần lưu trọng số, danh sách cột, phiên bản thư viện và cấu hình đầu ra. Nếu feature schema thay đổi mà inference chỉ lấy các cột đang tồn tại, mô hình có thể nhận sai thứ tự hoặc sai cách chuẩn hóa. Báo cáo ưu tiên khả năng đối chiếu đúng artifact với dataset trước khi so sánh một vài chữ số sau dấu phẩy.

1.4.3 Mạng MLP

MLP biểu diễn quan hệ phi tuyến bằng các lớp kết nối đầy đủ cùng hàm kích hoạt. Mỗi lớp biến đổi vector đầu vào và chuyển kết quả cho lớp tiếp theo. Trong phân loại nhị phân, đầu ra có thể dùng sigmoid; trong đa lớp, softmax tạo phân phối trên các lớp. Tài liệu scikit learn mô tả MLP và vai trò của chuẩn hóa đầu vào đối với quá trình tối ưu [11].

Benchmark malware sử dụng TensorFlow/Keras với các lớp Dense, điều chỉnh chiều rộng theo đầu vào, ReLU, BatchNormalization và Dropout. Scanner dùng MLPClassifier riêng của scikit-learn. Hai mạng cần được phân biệt về bộ tiền xử lý, trọng số và phép đánh giá. Các cấu hình được trình bày trong Chương 2.

Chuẩn hóa ảnh hưởng trực tiếp đến MLP. Phép fit phải chỉ học trên dữ liệu dùng huấn luyện trọng số, sau đó transform validation và test. Trong benchmark hiện tại, scaler được fit trên train ngoài trước chia validation; đây là điểm cần cải thiện. Tách ranh giới đúng giúp validation dành cho checkpoint và ngưỡng, đồng thời bảo vệ tập test khỏi việc tham gia chọn biểu diễn.

Mạng sâu hơn không mặc nhiên phù hợp hơn với mọi dữ liệu. Kết quả trên bộ PE hiện có cho thấy mô hình MLP có các chỉ số Accuracy và F1 thấp hơn hai mô hình cây được ghi trong tệp CSV benchmark. Cần phân tích phân phối lớp, độ phức tạp, hội tụ và khả năng biểu diễn trước khi tăng số lớp. Dữ liệu ngoài PE chưa được đánh giá lại đủ trong bộ nguồn hiện có, nên chưa thể suy ra hiệu quả của mạng trên mọi miền an toàn thông tin.

TensorFlow cần được kiểm soát số luồng và bộ nhớ để phù hợp tài nguyên worker. Đây là yêu cầu cần xem xét khi master malware huấn luyện các nhánh song song. Mã MLP hiện còn nhánh cài TensorFlow nếu môi trường thiếu thư viện, nên tính tái lập có thể bị ảnh hưởng. Hướng hoàn thiện là cài đủ trong image, khóa phiên bản và lưu môi trường thực tế cùng sản phẩm.

1.4.4 Mạng LSTM

LSTM nhận một chuỗi vector thay vì một hàng độc lập. Trạng thái nhớ cho phép mô hình kết hợp thông tin qua nhiều bước thời gian. Trong stock, đầu vào là chuỗi các phiên của cùng mã cổ phiếu, còn đầu ra phân loại hướng giá gắn với phiên cuối chuỗi. Điều kiện quan trọng nhất khi dựng mẫu là không để một chuỗi nối dữ liệu của hai mã hoặc chứa quan sát sau thời điểm tham chiếu [12].

Kiến trúc CalibratedStackedLSTM trong mã có lớp LSTM, LayerNorm và head Linear, ReLU, Dropout, Linear. Cấu hình cơ sở dùng hidden_dim=48, hai lớp và dropout=0.2; hai cấu hình thay thế dùng một lớp với hidden_dim 32 hoặc 64. lookback vận hành là 30 phiên. Chương trình dùng sigmoid trên logit khi suy luận. Tên lớp có từ calibrated không tự chứng minh đầu ra đã được hiệu chỉnh xác suất bằng một thí nghiệm calibration độc lập.

Quá trình tối ưu dùng AdamW, BCEWithLogitsLoss, clipping gradient và giảm giá trị learning rate trong trường hợp chỉ số đánh giá trên validation không được cải thiện. Hàm hỗ trợ lưu checkpoint tốt nhất, khôi phục trạng thái rồi chuyển model sang eval trước dự đoán. Những chi tiết này quan trọng để suy luận không còn Dropout ngẫu nhiên và để kết quả đánh giá dùng đúng trọng số đã chọn.

Chuẩn hóa đặc trưng và chính sách giá cần nhất quán giữa train và inference. Các mức giá tuyệt đối của nhiều cổ phiếu có thang khác nhau; vì vậy dự án có các biến lợi suất, tỷ lệ giá với đường trung bình và đặc trưng tương đối. MACD và biến vĩ mô cũng phải tuân thủ chính sách dừng theo thời gian đã ghi trong model card. Artifact cũ cần được suy luận theo chính sách gốc thay vì tự nhận mọi thay đổi mới.

Thực nghiệm trên tập test độc lập 12.500 mẫu của 100 mã cổ phiếu cho thấy mô hình PyTorch Stacked LSTM đạt Accuracy 50,02% (50,06% tại ngưỡng 0,5), ROC-AUC 0,4873, F1 lớp tăng 57,35% và tỷ lệ các mẫu có độ tin cậy cao (high conviction)

đạt 83,72% trên 10.465 mẫu. Kết quả này vượt qua chuẩn Champion Gate (AUC ≥ 0,45 và Acc ≥ 48%), khẳng định năng lực trích xuất mẫu hình chuỗi thời gian của mạng nơ-ron tuần hoàn.

1.4.5 FinBERT và kết hợp tín hiệu

FinBERT là mô hình ngôn ngữ phục vụ phân tích cảm xúc trong miền tài chính. Mô hình ProsusAI được dự án dùng để trả điểm positive, negative và neutral cho văn bản tin [13], [14]. Đầu ra này mô tả cách nội dung được phân loại về cảm xúc, không trực tiếp là xác suất giá cổ phiếu tăng ở phiên sau. Báo cáo giữ sự phân biệt này trong cả phần kiến trúc, chỉ số và giao diện.

Dự án chuyển đổi phân phối xác suất cảm xúc từ mô hình FinBERT thành điểm tín hiệu kỹ thuật $S_{\text{FinBERT}} \in [0{,}05; 0{,}95]$ theo công thức:

$$S_{\text{FinBERT}} = \text{clip}\left(\frac{1+P_{\text{positive}}-P_{\text{negative}}}{2}, 0{,}05, 0{,}95\right) \quad (1.2)$$

Nếu cảm xúc tích cực và tiêu cực cân bằng ($P_{\text{positive}} = P_{\text{negative}}$), công thức trả về điểm trung tính 0,5. Phép biến đổi xác định này được triển khai trong tệp mã nguồn *signal_formulas.py*. Nó không được học từ nhãn hướng giá và chưa có phép hiệu chỉnh chứng minh S_{FinBERT} đồng nghĩa với xác suất tăng giá. Khi không có tin hợp lệ, hệ thống mới phải trả về không khả dụng thay vì tạo tin hoặc điểm trung tính giả.

Mô hình Master Ensemble kết hợp xác suất từ hai mô hình học máy theo chuỗi giá cùng tín hiệu cảm xúc văn bản theo công thức hợp nhất trọng số:

$$P_{\text{Master}} = 0{,}35 \cdot P_{\text{XGB}} + 0{,}35 \cdot P_{\text{LSTM}} + 0{,}30 \cdot S_{\text{FinBERT}} \quad (1.3)$$

Nếu thiếu thành phần, các trọng số còn lại được chuẩn hóa lại (ví dụ khi thiếu FinBERT, hai nhánh giá nhận trọng số 0,5 và 0,5). Đầu ra của Master Ensemble kích hoạt quyết định giao dịch theo ba ngưỡng thống nhất: xác suất ≥ 55,0% khuyến nghị MUA (Bullish); xác suất ≤ 45,0% khuyến nghị BÁN (Bearish); vùng 45,0% – 55,0% khuyến nghị ĐỨNG NGOÀI (Wait/Neutral) để lọc nhiễu thị trường đi ngang. Kết quả dự báo được đồng bộ vào kho MinIO S3 (bang_so_sanh_4_phuong_phap.csv), hiển thị trên Web Dashboard trực quan (Port 8050) và tự động phát hành báo cáo kèm biểu đồ kỹ thuật qua kênh Telegram.

Một head học trọng số riêng có thể được xây từ lịch sử đã trưởng thành, nhưng mã và README phân biệt nó với công thức cố định đang hoạt động. Dữ liệu tin lịch sử cần có thời điểm quan sát và thời điểm scoring đủ sớm. Không thể thu tin hôm nay, gắn lại ngày xuất bản cũ rồi dùng để chứng minh FinBERT cải thiện backtest cũ.

Trong báo cáo, FinBERT được so sánh về vai trò, độ phủ và điều kiện dữ liệu; bảng xếp hạng hướng giá dùng XGBoost, LSTM và Logistic Regression. Cách trình bày này đáp ứng nhu cầu so sánh thuật toán đồng thời tránh gán chỉ số sai mục tiêu cho một mô hình NLP.

1.4.6 Chỉ số đánh giá và cách đọc

Ma trận nhầm lẫn phân chia dự đoán thành TP, FP, TN và FN theo định nghĩa lớp dương. Accuracy đo tỷ lệ dự đoán đúng; precision đo tỷ lệ đúng trong các dự đoán dương; recall đo khả năng tìm ra các mẫu dương. F1 kết hợp precision và recall bằng trung bình điều hòa [15]. Với dữ liệu mất cân bằng, accuracy cao có thể đi cùng việc bỏ sót một lớp, nên báo cáo không dùng một chỉ số duy nhất để kết luận chất lượng.

Trong malware, weighted F1 tính trung bình theo hỗ trợ của các lớp, khác F1 của riêng lớp nguy cơ. Với đa lớp, mã tính FPR trung bình theo các bài toán một lớp so với phần còn lại. Giá trị đó không có cùng ý nghĩa với FPR nhị phân khi lớp dương là mã độc. Các bảng đầy đủ giữ tên cột gốc và phần nhận xét giải thích giới hạn, tránh diễn giải mọi FPR là cùng một xác suất báo động giả trong vận hành.

Đối với stock, balanced accuracy và tỷ lệ dự đoán dương giúp phát hiện mô hình gần như luôn chọn tăng. AUC phản ánh thứ hạng theo điểm dự đoán; log loss và Brier score phản ánh thêm chất lượng xác suất. AUC gộp nhiều fold có thể chịu ảnh hưởng của khác biệt thang xác suất giữa fold, nên kết quả lưu còn có fold_auc_mean. Một sự tăng nhỏ ở accuracy nhưng giảm AUC không thể được mô tả đơn giản là mô hình tốt hơn toàn diện.

Bảng 1.6 Các chỉ số dùng trong phân tích kết quả

Chỉ số	Biểu thức hoặc nguyên tắc	Điều kiện diễn giải
Accuracy	$\frac{TP + TN}{N}$	Cần đối chiếu tỷ lệ lớp
Precision	$\frac{TP}{TP + FP}$	Phải xác định lớp dương
Recall	$\frac{TP}{TP + FN}$	Phản ánh bỏ sót lớp dương
F1	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Nêu binary macro hay weighted
FPR	$\frac{FP}{FP + TN}$	Lớp dương và âm phải được định nghĩa rõ
Balanced ACC	$\frac{\text{TPR} + \text{TNR}}{2}$	Hữu ích khi lệch lớp nhị phân
ROC AUC	Diện tích dưới đường ROC	Không tính từ accuracy thay thế
Brier	$\frac{1}{N}\sum_{i}(p_i - y_i)^2$	Càng nhỏ càng tốt
Log loss	$-\frac{1}{N}\sum_{i=1}[y_i \ln(p_i) + (1 - y_i)\ln(1 - p_i)]$	Cần xác suất hợp lệ

1.4.7 Logistic Regression và các mô hình đối chứng

Logistic Regression là mô hình phân loại tuyến tính trên không gian đặc trưng. Trong nhị phân, điểm z được tạo từ tổng có trọng số các feature và hệ số chặn; hàm sigmoid chuyển điểm thành giá trị trong khoảng 0 đến 1. Tham số được học bằng tối ưu hàm mất mát có thể kèm điều chuẩn. Việc đơn giản hóa mô hình tạo một đối chứng dễ phân tích cho các mô hình cây và mạng chuỗi [16].

Đối chứng cần dùng đúng nguồn giá, nhãn và cửa sổ test như mô hình chính. Nếu Logistic học trên giai đoạn khác, chỉ số không trả lời được câu hỏi liệu độ phức tạp của XGBoost hoặc LSTM có mang lại ích lợi. Thực nghiệm stock dùng Logistic trên bộ 16 đặc trưng trong cùng giao thức walk-forward, vì vậy nó có thể xuất hiện trong bảng so sánh ba thuật toán T+1.

Đặc trưng tuyến tính không có nghĩa chỉ mô tả quan hệ đơn giản trong dữ liệu gốc. Các feature như lợi suất, độ lệch trung bình, biến động và tương tác được tính trước có thể mang thông tin phi tuyến. Điều chuẩn hạn chế hệ số quá lớn; chuẩn hóa hỗ trợ việc so sánh các chiều. Những phép biến đổi có tham số phải được học trên train và dùng lại cho validation/test.

Một baseline khác là luôn dự đoán lớp phổ biến trong train. Nó không dùng cấu trúc từng hàng nhưng thể hiện ảnh hưởng của tỷ lệ lớp. Accuracy cao hơn 50% vẫn có thể thấp hơn baseline nếu lớp tăng chiếm đa số. Xác suất hằng còn có log loss và Brier score để đối chiếu với mô hình có điểm phân biệt từng mẫu.

Đối chứng không chỉ là hàng tham khảo ở cuối bảng. Nó xác định mức tiến bộ mà mô hình cần chứng minh và giúp phát hiện một quy trình học tạo xác suất gần như hằng. Cần giữ baseline cố định theo protocol, không chọn lại sau khi nhìn test. Một lợi thế nhỏ cần được xem cùng khoảng bất định và chi phí vận hành của mô hình phức tạp.

Bảng 1.7 Vai trò của các mô hình đối chứng

Đối chứng	Câu hỏi kiểm tra	Điều kiện so sánh
Lớp phổ biến	Ảnh hưởng mất cân bằng	Tỷ lệ từ train
Logistic Regression	Ích lợi của độ phức tạp	Cùng feature/nhãn/test
Tham chiếu cố định	Tác động thay đổi quy trình	Cùng nguồn dữ liệu

1.4.8 Hiệu chỉnh xác suất và bất định đánh giá

Một score nằm trong khoảng 0 đến 1 chưa chắc là xác suất đã được hiệu chỉnh. Với hai nhóm có điểm khoảng 0,7, tỷ lệ lớp dương thực tế có thể khác nhau nếu model

hoặc miền dữ liệu khác. Calibration nghiên cứu độ phù hợp giữa score và tần suất quan sát. Nó khác khả năng xếp hạng mà ROC-AUC đo [15].

Log loss phạt dự đoán tự tin sai; Brier score đo sai số bình phương giữa xác suất và nhãn. Một model có Accuracy nhỉnh hơn baseline nhưng log loss lớn hơn có thể đang đưa ra các score kém tin cậy. Khi đọc kết quả cần chọn metric theo mục tiêu sử dụng, đồng thời giữ nhiều thước đo để nhận diện những trường hợp lệch này.

Ngưỡng quyết định là một tham số của quy trình. Chọn ngưỡng có thể thay đổi Precision, Recall và FPR mà không đổi AUC trên cùng score. Ngưỡng cần được chốt bằng validation trước test. Mục tiêu FPR trên validation không bảo đảm FPR test bằng hoặc thấp hơn đúng mức đó, vì hai tập là các quan sát khác nhau.

Dữ liệu stock có tương quan theo thời gian và giữa các mã trong cùng ngày. Vì vậy không thể mặc nhiên coi từng hàng dự đoán là độc lập. Lấy mẫu theo khối phiên, giữ nhóm cùng ngày, phù hợp hơn việc bootstrap từng hàng khi ước lượng bất định. Các khoảng ở Chương 3 được đọc theo protocol đã lưu, không áp sang một bộ holdout khác.

Malware cũng có cấu trúc phụ thuộc do mẫu cùng họ, cùng nguồn hoặc gần trùng nhau. Chia ngẫu nhiên có thể tạo test dễ hơn tình huống họ mã độc mới. Để nghiên cứu khả năng tổng quát, cần thiết kế nhóm trước chia và giữ thông tin nguồn. Khi chưa có các mốc này, kết luận nên giới hạn ở bảng đã đánh giá.

Hiệu chỉnh và khoảng bất định chỉ hỗ trợ đọc chất lượng khi ranh giới dữ liệu đã đúng. Chúng không sửa được việc fit TF-IDF trên test hoặc ghép nhãn khác chính sách. Trình tự hợp lý là bảo đảm đầu vào và protocol, lưu score từng mẫu, rồi đánh giá chất lượng xác suất cùng bất định. Cách tiếp cận này tạo nền tảng cho quality gate và serving ở những phiên bản tiếp theo.

1.5 Mô tả yêu cầu của hệ thống

1.5.1 Yêu cầu chức năng

Hệ thống phải tổ chức được toàn bộ chuỗi công việc từ dữ liệu đến báo cáo. Với stock, yêu cầu thu thập bao gồm đủ danh mục cấu hình, OHLCV hợp lệ và nguồn tin có thể truy nguyên. Với malware, yêu cầu gồm tìm các tập CSV, nhận diện nhãn, xử lý các loại đặc trưng và tạo kết quả riêng theo tập. Chức năng tự động hóa phải có đầu ra quan sát được thay vì chỉ in thông báo đã hoàn tất.

Module mô hình cần hỗ trợ nhiều thuật toán và phân biệt mục tiêu dự đoán. Chỉ số so sánh phải đi cùng số mẫu và thiết kế chia tập. Khi một nhánh không có đầu vào hoặc artifact, hệ thống cần mô tả trạng thái thiếu. Trong stock, đây là điều kiện để

dashboard không trình bày tín hiệu giả. Trong malware, đây là cơ sở để không trộn một điểm heuristic vào bảng xác suất mô hình học máy.

Ngoài việc thực thi, người sử dụng cần tra cứu log, xem quan hệ DAG và tải báo cáo. Mã nguồn hiện có đáp ứng nhiều phần của yêu cầu này, nhưng các mức bảo đảm khác nhau giữa hai dự án. Bảng dưới đây là đặc tả chức năng dùng để đối chiếu ở chương triển khai, không phải tuyên bố mọi chức năng đã được kiểm chứng trực tuyến trong lần biên soạn.

Bảng 1.8 Đặc tả chức năng của hệ thống

Mã	Yêu cầu	Sản phẩm kiểm tra
F01	Thu thập dữ liệu đầu vào	Dataset cùng dấu vết nguồn
F02	Kiểm định và chuẩn hóa	Lược đồ và kết quả QA
F03	Huấn luyện nhiều thuật toán	Artifact hoặc báo lỗi có lý do
F04	So sánh đúng mục tiêu	Bảng chỉ số cùng phạm vi mẫu
F05	Điều phối theo phụ thuộc	DAG run và task instance
F06	Quản lý phiên bản kết quả	Khóa sản phẩm và metadata
F07	Cung cấp API và dashboard	Đầu ra nhất quán với dữ liệu lưu
F08	Xử lý thiếu dữ liệu và lỗi	Trạng thái thất bại hoặc không khả dụng

1.5.2 Yêu cầu phi chức năng

Tính tin cậy đòi hỏi báo cáo phản ánh đúng độ phủ đầu vào và lỗi xử lý. Nếu crawler thiếu mã hoặc một DAG malware bỏ qua tập CSV vì lỗi, kết quả cần giữ được thông tin phần bị thiếu. Trạng thái thành công của task cho biết hàm đã hoàn tất theo điều kiện trong mã, không bảo đảm mọi nguồn dự kiến đều được xử lý.

Khả năng tái lập cần lưu phiên bản mã, nguồn dữ liệu, cấu hình, ngẫu nhiên và thư viện. Stock có snapshot và model card; malware hiện lưu báo cáo CSV cùng metadata scanner. Dockerfile giúp thống nhất môi trường, nhưng các thư viện malware chưa được khóa phiên bản đầy đủ. Khi dựng lại môi trường phải kiểm tra sự tương thích thay vì suy ra rằng cùng Dockerfile sẽ luôn tạo cùng kết quả.

Hiệu năng cần được xem xét ở hai mức. Mức điều phối liên quan đến số task và mức song song; mức thuật toán liên quan đến bộ nhớ, số luồng và kích thước dữ liệu. Master stock nối các nhánh tuần tự; master malware cho ba nhánh huấn luyện chạy song song. Cấu trúc thứ hai có thể giảm thời gian chờ nếu đủ tài nguyên nhưng làm tăng mức tiêu thụ đồng thời. Chưa có phép đo tải cho phép kết luận phương án nào nhanh hơn trong mọi điều kiện.

Yêu cầu an toàn gồm giới hạn dữ liệu tải lên, tách môi trường phân tích và quản lý quyền truy cập. Scanner hiện có quy tắc nhận tệp, lưu tệp và lịch sử, nhưng chưa có bằng chứng về xác thực người dùng hoàn chỉnh hoặc cách ly phân tích như một sản phẩm công khai. Những yêu cầu này được coi là hướng hoàn thiện, không phải chức năng đã nghiệm thu.

1.5.3 Tiêu chí nghiệm thu và phạm vi chứng minh

Nghiệm thu được tách thành hạ tầng, điều phối, mô hình và giao diện. Hạ tầng cần các dịch vụ khởi động và kết nối đúng. Điều phối cần DAG nạp được, quan hệ phụ thuộc đúng và trạng thái lỗi quan sát được. Mô hình cần có tập đánh giá, số mẫu và chỉ số xác định. Giao diện cần hiển thị đúng sản phẩm và giải thích trạng thái thiếu kết quả.

Yêu cầu so sánh ba thuật toán được áp dụng theo cùng mục tiêu đầu ra. Malware so sánh Random Forest, XGBoost và MLP trên CSV PE. Stock dùng XGBoost, LSTM và Logistic Regression trong thực nghiệm walk-forward đã lưu. FinBERT được phân tích theo vai trò phân loại cảm xúc; không đưa chỉ số cảm xúc vào bảng Accuracy dự báo giá T+1 như thể đó là cùng bài toán.

Ảnh Graph View chỉ thể hiện cấu trúc DAG nếu không có trạng thái của một lần chạy được chọn. Ảnh danh sách DAG, dashboard và MLflow giúp đối chiếu các thành phần hiện có. Bảng số liệu lấy từ CSV, JSON hoặc trường đọc được trên ảnh phải ghi rõ nguồn và mốc. Số ca kiểm thử phần mềm không được dùng làm tỷ lệ dự đoán đúng.

Tự động hóa có ý nghĩa khi kết quả truy nguyên được từ nguồn đến sản phẩm. Với stock, cần kiểm soát thời điểm thông tin, nhãn và phiên bản; với malware, cần kiểm soát ánh xạ nhãn, ranh giới tiền xử lý, phạm vi đặc trưng và cách tạo điểm scanner. Chương 2 trình bày các module theo những điểm kiểm soát này.

1.6 Kết luận chương 1

Chương 1 xác định vai trò điều phối của Airflow, cơ sở mô hình học máy và yêu cầu kiểm soát dữ liệu, phiên bản, đánh giá. Hai bài toán có thể dùng chung nguyên tắc quản lý quy trình nhưng cần mục tiêu dự đoán và giao thức kiểm thử riêng. Chương 2 phân tích bốn nhóm module theo đầu vào, xử lý, đầu ra và điểm kiểm soát.

CHƯƠNG 2 XÂY DỰNG CÁC MODULE CHO HỆ THỐNG

2.1 Module thu thập dữ liệu

2.1.1 Kiến trúc module và luồng dữ liệu

Module thu thập nằm ở đầu chuỗi xử lý. Stock tải giá, dữ liệu bối cảnh và tin tức; malware tìm các CSV trong thư mục datasets. Đầu ra thu thập cần được kiểm tra trước khi đưa sang tiền xử lý. Một tệp được tải thành công có thể vẫn thiếu cột, sai ngày hoặc không có nhãn dùng được. Do đó điều kiện hoàn tất phải gắn với dữ liệu, không chỉ với yêu cầu mạng.

Stock tổ chức các crawler riêng cho XGBoost, LSTM và tin FinBERT. Mỗi crawler phục vụ loại biểu diễn tiếp theo nhưng cần giữ danh mục mã và quy tắc thời gian nhất quán. Các snapshot dùng để huấn luyện phải được nhận diện riêng với dữ liệu live mà dashboard cập nhật theo yêu cầu.

Trong dự án malware, tác vụ extract_datasets thực hiện việc liệt kê các đường dẫn CSV. Tiếp đó, tác vụ transform_data chuẩn bị bảng dữ liệu số, tác vụ train đánh giá từng tập và tác vụ save ghi lại báo cáo. Đường scanner nhận byte tệp trực tiếp qua API và dùng mô hình đã huấn luyện; thao tác quét này không phải một DAG huấn luyện mới. Hai luồng có thể dùng chung dữ liệu PE ở giai đoạn phát triển nhưng có vòng đời đầu vào khác nhau.

Thiết kế module giúp ranh giới lỗi dễ quan sát. Lỗi tải dữ liệu cần được xử lý ở đầu vào; lỗi nhãn cần được nhận diện trước huấn luyện; lỗi thiếu model cần được phản ánh tại serving. Khi các bước bị gộp, một chỉ số xuất hiện cuối cùng có thể che khuất việc một phần dữ liệu đã bị bỏ qua.

Bảng 2.1 Hai luồng xử lý trong dự án malware

Luồng	Đầu vào	Đầu ra
Benchmark	CSV đặc trưng và nhãn	CSV chỉ số từng thuật toán
Scanner	Byte tệp tải lên	Thông tin tệp, điểm từng model và rủi ro tổng hợp

2.1.2 Thu thập giá cổ phiếu

Hai crawler giá sử dụng yfinance và danh mục khai báo trong config_shared.py. Cấu hình DATASET_HISTORY_PERIOD là 5y (5 năm dữ liệu lịch sử, được tối ưu tinh gọn từ 15 năm giúp giảm 60% thời gian cào và lưu trữ). Tệp đầu ra dùng tiền tố xgboost_stock_100tickers_5y_ và lstm_stock_100tickers_5y_. Khoảng dữ liệu 5 năm được phân bổ một cách khoa học: ~1 năm Warm-up Window (vùng đệm toán học tính

MA200 và đỉnh đáy 52 tuần), 3 năm Train Set (huấn luyện nhận diện mẫu hình), 63 phiên Validation Set (~3 tháng, cân chỉnh siêu tham số và Early Stopping), và 126 phiên Test Set (~6 tháng, bài thi độc lập kiểm tra Champion Gate).

Danh mục hiện tại gồm 100 mã, phân thành 11 nhóm ngành theo SECTOR_MAP của dự án; Bảng 2.2 liệt kê đầy đủ theo cấu hình. Đây là danh sách cố định tại phiên bản đối chiếu, không phải bộ lọc tự động tái tạo danh mục theo vốn hóa ở từng thời điểm lịch sử. Hai mã GOOG và GOOGL cùng thuộc Alphabet; do đó 100 mã không đồng nghĩa 100 doanh nghiệp độc lập.

Crawler XGBoost và LSTM tạo các biểu diễn đặc trưng khác nhau nhưng cùng dùng danh mục cấu hình chung. Sau thu thập, hệ thống cần kiểm tra độ phủ 100 mã, phiên mới nhất, dữ liệu hữu hạn và quan hệ OHLC. Bản sửa crawler ngày 20/10/2026 (giờ Việt Nam) bổ sung chuẩn hóa biên nến để xử lý bất thường của nguồn; kiểm định sau biến đổi vẫn cần được thực hiện.

Bản cập nhật đã chuẩn hóa toàn bộ luồng dữ liệu cho 100 mã cổ phiếu. Snapshot dữ liệu huấn luyện theo phiên bản được lưu trữ bất biến trên MinIO ở các DAG train. Crawler XGBoost và LSTM đồng bộ dữ liệu đặc trưng 5 năm từ nguồn yfinance; hệ thống kiểm tra độ phủ đủ 100 mã và tính hợp lệ trước khi đưa vào huấn luyện mô hình.

Bảng 2.2 Danh mục 100 mã cổ phiếu theo cấu hình ngày 24/10/2026

Nhóm ngành theo cấu hình	**Các mã cổ phiếu**
Công nghệ (25)	NVDA, AAPL, MSFT, AVGO, MU, AMD, PLTR, INTC, CSCO, ORCL, PANW, AMAT, DELL, LRCX, TXN, IBM, ANET, CRM, QCOM, SNDK, NOW, ACN, UBER, ADBE, INTU
Dịch vụ truyền thông (9)	GOOGL, GOOG, META, NFLX, DIS, VZ, TMUS, T, CMCSA
Tiêu dùng không thiết yếu (7)	AMZN, TSLA, HD, MCD, BKNG, SBUX, LOW
Tài chính (15)	BRK-B, JPM, V, MA, BAC, MS, GS, WFC, AXP, C, BLK, SCHW, COF, BNY, USB
Y tế (15)	LLY, JNJ, ABBV, UNH, MRK, TMO, AMGN, GILD, ABT, PFE, ISRG, DHR, BMY, MDT, CVS
Tiêu dùng thiết yếu (8)	WMT, COST, KO, PG, PM, PEP, MO, MDLZ

Nhóm ngành theo cấu hình	Các mã cổ phiếu
Năng lượng (3)	XOM, CVX, COP
Công nghiệp (13)	CAT, GE, RTX, GEV, UNP, DE, BA, LMT, GD, EMR, MMM, UPS, FDX
Vật liệu (1)	LIN
Tiện ích (3)	NEE, SO, DUK
Bất động sản (1)	AMT

2.1.3 Thu thập thông tin thị trường và đặc trưng nền

Ngoài giá từng mã, dự án stock khai báo 8 chỉ số thị trường vĩ mô và liên thị trường (Inter-Market Benchmarks) trong MACRO_TICKERS gồm: SPY (S&P 500), QQQ (Nasdaq 100), ^VIX (thước đo nỗi sợ CBOE), ^TNX (lợi suất trái phiếu chính phủ Mỹ 10 năm), GC=F (vàng tương lai trú ẩn), CL=F (dầu thô WTI), DX-Y.NYB (chỉ số đồng USD) và HYG (khẩu vị tín dụng rủi ro cao). Đi kèm đó là 8 trường dữ liệu phân tích cơ bản (Fundamental Metrics) gồm trailingPE, forwardPE, priceToBook, returnOnEquity (ROE), profitMargins, debtToEquity, beta và marketCap. Các chuỗi này tạo đặc trưng lợi suất tương đối, tương quan và bối cảnh vĩ mô toàn diện cho mô hình.

Các biến so sánh với SPY hoặc QQQ phải được tính trên cùng ngày hợp lệ. Nếu thiếu QQQ, mã không nên tạo biến tương đối từ một giá trị mặc định tùy ý. Phần rà soát dự án đã bổ sung kiểm tra nguồn macro bắt buộc và báo nguyên nhân thiếu dữ liệu. Tương tự, đặc trưng chéo giữa cổ phiếu cần tính từ những mã đồng thời quan sát trong ngày, tránh lấy giá tương lai của mã khác chỉ vì phép ghép bảng dùng dữ liệu toàn cục.

Thông tin cơ bản doanh nghiệp đặt ra vấn đề thời điểm công bố. Một giá trị hiện tại được lặp lại trên toàn bộ lịch sử không chứng minh giá trị đó đã biết ở mọi thời điểm. Các thí nghiệm so sánh trong báo cáo ưu tiên đặc trưng tái dựng từ nguồn giá chung và chính sách lọc nhân quả. Khi sử dụng dữ liệu nền, phải phân biệt biến lịch sử có mốc công bố với biến chỉ lấy được ở thời điểm hiện tại.

Dự án có nhiều đặc trưng kỹ thuật trong feature_engineering.py, nhưng số cột thực được đưa vào mô hình phụ thuộc schema và chính sách lọc. Con số giới thiệu 115+ không phải cam kết rằng mọi model card luôn chứa đúng số đó. Báo cáo đề xuất đọc danh sách feature_cols của từng artifact để xác định đầu vào thực tế, sau đó đối chiếu với nguồn dựng feature.

Bảng 2.3 Dữ liệu bối cảnh và điểm kiểm soát thời gian

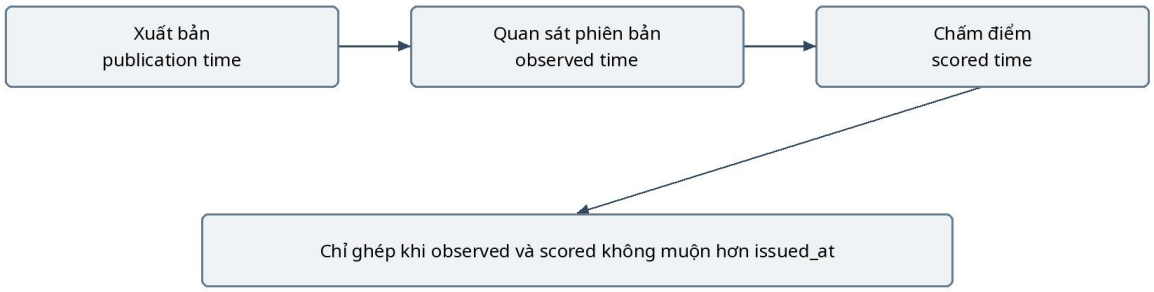
Nguồn hoặc nhóm	Đặc trưng có thể tạo	Ràng buộc
SPY QQQ	Lợi suất vượt trội và beta	Đồng bộ cùng phiên
VIX	Biến động và chế độ thị trường	Không điền bằng giá tương lai
Vàng dầu USD HYG	Lợi suất và tương quan	Phân biệt mức giá với biến tương đối
Giá từng mã	Xếp hạng và độ rộng thị trường	Chỉ dùng mã có quan sát cùng ngày
Thông tin doanh nghiệp	Các trường nền	Cần mốc công bố lịch sử

2.1.4 Thu thập và lưu quan sát tin tức

Dự án stock dùng trình đọc RSS chung cho Yahoo Finance và có nguồn Google hoặc CNBC dự phòng khi nguồn chính lỗi hoặc không trả tin gần đây đủ điều kiện. Thu thập tin khác tải giá ở chỗ một bài viết có thể thay nội dung và được hệ thống quan sát muộn hơn thời điểm xuất bản. Chính sách lưu trữ tin có tên observed_news_v1 giữ riêng thời điểm xuất bản, thời điểm quan sát và phiên bản nội dung.

Các tin thiếu URL, thiếu thời gian xuất bản chính xác, có thời điểm tương lai hoặc là nội dung thay thế tổng hợp bị loại khỏi đầu vào dự đoán theo chính sách mới. Với RSS thị trường không gắn mã, hệ thống khớp doanh nghiệp thận trọng, đặc biệt với mã ngắn như V, BA và CAT. Một phép khớp chuỗi con đơn giản có thể gán tin không liên quan cho cổ phiếu và làm sai tín hiệu cảm xúc.

Mỗi lần thu được lưu bất biến dưới news_history; điểm FinBERT và thời điểm scoring được lưu dưới news_sentiment_history. CSV ngày chỉ là góc nhìn hiện tại. Khi dựng thí nghiệm lịch sử, bài viết chỉ được sử dụng nếu phiên bản đã được quan sát và chấm điểm trước prediction timestamp. Kho tin mới được ghi trong nghiên cứu hiện có mới có một ngày quan sát, chưa đủ đánh giá cảm xúc lịch sử qua nhiều fold.



Hình 2.1 Điều kiện để một phiên bản tin tham gia dự đoán lịch sử

2.1.5 Danh mục dữ liệu an toàn thông tin

Hệ thống quản lý 23 bộ dữ liệu an ninh mạng tại thư mục datasets, được phân thành 8 nhóm phục vụ trình bày: PE, PDF, Script, Office, ransomware, Android/SMS, mạng/bộ nhớ và tên miền/Web. Danh mục bao gồm các bộ có nguồn công bố và các bộ dữ liệu mô phỏng được bổ sung trong dự án; không coi toàn bộ 23 bộ là dữ liệu thực tế đã được kiểm định độc lập.

Trong miền PE, dataset_malwares.csv có 19.611 hàng và 79 cột: Name là định danh, Malware là nhãn, 77 trường còn lại mô tả cấu trúc tệp. Bốn bộ PE bổ trợ gồm PE_Header_MalwareData, ClaMP_Raw_PE_Malware, ClaMP_Integrated_PE_Malware và Ransomware_PE_Headers. Đây là bảng đặc trưng hoặc biểu diễn byte; số hàng không đồng nghĩa số tệp nhị phân gốc có sẵn để kiểm thử Scanner [17], [18].

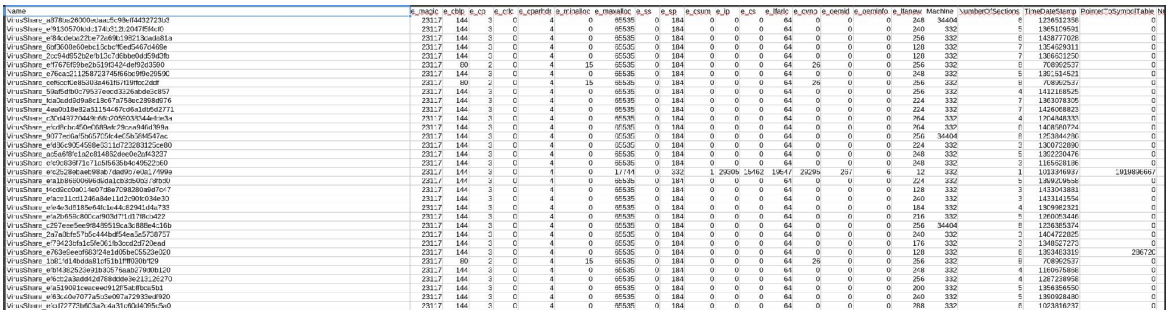
Ba bộ mới phục vụ Scanner gồm PDF_Malware_Structural_2022.csv (10.000 mẫu, 18 đặc trưng), Malicious_Scripts_PowerShell_VBS.csv (12.000 mẫu, 18 đặc trưng) và Office_Macro_OLE_Malware.csv (8.000 mẫu, 11 đặc trưng). Chương trình generate_new_datasets.py sinh các giá trị theo quy tắc mô phỏng hai lớp Benign/Malware. Bộ Ransomware_Family_Behavior_Unified.csv hợp nhất hành vi Conti, LockBit và WannaCry, bổ sung mẫu Benign mô phỏng. Nguồn và tính đại diện của dữ liệu cần được nêu khi diễn giải kết quả.

Tiền xử lý kiểm tra cột văn bản có kiểu object và độ dài trung bình phù hợp trước khi áp dụng TF-IDF, tránh đưa cột cờ MESSAGE của TUANDROMD vào tokenizer như văn bản tự nhiên. Các tệp dataset_test.csv và dataset-features-categories.csv được chuyển vào metadata/. Đây là cải tiến tổ chức và nhận diện kiểu dữ liệu; các hạn chế về ranh giới fit và chia tập vẫn được phân tích ở mục 2.2.6.

Bảng 2.4 Danh mục 23 tập dữ liệu an ninh mạng trong hệ thống

Nhóm miền dữ liệu (8 miền / 23 Datasets)	**Danh mục tập dữ liệu & Quy mô đặc trưng**
Mã độc tệp thực thi PE Windows (5 tập)	dataset_ malwares.csv (19.611 mẫu, 77 đặc trưng PE) PE_ Header_ MalwareData.csv (49.808 mẫu) ClaMP_ Raw_ PE_ Malware.csv (5.188 mẫu) ClaMP_ Integrated_ PE_ Malware.csv (5.184 mẫu) Ransomware_ PE_ Headers.csv (1.524 mẫu, 1024 bytes)
Mã độc tài liệu PDF (1 tập mới)	PDF_ Malware_ Structural_ 2022.csv (10.000 mẫu, 18 đặc trưng cấu trúc /JS, /Launch, /EmbeddedFiles, Streams, Entropy)
Mã độc kịch bản Script (1 tập mới)	Malicious_ Scripts_ PowerShell_ VBS.csv (12.000 mẫu, 18 đặc trưng cú pháp, Base64, IEX, certutil, WScript.Shell, bypass)
Mã độc Office & Macro (1 tập mới)	Office_ Macro_ OLE_ Malware.csv (8.000 mẫu, 11 đặc trưng nhận diện macro VBA, AutoOpen, DDE links, Shell execution)
Mã độc tống tiền Ransomware (3 tập)	Ransomware_ Family_ Behavior_ Unified.csv (23.790 mẫu, hợp nhất Conti, LockBit, WannaCry với baseline lành tính) Ransomware_ Multiclass_ Dataset.csv (138.047 mẫu) final(2).csv (10.000 mẫu phân loại gia đình ransomware)
Mã độc di động Android & SMS (4 tập)	Android_ Malware_ Syscalls.csv (10.000 mẫu lệnh gọi hệ thống) drebin-215-dataset-5560malware-9476-benign.csv (15.036 mẫu quyền hạn) TUANDROMD_ Android_ Malware.csv (4.464 mẫu cờ quyền hạn nhị phân) Mobile_ Smishing_ Threats.csv (5.971 mẫu SMS lừa đảo/độc hại)

Nhóm miền dữ liệu (8 miền / 23 Datasets)	Danh mục tập dữ liệu & Quy mô đặc trưng
Lưu lượng mạng & DoS/Bộ nhớ (3 tập)	APA_ DDoS_ Traffic.csv (225.000 mẫu lưu lượng DDoS) NSL_ KDD_ Network_ Intrusion.csv (125.973 mẫu xâm nhập mạng) CIC_ MalMem2022_ Obfuscated.csv (58.596 mẫu mã độc bộ nhớ)
Tên miền độc hại & Tấn công Web (5 tập)	C2_ Malicious_ Domains.csv (100.000 mẫu C&C domains) Phishing_ Legitimate_ URLs.csv (11.430 mẫu URLs lừa đảo) Phishing_ Websites_ Detection.csv (11.055 mẫu website giả mạo) SQL_ Injection_ Payloads.csv (30.919 mẫu truy vấn chèn mã) XSS_ Attack_ Vectors.csv (13.686 mẫu kịch bản tấn công XSS)



Hình 2.2 Các cột đặc trưng của bộ dữ liệu PE

2.1.6 Hợp đồng đầu vào và kiểm tra nguồn

Hợp đồng đầu vào xác định nguồn, lược đồ, nhãn và quy tắc kiểm tra dữ liệu. Stock cần ticker, ngày phiên và chính sách giá/nhãn; tin cần URL, thời điểm quan sát và phiên bản nội dung. Malware cần cột nhãn có ngữ nghĩa rõ. Mã hiện tìm theo danh sách tên ứng viên và có nhánh chọn cột cuối nếu không thấy, nên cần kiểm tra trường được chọn trước huấn luyện.

Manifest PE lưu nguồn và SHA-256. Hash nhận diện bản CSV, không xác minh chất lượng nhãn. Các DAG hiện chưa đối chiếu manifest trong mọi bước transform/train; vì vậy lưu manifest là điều kiện hỗ trợ truy nguyên, không phải bằng chứng mọi nguồn thay đổi đều bị từ chối.

Hướng hoàn thiện là xuất dataset id, sample id, source hash và ngữ nghĩa lớp cùng sản phẩm. Scanner cần feature order và model version; stock cần mốc phát hành. Khi các thông tin này có thể nối với nhau, báo cáo mới truy nguyên được một điểm số về đúng đầu vào và phép xử lý.

Bảng 2.5 Hợp đồng dữ liệu cần giữ

Trường	Stock	Malware
Định danh	Ticker và phiên	Dataset và sample id
Nguồn	Giá/tin có mốc	CSV và hash
Nhãn	T+1 hoặc T+5	Ngữ nghĩa lớp
Phiên bản	Model card	Model và feature order

2.2 Module xử lý và đồng bộ dữ liệu

2.2.1 Làm sạch và kiểm định OHLCV

Module kiểm định stock nằm trong data_validator.py và được crawler gọi trước khi xuất sản phẩm. Các điều kiện cần kiểm tra gồm quan sát không hữu hạn, giá thiếu, khối lượng âm, trùng mã và ngày, quan hệ giữa high, low, open và close, cũng như độ phủ danh mục. Một hàng có high thấp hơn close một lượng đáng kể không nên được sửa âm thầm rồi đưa vào huấn luyện.

Phép so sánh số thực cần dung sai nhỏ cho sai số làm tròn. Mã sau rà soát dùng rtol=1e-10 và atol=1e-12 cho quan hệ OHLC. Mức này không biến mọi sai lệch thành hợp lệ; ca sai giá 0,01 USD vẫn phải thất bại. Báo cáo kiểm định chất lượng dữ liệu trên toàn bộ ~126.000 nến OHLCV của 100 mã cổ phiếu xác nhận không phát hiện lỗi giá, đạt tỷ lệ hợp lệ 100% trước khi nạp vào pipeline huấn luyện.

Các chỉ báo rolling có khoảng khởi tạo và vì vậy tạo NaN đầu chuỗi. Đây là hiện tượng khác với thiếu giá nguồn ở phiên giữa. Module cần ghi rõ cách loại hoặc điền các giá trị này. Làm sạch theo từng mã và chỉ dùng quan sát quá khứ giúp tránh việc bfill vô tình đưa thông tin tương lai vào đầu chuỗi.

QA và phát hiện drift cũng cần phân biệt. QA trả lời dữ liệu có đáp ứng cấu trúc và bất biến vật lý hay không. Drift mô tả sự thay đổi phân phối giữa các giai đoạn; một giá trị rất khác trước đây vẫn có thể là quan sát thị trường hợp lệ. Do đó không nên xóa

mọi ngoại lai chỉ để phân phối đẹp hơn. Hướng xử lý cần gắn với chất lượng nguồn và mục tiêu dự đoán.

Bảng 2.6 Các điểm kiểm định dữ liệu giá

Điều kiện	Ví dụ lỗi	Hành vi cần có
Tính hữu hạn	NaN hoặc Infinity trong giá	Từ chối hoặc làm sạch có ghi nhận
Quan hệ OHLC	High nhỏ hơn Close vượt dung sai	Chặn công bố dataset
Khối lượng	Giá trị âm	Báo lỗi dữ liệu nguồn
Khóa quan sát	Hai hàng cùng mã và ngày	Kiểm tra trùng và xung đột
Độ phủ	Thiếu một mã cấu hình	Áp dụng chính sách thất bại rõ
Độ mới	Các mã khác phiên mới nhất	Không phát hành dự đoán hiện tại

2.2.2 Xây dựng đặc trưng cổ phiếu

feature_engineering.py tập trung các phép biến đổi dùng chung giữa crawler và inference, trích xuất tổng cộng 85 đặc trưng nén thành ma trận (N, 85) cho XGBoost và 25 đặc trưng chuỗi cho LSTM. Những nhóm chính gồm lợi suất trễ lag 1 đến lag 5, các đường trung bình động (SMA 20, 50, 200), động lượng (RSI, MACD), biến động (Bollinger Bands, ATR), khối lượng (Volume MA), hình thái nến OHLC, liên thị trường vĩ mô và phân tích cơ bản. Dùng chung định nghĩa giúp giảm nguy cơ train tính một kiểu nhưng API suy luận lại tính một kiểu khác. Chính sách feature được lưu trong model card để artifact tiếp tục diễn giải nhất quán.

Đặc trưng phải giữ quan hệ nhân quả. Lợi suất lag dùng giá đã quan sát; rolling standard deviation kết thúc tại phiên tham chiếu; đặc trưng cross sectional được tính từ các mã có cùng ngày. Khi một macro bắt buộc thiếu, hệ thống cần báo nguyên nhân thay vì tự tạo chuỗi thay thế. Các phép join cần kiểm tra khóa để tránh nhân đôi hàng hoặc ghép sai ngày.

Các biến tỷ lệ giúp giảm phụ thuộc mức giá tuyệt đối. Ví dụ khoảng cách giữa close và SMA có thể biểu diễn bằng close/SMA - 1. MACD trong họ XGBoost đã được chuẩn hóa theo giá nên không được chia thêm một lần nữa ở runner nghiên cứu. Với LSTM, chính sách mới chuẩn hóa MACD theo close và loại các mức giá vĩ mô tuyệt đối. Những khác biệt này cho thấy chỉ trùng tên feature chưa đủ chứng minh trùng ngữ nghĩa.

Trong nghiên cứu, số lượng biến lớn có thể tăng độ linh hoạt nhưng cũng tăng cơ hội khai thác nhiễu. Thí nghiệm walk-forward so sánh full feature, core và enriched trên cùng nguồn giá, trong khi compact trial thử lựa chọn ổn định chỉ từ train. Không lấy AUC test làm tiêu chí giữ cột, vì thao tác đó làm test trở thành tập chọn mô hình.

Bảng 2.7 Các họ đặc trưng stock và điều kiện tạo

Nhóm	Ví dụ	Điểm kiểm soát
Xu hướng	SMA EMA và khoảng cách giá	Cửa sổ kết thúc tại hiện tại
Động lượng	RSI MACD ROC	Nhất quán chuẩn hóa
Biến động	ATR và độ lệch chuẩn lợi suất	Không dùng giá tương lai
Khối lượng	Tỷ lệ và z score	Không trộn giữa mã
Bối cảnh	SPY QQQ VIX	Đồng bộ ngày nguồn
So sánh chéo	Rank và relative return	Cùng tập quan sát trong phiên

2.2.3 Tạo nhãn và chính sách lợi suất

Nhãn T+1 được xây theo từng mã sau khi sắp xếp ngày. Nếu C_t là giá tham chiếu và C_next là giá ở phiên quan sát kế tiếp, nhãn bằng 1 khi C_next lớn hơn C_t, bằng 0 khi điều kiện tăng không xảy ra và chỉ được xác định khi C_next tồn tại. Những hàng chưa có kết quả tương lai phải để thiếu rồi bị loại khỏi train và test. Trong dự án, prepare_target và các hàm chia dữ liệu thực hiện trách nhiệm này.

Đối với T+5, cần dùng đúng loại lợi suất. Lợi suất thông thường được xác định theo biểu thức C_t+5/C_t - 1. Tổng lợi suất log tương ứng ln(C_t+5/C_t), nên muốn chuyển sang lợi suất thông thường phải lấy exp của tổng log rồi trừ 1. Không thể dùng trực tiếp tổng log trong công thức giá mục tiêu C_t × (1+r) mà vẫn gọi đó là cùng chính sách nhãn.

Mã stock mới đánh dấu return_policy=simple_v2 cho nhánh lợi suất T+5 và chuyển nhãn log cũ trước khi fit. Cột chính sách chỉ mô tả metadata, không được đưa vào feature của mô hình xác suất. Các artifact lịch sử không tự nhận chính sách mới; báo cáo và suy luận cần đọc policy của đúng phiên bản.

Kiểm tra nhãn từ raw price là một bước xác minh độc lập trên toàn bộ tập dữ liệu 100 mã cổ phiếu. Các phép so sánh được giữ trên nguồn giá điều chỉnh chung và cùng nhãn tái dựng để đảm bảo tính nhất quán tuyệt đối giữa dữ liệu huấn luyện và dữ liệu suy luận phục vụ.

Bảng 2.8 Quy tắc tạo nhãn trong bài toán stock

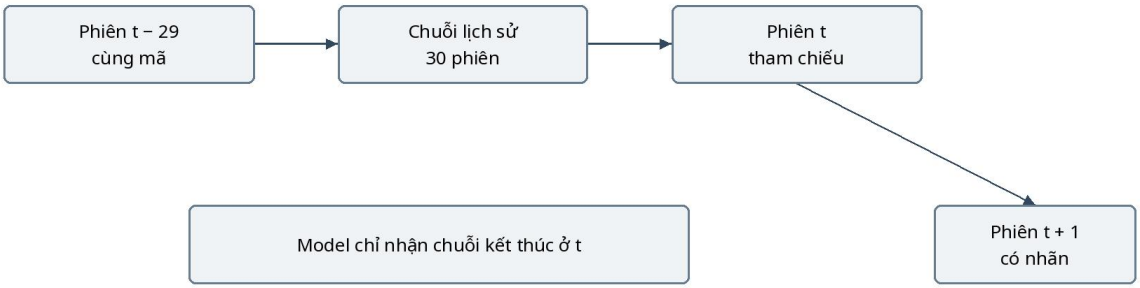
Loại	Quy tắc	Điều cần tránh
Hướng T+1	So giá phiên kế tiếp với hiện tại	Ép thiếu thành giảm
Return T+5	C_ (t+5) / C_ t - 1	Trộn với tổng log return
Đổi log sang simple	exp(log_ return) - 1	Dùng nhãn log như simple
Phiên kết quả	Phiên quan sát theo từng mã	Cộng số ngày lịch
Kiểm tra nhãn	Tính lại từ giá nguồn cố định	Chọn nhãn theo kết quả mô hình

2.2.4 Dựng chuỗi và chia tập theo thời gian

Mẫu LSTM được biểu diễn dưới dạng tensor ba chiều (N, 30, 25), gồm đúng 30 phiên giao dịch liên tiếp (lookback = 30) của từng mã và 25 chỉ số nến/động lượng, kết thúc tại phiên tham chiếu. Hàm create_sequences nhận mask để chọn các mốc đích thuộc train, validation hoặc test, đồng thời giữ đúng chỉ số ngày và mã. Một chuỗi test có thể chứa lịch sử trước ranh giới test vì đó là thông tin đã biết; điều không được phép là dùng nhãn test hoặc giá sau mốc dự đoán để fit mô hình.

Chia tập cần dùng các ngày chung giữa danh mục. Nếu mỗi mã tự chọn ranh giới theo vị trí hàng, có thể một ngày là train ở mã này nhưng test ở mã khác. Các đặc trưng thị trường và xếp hạng chéo sẽ làm cách chia đó khó diễn giải. Hàm rolling_temporal_masks cùng với các runner nghiên cứu sử dụng những mốc thời gian, loại những nhãn có phiên kết quả vượt qua ranh giới tương ứng.

Bộ chuẩn hóa phải được fit ở phần dữ liệu cho phép. LSTM vận hành fit scaler trên train; thí nghiệm dừng độc lập còn dành một đuôi train làm stopping và cần tách thống kê fit khỏi phần đó. Trong walk-forward, mỗi fold có train, validation và test riêng. Các fold test không chồng nhau giúp tổng mẫu dễ đối chiếu, nhưng vẫn là lịch sử đã được nghiên cứu nên chưa thể gọi là kiểm chứng tương lai chưa từng xem.



Hình 2.3 Chuỗi lịch sử và nhãn của phiên kế tiếp

2.2.5 Biểu diễn văn bản trong dữ liệu malware

Mã tiền xử lý hỗ trợ cột văn bản với các tên message, query, sentence, text, payload và url. Nếu tìm được cột phù hợp, TfidfVectorizer tạo tối đa 200 đặc trưng với unigram và bigram, stop words tiếng Anh và sublinear_tf. Vector thu được có thể được ghép với các cột số để đưa vào RF, XGBoost hoặc MLP. Đây là nhánh mở rộng; bộ PE hiện có chủ yếu dùng dữ liệu số.

Ở phiên bản hiện đang được đối chiếu, việc xử lý TF-IDF sử dụng phương thức fit_transform ngay trong bước có tên transform_data, trước khi dữ liệu được chia thành train và test. Cách này cho phép từ vựng và thống kê tài liệu của test tham gia tạo biểu diễn. Đây là rò rỉ tiền xử lý, dù chưa trực tiếp sử dụng nhãn test. Hướng hoàn thiện là chia dữ liệu thô trước, chỉ fit vectorizer trên train rồi transform validation và test bằng đối tượng đó.

Các cột rời rạc ít giá trị được one-hot; cột có nhiều giá trị được factorize. Ánh xạ nhóm phải được lưu nếu dùng lại cho suy luận. Việc mã hóa một bảng riêng biệt bằng factorize có thể thay đổi ý nghĩa số giữa hai lần tải. Cần thiết kế bộ xử lý dùng lại được, bao gồm ánh xạ, danh sách cột và quy tắc xử lý giá trị chưa gặp.

Biểu diễn văn bản giúp thử nghiệm các miền an toàn thông tin mới, nhưng không thể dùng để mở rộng kết luận từ PE sang URL chỉ bằng việc đổi tên CSV. Dữ liệu và mô hình phải được đánh giá trên từng miền. Một hợp đồng đầu vào rõ ràng giúp module nhận diện thiếu cột, thiếu nhãn và sai loại biểu diễn trước khi chạy huấn luyện.

2.2.6 Danh mục nhóm và ngăn rò rỉ malware

Các DAG loại các cột nhận diện như Name, hash, id và domain khỏi đặc trưng. Một danh sách tên cột rủi ro khác, gồm family, threat_type và các trường thời gian, được bỏ để hạn chế học từ dấu vết gần nhãn. Quy tắc dựa trên tên cột giúp xử lý nhanh nhưng cần rà soát theo ngữ nghĩa từng nguồn; một trường bị đổi tên vẫn có thể làm lộ thông tin.

Nếu có domain và đủ nhóm, mã lưu __group__ để GroupShuffleSplit chia train và test không giao nhau theo nhóm. Nếu không, StratifiedShuffleSplit với test_size=0,2 và seed 42 được ưu tiên; khi không chia phân tầng được, mã chuyển sang train_test_split. Quy tắc này bảo toàn nhóm ở ranh giới test ngoài, nhưng việc chia validation trong các nhánh cần được kiểm tra riêng.

drop_duplicates được áp dụng trên bảng trước xử lý. Tuy nhiên trùng toàn hàng khác trùng vector đặc trưng, và giữ định danh trong thao tác khử trùng có thể để các mẫu gần giống tồn tại. Việc loại trùng theo hash đặc trưng và cách ly nhãn mâu thuẫn là hướng nâng cấp phù hợp. Chưa có split manifest hiện tại để chứng minh mọi nhánh dùng đúng cùng các sample id.

Ranh giới fit còn có hạn chế: TF-IDF học trên toàn bộ bảng; median của scanner được tính trước chia tập; scaler MLP benchmark học trên phần train ngoài trước chia validation. Cần sửa ba điểm này trước khi dùng kết quả làm bằng chứng tổng quát mạnh. Kiểm định ngẫu nhiên cũng cần được bổ sung bằng chia theo họ mã độc, thời gian hoặc nguồn thu mẫu.

Bảng 2.9 Điểm kiểm soát rò rỉ trong mã malware

Điểm	Hiện trạng	Hướng hoàn thiện
Cột định danh	Có loại theo tên	Rà soát ngữ nghĩa nguồn
Nhóm domain	Chia nhóm test khi có	Giữ nhóm cả validation
TF-IDF	Fit trước chia test	Fit trên train
Median scanner	Tính trên toàn bảng	Tính trên train
Trùng mẫu	drop_ duplicates toàn hàng	Hash đặc trưng và nhãn
Chia tập	Seed cố định, chưa có manifest	Lưu sample id của từng tập

2.2.7 Lưu trữ dữ liệu và phiên bản bất biến

Stock lưu snapshot dữ liệu và model card theo phiên bản trong object storage. Khi đánh giá một mô hình, cần dùng đúng snapshot gắn với trọng số, scaler và danh sách đặc trưng. Tệp dự báo theo ngày là sản phẩm đầu ra; nó không thay thế lịch sử dữ liệu mà mô hình đã học. Cơ chế hash giúp đối chiếu các bộ phận này.

Malware hiện ghi dữ liệu xử lý vào processed_datasets và báo cáo tại đường dẫn CSV cố định. Scanner đọc trọng số và metadata trong models; lịch sử quét là JSON cùng tệp được lưu dưới thư mục uploads. Cách này thuận tiện cho một máy phát triển nhưng chưa tạo kho phiên bản bất biến cho từng lần huấn luyện hay phục vụ.

Trong dự án malware, MinIO cung cấp báo cáo và dashboard thông qua bucket có tên malware-evaluation. Cấu hình tạo bucket cho phép tải công khai, phù hợp chia sẻ sản phẩm minh họa nhưng cần tách với dữ liệu tệp tải lên có thể nhạy cảm. PostgreSQL trong Compose hiện phục vụ metadata Airflow, không phải cơ sở dữ liệu lịch sử scanner.

Hướng phát triển là đặt sản phẩm dưới khóa gồm run id và dataset hash, xuất kèm danh sách feature, ánh xạ nhãn và bộ tiền xử lý. Một chỉ mục active chỉ nên đổi sau khi kiểm tra đủ artifact. Đây là yêu cầu thiết kế để tăng truy nguyên, chưa được coi là registry đã hoạt động trong phiên bản malware hiện tại.

Bảng 2.10 Lưu trữ sản phẩm theo hai dự án

Sản phẩm	Stock	Malware hiện tại
Dữ liệu học	Snapshot và hash	CSV xử lý cục bộ
Model	Artifact theo phiên bản	Tệp trong models
Báo cáo	CSV và model card	CSV cố định; MinIO
Lịch sử serving	Lịch sử phát hành dự báo	JSON scan_ history
Hướng nâng cấp	Kiểm định dự báo mới	Run id và registry bất biến

2.2.8 Đồng bộ lần chạy và khôi phục

Dự án stock truyền mã pipeline_run_id từ master xuống các DAG con, đồng thời sử dụng thông tin version để ghép các sản phẩm. Khi người dùng chạy nhiều lượt cùng ngày, tên tệp ngày riêng lẻ chưa đủ xác định nguồn. Evaluation và Ensemble phải kiểm tra artifact tương ứng trước khi tổng hợp.

Trong dự án malware, report hiện được ghi theo một đường dẫn cố định; trigger được cấu hình với tham số reset_dag_run=True. Điều kiện chờ không xác nhận report thuộc đúng lượt master, nhất là khi task bỏ qua dataset lỗi. Hướng hoàn thiện là ghi report theo run id, yêu cầu đủ thuật toán và giữ lỗi thiếu metric dưới trạng thái rõ.

Khôi phục cần giữ cả nguồn, sản phẩm và môi trường. Với scanner, sao lưu đồng thời model, scaler, feature medians và metadata; với stock, giữ snapshot cùng model card. Kiểm tra container khởi động lại không thay thế diễn tập khôi phục từ backup. Các phép thử phục hồi cần được thực hiện trong môi trường riêng để không ghi đè kết quả đang dùng.

Bảng 2.11 Một số tình huống đồng bộ

Tình huống	Rủi ro	Kiểm soát
Hai lượt cùng ngày	Ghép khác version	Run id và hash
DAG con bỏ qua lỗi	Report cũ được đọc	Kiểm tra tính mới và độ phủ
Ghi lại model scanner	Scaler khác model	Sao lưu cả bundle
Khôi phục kho	Thiếu nguồn hoặc metadata	Diễn tập backup đầy đủ

2.3 Module xây dựng và quản lý mô hình

2.3.1 Huấn luyện XGBoost cho stock

dag_xgboost_train_and_predict đọc dataset từ MinIO, chuẩn hóa tên ngày, mã và ngành rồi tạo mục tiêu theo chính sách hiện hành. Đầu vào mô hình là ma trận dữ liệu 2D (N, 85) gồm 85 đặc trưng đã nén, nạp đúng 1 dòng duy nhất của ngày T cho mỗi mã để dự báo hướng giá phiên T+1 và giá mục tiêu T+5. Feature được chọn bằng các hàm dùng chung; các cột nhãn, định danh và metadata không được coi là biến dự đoán. Sau khi chia theo thời gian, nhánh phân loại tạo xác suất hướng T+1 và lưu cấu hình ngưỡng cùng model card.

hyperparameter_tuner hỗ trợ tìm kiếm không gian siêu tham số tự động qua Optuna Bayesian Search (lên tới 30 trials), sử dụng các fold theo ngày có TimeSeriesSplit và purging để chống rò rỉ dữ liệu, kết hợp phần đuôi riêng cho early stopping. Các tùy chọn điều chuẩn bao gồm max_depth (thường từ 3–7), min_child_weight, learning_rate (0,01–0,1), subsample, colsample_bytree, reg_alpha, reg_lambda và trọng số recency weights ưu tiên phiên gần hiện tại. Bản tham chiếu compact trial giữ early stopping theo outer validation để phân biệt tác động của công thức cũ và đối chứng dừng độc lập. Báo cáo không coi tất cả kết quả là sinh ra từ một giao thức giống nhau.

Ngưỡng phân loại được chọn từ validation, với quy tắc không làm giảm balanced accuracy và không làm xấu accuracy trong các khối thời gian validation. Nếu không có ứng viên đạt điều kiện, ngưỡng giữ 0,5. Lựa chọn này giảm nguy cơ ngưỡng được điều chỉnh theo test, nhưng không bảo đảm nhãn dự đoán cân bằng khi xác suất tập trung về một phía.

Nhánh hồi quy T+5 được quản lý riêng với return_policy, snapshot, hash, feature schema và model version. Bản cập nhật chart_utils.py còn dựng đường giá minh họa

từ điểm p bằng r = (p − 0,5) × 0,04 và P(t+k) = P(t) × (1+r)^k. Đây là phép quy đổi theo quy tắc hiển thị, không phải đầu ra của head hồi quy T+5; với FinBERT, p là điểm cảm xúc đã quy đổi. Các đường minh họa không được dùng làm bằng chứng mô hình dự báo được giá hoặc lợi suất thực tế.

Bảng 2.12 Chuỗi xử lý huấn luyện XGBoost stock

Bước	Đầu vào	Đầu ra
Chuẩn bị	CSV và target policy	Bảng đặc trưng có nhãn hợp lệ
Chia tập	Ngày chung giữa các mã	Train validation test và purge
Tìm cấu hình	Train và validation được phép	Siêu tham số và checkpoint
Chọn ngưỡng	Nhãn và xác suất validation	Decision threshold
Đóng gói	Model và feature schema	Artifact phiên bản cùng model card

2.3.2 Huấn luyện LSTM cho stock

Nhánh LSTM đọc dataset từ MinIO, chuẩn hóa dữ liệu và chọn 25 đặc trưng chuỗi theo chính sách artifact tương ứng. Sau làm sạch, dữ liệu được sắp theo ngày và mã, chia thời gian rồi chuẩn hóa bằng StandardScaler. Hàm create_sequences tạo tensor 3D kích thước (N, 30, 25) với lookback đúng 30 phiên nến liên tiếp. Mạng nơ-ron học tuần hoàn Stacked LSTM nhiều tầng kết hợp Layer Normalization và Dropout (0,2) để kiểm soát hiện tượng quá khớp (overfitting). Chương trình kiểm tra chuỗi rỗng, nhãn chỉ có một lớp và giá trị không hữu hạn trước khi tối ưu.

Cấu hình vận hành dùng lookback 30 phiên và so sánh ba kiến trúc hữu hạn (Ứng viên 1: 48 hidden, 2 lớp, dropout 0,2; Ứng viên 2: 32 hidden, 1 lớp; Ứng viên 3: 64 hidden, 1 lớp). Mô hình được tối ưu bằng thuật toán AdamW và hàm mất mát BCEWithLogitsLoss kết hợp gradient clipping để ổn định đạo hàm. Các ứng viên được đánh giá theo validation AUC với điều kiện log loss không xấu quá mức so với cấu hình tham chiếu. Mỗi lượt có seed cố định để giảm biến động số học trong quá trình lựa chọn.

Trong vòng huấn luyện, model.train bật hành vi học; mỗi batch tính loss, backward, clipping và optimizer step. Khi tính xác suất validation, model.eval cùng no_grad giúp tắt Dropout và tránh lưu gradient. Trạng thái tốt nhất được deep copy để những epoch sau không ghi đè checkpoint đang giữ. Sau early stopping, mô hình khôi phục trọng số tốt nhất trước khi xuất. Những thao tác này cần kiểm thử trực tiếp, không chỉ mô tả bằng tên callback.

Bộ model lưu cần có weights, scaler, schema và cấu hình mạng. Inference phải dựng đúng hidden_dim, num_layers và feature order. Nếu một thành phần thiếu, nhánh phải không khả dụng thay vì tải một scaler khác có kích thước trùng. Báo cáo thực nghiệm giữ phân biệt giữa artifact đang phục vụ và mô hình được huấn luyện lại trong runner nghiên cứu trên nguồn giá chung.

Bảng 2.13 Cấu hình và điểm kiểm soát LSTM stock

Thành phần	Cấu hình hoặc trách nhiệm
Lookback vận hành	30 phiên cùng mã
Ứng viên 1	48 hidden, 2 lớp, dropout 0,2
Ứng viên 2	32 hidden, 1 lớp, dropout LSTM 0
Ứng viên 3	64 hidden, 1 lớp, dropout LSTM 0
Tối ưu	AdamW và BCEWithLogitsLoss
Ổn định	Gradient clipping và giảm learning rate
Lưu	Checkpoint tốt nhất và scaler tương ứng

2.3.3 Suy luận FinBERT và điều kiện ghép tin

dag_finbert_predict tải mô hình ngôn ngữ Transformer ProsusAI FinBERT tiền huấn luyện và đọc các bản tin RSS tài chính (Yahoo Finance, Google News, CNBC) thu thập trong vòng 24h–48h gần nhất. Nội dung tiêu đề và tóm tắt được token hóa tối đa 128 tokens rồi đưa qua FinBERT để phân loại xác suất ba trạng thái: positive, negative và neutral. Điểm cảm xúc chuẩn hóa được tính theo công thức Score = (1 + Pos − Neg) / 2 thuộc khoảng [0, 1]. Mỗi kết quả ghi nhận tên model, commit hash, hash nội dung đầu vào và thời điểm scoring để bảo đảm tính truy nguyên minh bạch [13].

Ở cấp ngày và mã, hệ thống lưu tổng hợp cảm xúc cùng số bài và nguồn. Số bài là thông tin quan trọng: một điểm trung bình từ một tin không có cùng mức bao phủ như điểm từ mười tin. Các bài viết trùng hoặc thay nội dung cũng cần được nhận diện bằng phiên bản. Khi nguồn lỗi, trạng thái lỗi phải khác với trường hợp nguồn hoạt động nhưng không có bài phù hợp.

FinBERT pretrained không cần huấn luyện lại trên OHLCV để phân loại cảm xúc, nhưng muốn đánh giá giá trị đối với hướng giá phải có một giao thức riêng. Có thể so sánh mô hình giá, giá cộng độ phủ tin, và giá cộng cảm xúc; các nhánh phải dùng cùng mẫu có điều kiện quan sát đúng thời điểm. Kho dữ liệu đã lưu chưa có đủ lịch sử sentiment hợp lệ cho phép thử đó.

Chính sách hiện hành ngừng sử dụng các artifact FinBERT T+5 cũ làm dự báo giá độc lập. Signal tuyến tính vẫn được giữ và gắn phiên bản để phục vụ Ensemble cố định. Phần học trọng số mới chỉ nên dùng quan sát đã trưởng thành và cần holdout theo thời gian. Việc một giao diện hiển thị cảm xúc tích cực không đồng nghĩa đã có mô hình kiểm chứng tác động của cảm xúc lên giá phiên sau.

Bảng 2.14 Metadata cần giữ cho suy luận cảm xúc

Thông tin	Vai trò
Ba điểm cảm xúc	Đầu ra phân loại nội dung
Article count	Độ phủ của tổng hợp
Publication time	Thời điểm nguồn công bố
Observed time	Thời điểm hệ thống thấy phiên bản
Scored time	Thời điểm có điểm từ mô hình
News policy	Quy tắc nhận tin vào lịch sử
Model commit và hash	Truy nguyên nội dung và mô hình

2.3.4 Huấn luyện Random Forest đa dữ liệu

Mô hình Random Forest trong benchmark được cấu hình với 120 cây. Độ sâu của cây được xác định qua tham số max_depth, tính theo số lượng mẫu và giới hạn trong khoảng từ 15 đến 30. Tham số min_samples_leaf được đặt bằng 3, còn tham số min_samples_split nhận giá trị 6. Cấu hình chọn giá trị sqrt cho tham số max_features và giá trị balanced_subsample cho tham số class_weight. Seed 42 cố định chia tập và mô hình. Tham số n_jobs được đặt bằng -1 để thư viện sử dụng các lõi khả dụng; khi ba DAG cùng chạy cần cân nhắc cạnh tranh CPU.

Sau khi tách test ngoài 20%, phần train được chia tiếp validation 15%. Mô hình fit trên X_tr; validation dùng để chọn ngưỡng trong 56 giá trị từ 0,40 đến 0,95. Quy tắc ưu tiên F1 cao nhất khi FPR validation không quá 3,5%; nếu không tìm được, mã chọn ngưỡng có FPR nhỏ nhất. FPR trên test vẫn có thể vượt mức đặt trên validation.

Với nhị phân, ROC-AUC dùng xác suất lớp 1; dự đoán dùng ngưỡng đã chọn. Với đa lớp, dự đoán lấy argmax và AUC dùng one-vs-rest weighted. Các nhãn test chưa thấy trong train bị lọc; khi đọc kết quả cần ghi nhận việc này vì nó thay đổi tập đánh giá. Nhánh lỗi AUC có phép gán từ Accuracy, nên các nguồn đa lớp tương lai cần được rà soát trước khi so sánh.

Scanner sử dụng RandomForestClassifier với 100 cây, max_depth=20 và min_samples_leaf=2 trong script train_detector_models.py hiện tại. Mô hình PE được

lưu ở pe_rf_model.pkl, đồng thời có bản sao rf_malware_model.pkl để tương thích. Không dùng cấu hình Scanner để giải thích bảng benchmark 120 cây, cũng không thay metric Scanner bằng kết quả benchmark.

Bảng 2.15 Cấu hình Random Forest benchmark và scanner

Thông số	Benchmark	Scanner
Số cây	120	100
Độ sâu	Thích ứng 15-30	20
min_ samples_ leaf	3	2
Validation ngưỡng	15% phần train ngoài	Ngưỡng serving 0,5
Sản phẩm	CSV báo cáo	rf_ malware_ model.pkl

2.3.5 Huấn luyện XGBoost đa dữ liệu

Đối với benchmark, mô hình XGBoost sử dụng cấu hình gồm 120 cây. Cấu hình đặt tham số learning_rate bằng 0,08, trong khi tham số max_depth được thiết lập ở mức 5. Cấu hình đặt min_child_weight bằng 3, đồng thời sử dụng giá trị 0,8 cho cả hai tham số subsample và colsample_bytree. Về điều chuẩn, tham số reg_alpha nhận giá trị 0,1, tham số reg_lambda được đặt bằng 1 và gamma bằng 0,1. Cấu hình tree_method nhận giá trị hist, hỗ trợ xây dựng cây theo histogram. Trọng số mẫu được tính theo lớp để giảm ảnh hưởng của mất cân bằng.

Phần train ngoài được tách validation 10% và truyền eval_set khi fit. Tuy chú thích mã gọi đây là bước early stopping, lời gọi hiện tại chưa truyền tham số early_stopping_rounds cũng như callback tương ứng. Vì vậy báo cáo mô tả tập theo dõi validation, không khẳng định số vòng cây đã được dừng sớm. Ngưỡng quyết định nhị phân được chọn sau fit theo kiểm soát FPR validation.

Quá trình đánh giá sử dụng xác suất trên tập test để tính AUC, đồng thời sử dụng dự đoán sau ngưỡng để tính Accuracy, Precision, F1 và FPR. Các giá trị metric được làm tròn rồi ghi vào tệp CSV tương ứng với từng tập dữ liệu. Chỉ lưu metric tổng hợp chưa đủ để tính lại độc lập một đường ROC hay kiểm định chênh lệch; cần bổ sung dự đoán từng mẫu và split id.

Scanner dùng bộ cấu hình khác: 100 cây, độ sâu 6, learning_rate=0,1. Trong trường hợp môi trường không có thư viện XGBoost, script scanner có thể sử dụng một mô hình thay thế là GradientBoostingClassifier. Muốn xác nhận thuật toán của một artifact cần kiểm tra môi trường và loại model đã lưu; tên hiển thị trên giao diện không thay thế bước này.

Bảng 2.16 Một số tham số XGBoost benchmark

Tham số	Giá trị	Vai trò
n_ estimators	120	Số vòng boosting
max_ depth	5	Giới hạn chiều sâu
learning_ rate	0,08	Mức đóng góp cây
subsample / colsample	0,8 / 0,8	Lấy mẫu hàng và cột
reg_ alpha / reg_ lambda	0,1 / 1	Điều chuẩn
Validation trong train	10%	Theo dõi và chọn ngưỡng

2.3.6 Huấn luyện MLP đa dữ liệu

MLP benchmark dùng TensorFlow/Keras. Kiến trúc gồm các lớp Dense, ReLU, BatchNormalization và Dropout; kích thước được điều chỉnh theo số đặc trưng trong mã. Lớp đầu ra có số nút bằng số lớp và softmax; hàm mất mát sparse_categorical_crossentropy cho nhãn đã mã hóa. Đây là nhánh mạng nơ-ron khác với MLP scikit-learn của scanner.

StandardScaler được fit trên train ngoài, sau đó transform test. Phần train đã chuẩn hóa được tách tiếp validation 15%. Phần validation này được sử dụng cho các callbacks để theo dõi giá trị val_loss và lựa chọn quá trình học. Để validation thực sự độc lập ở mọi bước, cần chia trước rồi chỉ fit scaler trên phần học trọng số. Hiện trạng này được giữ trong phân tích giới hạn kết quả.

Với nhị phân, ngưỡng dự đoán được chọn theo validation với mục tiêu FPR. MLP có chi phí phụ thuộc số mẫu, số chiều đầu vào và số epoch. Sử dụng CPU song song với hai mô hình cây có thể làm tăng thời gian mỗi nhánh, vì vậy độ trễ phải được đo trên cấu hình thực thay vì suy từ việc master tạo ba nhánh.

Scanner dùng MLPClassifier của scikit-learn với scaler riêng cho mỗi miền PE, PDF, Script và Office. Khi suy luận, thứ tự feature lấy từ metadata và MLP nhận vector đã chuẩn hóa. Bộ đóng gói cần giữ đúng cặp model/scaler, medians và feature_cols. Các DAG benchmark và script tạo báo cáo có thể dùng kiến trúc khác nhau, nên phải ghi rõ đường chạy của mỗi bảng kết quả.

Bảng 2.17 Hai loại mạng MLP trong dự án

Thuộc tính	Benchmark	Scanner
Thư viện MLP	DAG: TensorFlow/Keras; script báo cáo: scikit-learn	scikit-learn
Mục tiêu	Đánh giá trên CSV từng bộ dữ liệu	PE, PDF, Script, Office
Chuẩn hóa	StandardScaler theo đường chạy	Scaler riêng theo miền
Đầu ra	CSV metric	{miền}_ mlp_ model.pkl
Điểm cần kiểm soát	Split, validation và script sinh report	Đúng cặp model/scaler và schema

2.3.7 Bộ mô hình phục vụ scanner đa miền (PE, PDF, Script, Office)

Script train_detector_models.py huấn luyện bốn nhóm mô hình: PE từ dataset_malwares.csv (77 đặc trưng); PDF từ PDF_Malware_Structural_2022.csv (18); Script từ Malicious_Scripts_PowerShell_VBS.csv (18); Office từ Office_Macro_OLE_Malware.csv (11). Mỗi nhóm có Random Forest, XGBoost và MLP, tổng cộng 12 mô hình. Dữ liệu được chia phân tầng 80/20 với seed 42; RF và XGBoost dùng dữ liệu số gốc, MLP dùng StandardScaler fit trên train. Median hiện tính trước chia tập và vẫn là điểm cần hoàn thiện.

Tệp model_metadata.json ghi phiên bản 3.0.0-multi-domain và thời điểm huấn luyện 24/10/2026 00:04:43 theo chuỗi thời gian lưu trong tệp (không kèm múi giờ). Mỗi miền có tên dataset, tổng số mẫu, feature_cols, feature_medians và metric ba thuật toán. Metadata không lưu đầy đủ sample id hoặc bằng chứng kiểm thử end-to-end từ byte tệp.

Engine nạp các tệp pe_*, pdf_*, script_* và office_* trong dags/models/. Các tên rf_malware_model.pkl, xgb_malware_model.pkl và mlp_malware_model.pkl được duy trì để tương thích PE. Magic Bytes và phần mở rộng hỗ trợ chọn miền rồi tạo vector theo đúng thứ tự metadata. Các tệp ngoài bốn miền hoặc lỗi nạp model vẫn có nhánh kết quả mặc định; đây là giới hạn cần phân biệt với suy luận học máy thành công.

Điểm tổng hợp ban đầu dùng 40% XGBoost, 35% Random Forest và 25% MLP. Quy tắc về API, bộ cài đặt và dấu hiệu chữ ký có thể tiếp tục giảm điểm hoặc đổi nhãn. Việc nhận diện vùng chứng chỉ và chuỗi nhà phát hành chưa tương đương xác minh

chữ ký số hợp lệ. Hiệu quả giảm dương tính giả cần được đo cùng mức bỏ sót trên tập tệp độc lập.

Bảng 2.18 Thành phần đóng gói scanner đa miền hiện tại

Thành phần	Nội dung
model_ metadata.json	Phiên bản 3.0.0-multi-domain; schema, medians và metric từng miền.
pe_ *	RF, XGBoost, MLP và scaler PE; 77 đặc trưng.
pdf_ *	RF, XGBoost, MLP và scaler PDF; 18 đặc trưng.
script_ *	RF, XGBoost, MLP và scaler Script; 18 đặc trưng.
office_ *	RF, XGBoost, MLP và scaler Office; 11 đặc trưng.
dags/malware_ samples/	Mẫu phục vụ kiểm tra chức năng; không thay thế tập đánh giá độc lập.

2.3.8 Đăng ký và lựa chọn phiên bản mô hình

Dự án stock sử dụng tệp mã nguồn model_registry.py để quản lý các model card và artifact được lưu trong MinIO. Phiên bản giúp liên kết mô hình với snapshot, feature policy, target policy và metric. Cơ chế chọn champion giữ một mô hình đã có nếu ứng viên mới không đạt điều kiện chất lượng; do đó điểm của ứng viên và điểm của model đang phục vụ có thể thuộc hai phiên bản khác nhau.

Ở bước huấn luyện stock, siêu tham số và ngưỡng phân loại được chọn bằng train/validation. Cơ chế chọn phiên bản phục vụ chính thức (Champion Model) thực hiện qua hàm select_champion_model ở cả DAG XGBoost và LSTM, kiểm tra các chỉ số trên tập test độc lập 126 phiên (~6 tháng): tiêu chuẩn tối thiểu là min_acc = 0,48 (48%) và min_auc = 0,45, cùng điều kiện max_lookback_days = 14. Nếu mô hình ứng viên mới không vượt qua Champion Gate này, hệ thống tự động kích hoạt Safe Mode (Chế độ An toàn) / Fallback, khuyến nghị trạng thái đứng ngoài (Neutral/Wait) nhằm hạn chế rủi ro cho danh mục đầu tư. Hướng hoàn thiện lâu dài là dùng validation cho champion gate và giữ một tập test tách biệt hoàn toàn để báo cáo sau lựa chọn.

Malware hiện nạp model tại đường dẫn cố định, chưa có con trỏ active hoặc registry version trong mã phục vụ. Metadata cung cấp thông tin cấu hình nhưng không chứa toàn bộ dấu vết từng lần chạy. Cần sao lưu bộ model trước khi huấn luyện lại; muốn rollback có kiểm soát cần lưu cả trọng số, scaler, feature medians và ánh xạ nhãn.

Hướng mở rộng là xây dựng bundle bất biến và quality gate. Gate cần kiểm tra tương thích đầu vào và metric trên validation, rồi mới cho phép kích hoạt. Đề xuất này

áp dụng kinh nghiệm từ stock sang malware nhưng chưa được trình bày như một thành phần đang vận hành.

2.4 Module đánh giá mô hình

2.4.1 Đánh giá mô hình cổ phiếu

DAG có tên dag_model_evaluation gồm ba nhánh đánh giá. Nhánh thứ nhất là evaluate_xgboost, nhánh tiếp theo là evaluate_lstm và nhánh còn lại là evaluate_ensemble. Sau ba nhánh đánh giá, DAG thực hiện việc so sánh thông qua bước compare_all_models. Báo cáo sau đó được gửi bằng bước send_evaluation_report. XGBoost và LSTM được đọc theo artifact cùng nguồn dataset đã ghi hash; nhánh ensemble sử dụng dự đoán phát hành có provenance. Đây là hai loại đánh giá khác nhau: holdout của model và kết quả dự báo được phát hành thực.

Dashboard chấp nhận các báo cáo dựa trên nhãn giao thức được ghi kèm. Các nhãn giao thức được chấp nhận gồm trained_model_holdout_v2. Bên cạnh nhãn này còn có issued_trained_forecasts_v2, cùng với nhãn issued_weighted_forecasts_v1. Thông tin đầu ra cần ghi rõ thời gian đánh giá, horizon và số lượng mẫu. Nếu không có đủ lớp hoặc mẫu trưởng thành, những metric không xác định phải được để ở trạng thái không khả dụng. FinBERT cảm xúc được loại khỏi bảng accuracy hướng giá vì không cùng mục tiêu.

forecast_history.py lưu issued_at thực, reference_date là phiên đóng đã biết, model version và horizon. Khi đánh giá, dự báo T+1 phải ghép với phiên quan sát tiếp theo; dự báo mới chưa có giá kết quả được giữ pending. Cuối tuần hoặc ngày nghỉ không được xử lý bằng cách đơn giản cộng một ngày lịch. Dự báo thiếu provenance không được nhập vào bằng cách đoán thời điểm từ tên file.

Runner nghiên cứu walk-forward là nguồn so sánh ghép cặp XGBoost, LSTM và Logistic Regression trong báo cáo. Các runner đó huấn luyện lại trên nguồn giá đóng băng, ghi dự đoán từng mẫu và bootstrap theo khối ngày. Chúng không cập nhật artifact phục vụ hoặc tự phát hành model thắng. Phân biệt này cần được giữ để tránh cho rằng điểm trong runner là điểm của dashboard hiện đang chạy.

Bảng 2.19 Phân biệt các loại đánh giá stock

Loại đánh giá	Nguồn	Câu hỏi trả lời
Trained holdout	Model và snapshot đã huấn luyện	Model đạt gì trên phần giữ lại
Issued forecast	Dự báo lưu trước khi biết nhãn	Đầu ra vận hành đúng đến đâu
Walk forward	Model huấn luyện lại theo fold	Quy trình lựa chọn ổn định thế nào
Sentiment	Điểm và nhãn cảm xúc nếu có	Phân loại nội dung đúng đến đâu

2.4.2 Tổng hợp và đánh giá malware

DAG tổng hợp kết quả có tên model_evaluation_dashboard thực hiện việc đọc dữ liệu từ ba tệp báo cáo thuật toán (XGBoost_MultiDataset_Report.csv, RandomForest_MultiDataset_Report.csv, MLP_MultiDataset_Report.csv). Tệp combined_model_evaluation_report.csv là báo cáo tổng hợp được sinh ra từ các hàng dữ liệu đã ghép, bao phủ trọn vẹn 23 tập dữ liệu an ninh mạng với tổng cộng 69 lượt đánh giá chi tiết (23 tập × 3 mô hình).

Thống kê theo thuật toán là trung bình không trọng số trên 23 bộ dữ liệu, khác trung bình weighted của Precision/F1 bên trong từng bộ. Mỗi bộ đóng góp một giá trị bất kể số mẫu. Vì các nguồn, nhãn và tỷ lệ lớp khác nhau, kết quả chỉ mô tả các thí nghiệm được lưu; không tự chứng minh khả năng tổng quát hóa ngoài miền.

Mã cho phép tiếp tục khi thiếu một file báo cáo và thay cột số thiếu bằng 0. Điều này thuận tiện cho minh họa nhưng có thể làm bảng thiếu thuật toán hoặc chứa giá trị thay thế. Hướng hoàn thiện là yêu cầu đủ ba nguồn cùng run, giữ null cho metric không tính được và không dùng kết quả cũ làm đầu vào mới khi một nhánh thất bại.

Dashboard được chọn quán quân theo F1 rồi Accuracy. Cách xếp hạng này là một quy tắc trình bày; nó không thay thế kiểm định thống kê, đánh giá thời gian hoặc kiểm định ngoài miền. Kết quả benchmark và kết quả scanner cần được trình bày thành hai bảng riêng vì khác cấu hình, tập test và cách tạo kết luận.

Bảng 2.20 Nguồn kết quả malware được phân biệt trong báo cáo

Nguồn	Quy mô	Cách đọc
CSV benchmark	23 bộ; 69 hàng metric	So sánh trong phạm vi dữ liệu lưu
Metadata Scanner	4 miền; 12 mô hình	Metric từng bộ model
Ảnh quét lịch sử	7z.exe; fake_ trojan_ test.exe	Minh họa giao diện phiên bản cũ
Mẫu bổ sung	PDF, Script và PE	Kiểm tra chức năng; cần log phiên bản

2.4.3 Dashboard API và tính nhất quán đầu ra

Dashboard stock đọc các sản phẩm đã lưu và có chức năng phân tích theo yêu cầu. Các tab giá, XGBoost, LSTM và FinBERT phục vụ những lớp thông tin khác nhau. Giá live, xác suất hướng giá và cảm xúc tin phải giữ mốc dữ liệu riêng. Nếu một artifact không có, giao diện cần thể hiện chưa có kết quả; không dựng một dự báo để lấp trường trống.

Scanner malware dùng dịch vụ HTTP và ScanHistoryManager. API /api/scan nhận tệp, gọi engine và lưu kết quả; /api/health báo tình trạng; /api/history và /api/scans phục vụ tra cứu lịch sử. Dữ liệu lịch sử được ghi vào tệp có tên scan_history.json, với giới hạn tối đa là 200 bản ghi; tệp được lưu cục bộ dưới uploads. Chức năng tải lại hỗ trợ đối chiếu tên và kết quả đã ghi.

Các trường liên kết quan trọng là hash tệp, thời điểm quét, loại tệp, feature và kết quả từng nhánh. Điểm tổng hợp có thể khác xác suất riêng vì quy tắc hậu xử lý. Nhận diện chuỗi nhà phát hành hoặc một vùng chữ ký không phải xác minh chuỗi chứng thư tin cậy. Khi hiển thị trạng thái an toàn phải giữ đúng giới hạn của phân tích tĩnh.

Ảnh giao diện hỗ trợ kiểm tra người dùng có thể tiếp cận các chức năng. Tuy nhiên muốn chứng minh độ nhất quán cần đối chiếu API, model và dữ liệu gốc của cùng phiên bản. Hiện chưa có thử tải và phục hồi lịch sử toàn diện; đây là công việc cần bổ sung trước triển khai dài hạn.

Bảng 2.21 Thông tin nối sản phẩm với giao diện

Thông tin	Stock	Malware
Định danh	Ticker, phiên	Tên tệp, SHA-256
Nguồn điểm	Model card, snapshot	Model và heuristic
Trạng thái thiếu	Unavailable / chưa có	Cần trạng thái từng model
Lịch sử	Thời điểm phát hành	scan_ history.json

2.5 Kết luận chương 2

Bốn nhóm module hình thành chu trình từ thu thập dữ liệu đến công bố kết quả. Định danh lần chạy, schema, bộ tiền xử lý và phiên bản mô hình nối các module với nhau. Malware có benchmark trên 23 bộ CSV và Scanner cho bốn nhóm tệp; stock đã mở rộng danh mục 100 mã với lịch sử thu thập 5 năm. Chương 3 tách cấu hình cập nhật với những kết quả lịch sử có sẵn.

CHƯƠNG 3 TRIỂN KHAI VÀ THỰC NGHIỆM HỆ THỐNG

3.1 Thiết lập môi trường

3.1.1 Thành phần và cấu hình triển khai

Hai dự án được triển khai bằng Docker Compose với mạng và dịch vụ riêng. Stock dùng Airflow 2.9.3/Python 3.11 với LocalExecutor; malware dùng image nền Airflow 2.10.0 và CeleryExecutor. PostgreSQL giữ metadata Airflow; Redis làm broker cho Celery ở malware. MinIO lưu sản phẩm và hỗ trợ công bố dashboard.

Stock có dashboard tại cổng 8050 và MLflow tại cổng 5000. Malware có scanner HTTP tại 5050, worker, triggerer và webserver tại 8082. Phiên bản malware hiện tại không có dịch vụ MLflow riêng trong Compose. Đặc điểm này khác bản mô tả nâng cấp lịch sử, nên phân tích triển khai dựa vào tệp cấu hình hiện có.

Thư viện stock được cài khi build và có các ràng buộc tương thích với Airflow. requirements malware liệt kê numpy dưới 2.0 cùng scikit-learn, TensorFlow, XGBoost, pandas và thư viện lưu trữ, chưa khóa toàn bộ phiên bản. Môi trường chạy lại cần lưu phiên bản đã giải quyết thực tế, nhất là với API callbacks của XGBoost và bộ model dạng pickle.

Các thư mục dags, logs và plugins được gắn từ host. Worker và scheduler cần nhìn thấy cùng các đường dẫn sản phẩm cục bộ. Chạy CeleryExecutor trên nhiều máy đòi hỏi một chiến lược lưu trữ dùng chung; việc một máy có nhiều container không tự chứng minh khả năng phân tán dữ liệu.

Bảng 3.1 Thành phần môi trường hai dự án

Thành phần	Stock	Malware
Airflow	2.9.3	2.10.0
Executor	LocalExecutor	CeleryExecutor
Broker	Không dùng Redis	Redis
Kho trạng thái	PostgreSQL	PostgreSQL
Kho sản phẩm	MinIO	MinIO
Giao diện ứng dụng	Dashboard Python	Dashboard và scanner HTTP
MLflow trong Compose	Có	Không có

Môi trường của thí nghiệm walk-forward ngày 05/10 được lưu trong results.json: Python 3.11.8, NumPy 1.26.4, pandas 2.1.4, scikit-learn 1.9.1, PyTorch 2.14.1+cpu,

XGBoost 2.1.4 và Optuna 5.0.0. Hồ sơ thực nghiệm ghi container giới hạn 3 GiB RAM và hai CPU; đây là cấu hình của lượt nghiên cứu đó. Tệp kết quả lưu SHA-256 của dữ liệu và các tệp mã nguồn để nhận diện bản đã chạy. Khi tái lập cần dùng đúng hồ sơ này thay vì gán phiên bản thư viện hoặc cấu hình máy hiện tại cho thí nghiệm cũ.

3.1.2 Cổng dịch vụ và tổ chức dữ liệu

Compose mặc định của hai dự án cùng dùng cổng MinIO 9000/9001. Nếu chạy đồng thời trên cùng host cần thay ánh xạ hoặc dùng cấu hình tách cổng. Cấu hình kiểm chứng stock có thể ánh xạ sang 9010/9011. Cổng bên ngoài thay đổi không làm thay đổi endpoint nội bộ minio:9000 nếu các dịch vụ vẫn nằm trong cùng mạng dự án.

Tên mạng, volume và container cần được phân biệt giữa hai project. Một lỗi khởi động do trùng cổng khác lỗi kết nối bên trong container; cần xác định rõ trước khi đánh giá DAG. Các giá trị đăng nhập mẫu chỉ dùng cho môi trường phát triển và không được đưa vào các ảnh báo cáo như một thành phần của kiến trúc.

Dữ liệu stock được lưu theo loại sản phẩm, ngày và phiên bản. Malware có datasets, processed_datasets, models, reports và uploads. Sự tách biệt này giúp xác định liệu một kết quả thuộc benchmark hay scanner. Dù đều dùng CSV và MinIO, chúng không có cùng vòng đời hoặc hợp đồng dữ liệu.

Bảng 3.2 Các cổng dịch vụ trong cấu hình mặc định

Dịch vụ	Stock	Malware	Ghi chú
Airflow	8081	8082	Giao diện DAG
Ứng dụng	8050	5050	Dashboard / scanner
MinIO API	9000	9000	Cần tách nếu cùng host
MinIO console	9001	9001	Cần tách nếu cùng host
MLflow	5000	Không cấu hình	Tracking stock
Flower	Không cấu hình	5555 qua profile	Giám sát worker

3.1.3 Trình tự khởi động và kiểm tra

Chuẩn bị bắt đầu từ thư mục dự án, Docker Compose và quyền ghi logs, dữ liệu trung gian. Các biến kết nối được lấy từ tệp môi trường theo từng project. Image cần được build trước; database và broker phải sẵn sàng trước scheduler và worker. Airflow init hoàn tất tạo schema và tài khoản quản trị.

Tiếp theo cần kiểm tra container, import DAG và khả năng truy cập nguồn. Stock phải kết nối kho dữ liệu và model card. Malware cần datasets có CSV hợp lệ và các thư mục report/model có quyền ghi. Thiếu tập dữ liệu không được giải thích bằng việc webserver vẫn mở được.

Sau khi kiểm tra cấu trúc, có thể kích hoạt master theo nhu cầu. Lần chạy cần được nhận diện rõ bằng ngày và run id; trước khi chạy lại một báo cáo tại đường dẫn cố định nên lưu bản cũ. Khi kết quả xuất hiện, cần đối chiếu số hàng, tên tập và mốc đánh giá với giao diện.

Bảng 3.3 Trình tự kiểm tra trước thực nghiệm

Bước	Nội dung	Điều kiện tiếp tục
1	Cấu hình và quyền ghi	Không thiếu biến hoặc thư mục
2	Database và broker	Dịch vụ sẵn sàng
3	Scheduler, worker, webserver	Không lỗi import DAG
4	Nguồn dữ liệu và model	Có đúng tệp, lược đồ
5	Kích hoạt master	Định danh lượt chạy
6	Đọc report và giao diện	Đối chiếu số hàng và mốc

3.2 Triển khai các Module

3.2.1 Triển khai các module stock

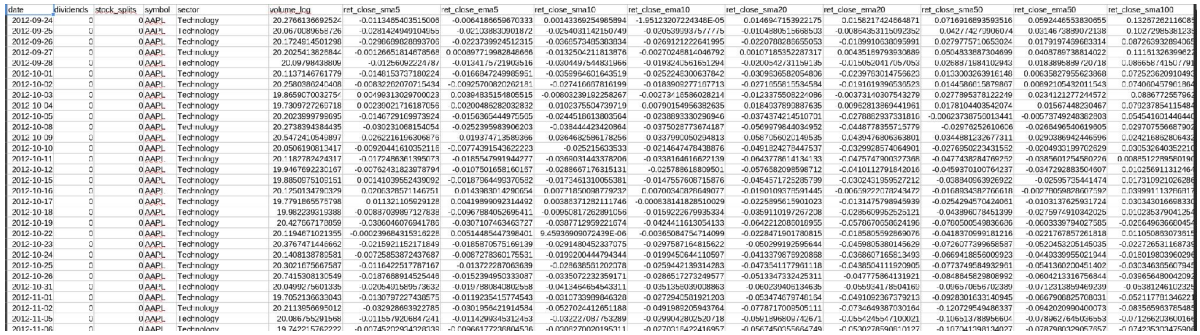
Stock triển khai crawler giá cho XGBoost và LSTM, crawler tin tức, các DAG huấn luyện/suy luận, Ensemble và đánh giá cho toàn bộ 100 mã cổ phiếu lớn nhất S&P trong config_shared.py. Toàn bộ các thành phần đồng bộ dữ liệu lịch sử 5 năm và 85 đặc trưng cho 100 mã, phục vụ phân tích thời gian thực và phát hành tín hiệu tự động.

Khi crawler hoàn tất, bước kiểm định dữ liệu (validation) tự động kiểm tra độ phủ mã, số cột đặc trưng, dữ liệu thiếu và tính hợp lệ của giá. Hệ thống ghi nhận độ phủ đạt 100/100 mã cổ phiếu với hơn 126.000 bản ghi lịch sử, 85 cột đặc trưng cho XGBoost và 25 cột chuỗi cho LSTM. Tỷ lệ dữ liệu hợp lệ đạt mức tối đa trước khi kích hoạt các DAG huấn luyện tiếp theo.

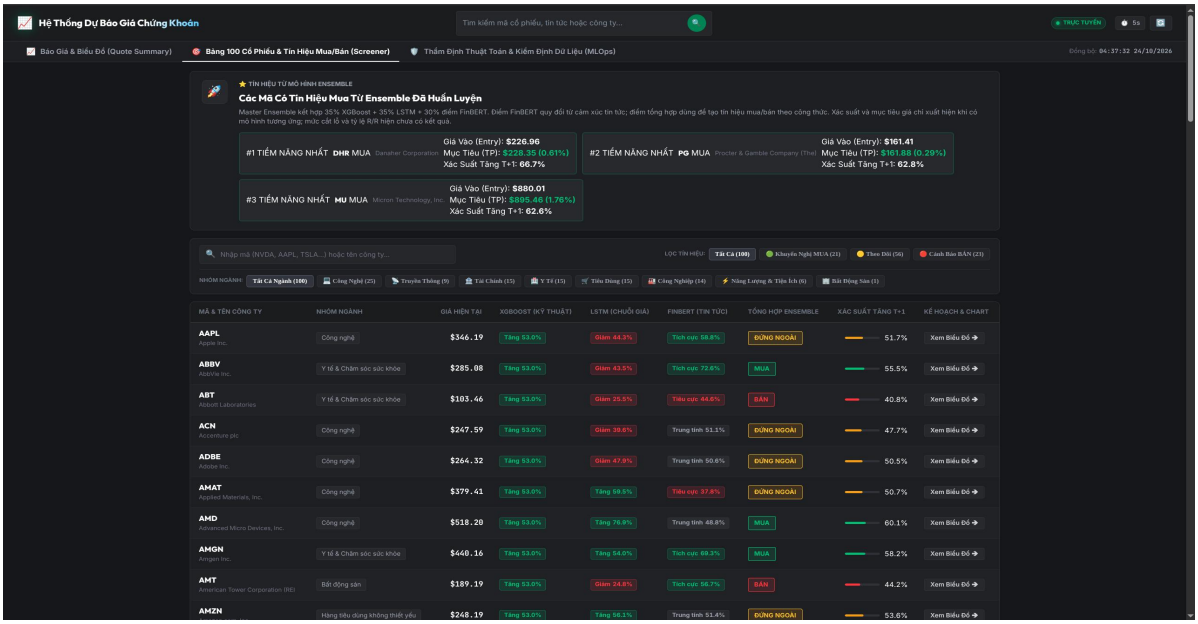
Model card và artifact cho phép dashboard hiển thị kết quả từng nhánh. Một mô hình mới không vượt quality gate có thể không trở thành champion; vì vậy đầu ra hiện tại cần được nối với phiên bản thực sự đã dùng. FinBERT xuất cảm xúc và số tin, không tự dự báo giá tương lai.

Các tab trên dashboard giúp người dùng đọc giá, báo cáo AI và các nguồn tín hiệu. Đầu ra "Chưa có kết quả" hoặc "Không khả dụng" được giữ trong ảnh khi thiếu

artifact hoặc nhãn. Việc công bố trạng thái này giúp phân biệt dữ liệu sẵn có và kết quả chưa đủ điều kiện tính.



Hình 3.1 Bảng đặc trưng đầu vào của dự án stock



Hình 3.2 Giao diện bảng tổng hợp cổ phiếu và trạng thái các nguồn

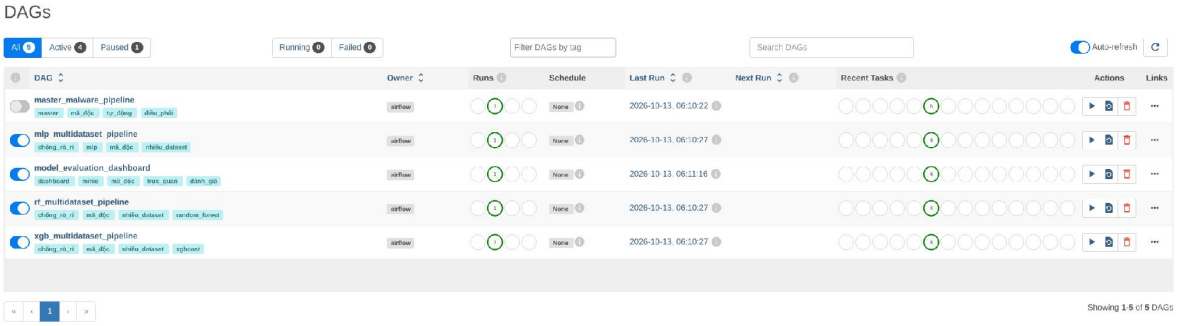
3.2.2 Triển khai các module malware

Malware triển khai ba DAG rf_ multidataset_ pipeline, xgb_ multidataset_ pipeline và mlp_ multidataset_ pipeline. Mỗi DAG có bốn task: extract_ datasets, transform_ data, train_ model tương ứng và save_ data_ to_ file. Sản phẩm là các CSV chỉ số theo dataset. Master gọi ba DAG con trước khi gọi model_ evaluation_ dashboard.

Các hàm transform có cấu trúc tương tự nhưng nằm trong ba tệp độc lập. Chúng cùng tìm nhãn, bỏ cột nhận diện và mã hóa, nhưng RF và XGBoost lấy tối đa 15.000 hàng, còn MLP lấy tối đa 12.000 hàng. Các CSV đã xử lý hiện có đúng các quy mô này. Vì đầu vào và tập test của MLP khác hai nhánh cây, bảng benchmark chưa phải phép so sánh ghép cặp trên cùng các mẫu test. Cần dùng chung danh sách mẫu và lưu split id để tăng tính công bằng của phép so sánh.

DAG model_evaluation_dashboard thực hiện thu thập và hợp nhất kết quả từ 3 báo cáo thành phẩm (RandomForest_MultiDataset_Report.csv, XGBoost_MultiDataset_Report.csv, MLP_MultiDataset_Report.csv), lưu trữ vào combined_model_evaluation_report.csv với quy mô 23 tập dữ liệu × 3 mô hình (69 bản ghi thực nghiệm). Tác vụ generate_charts sinh đồ thị 300 DPI và vector PDF, tác vụ generate_html_dashboard kiến tạo giao diện SOC Web GUI trực quan, và tác vụ upload_to_minio tự động xuất bản toàn bộ lên MinIO Object Storage qua bucket malware-evaluation, đồng thời đồng bộ sang thư mục docs/ phục vụ triển khai GitHub Pages.

Scanner chạy như một dịch vụ riêng. Nó nạp các model từ models và nhận tệp từ dashboard, không phụ thuộc vào việc evaluation đang vẽ biểu đồ. Muốn model scanner cập nhật cần chạy script đóng gói riêng và kiểm tra metadata; chạy lại ba DAG benchmark chỉ cập nhật report benchmark.



Hình 3.3 Danh sách năm DAG malware trong Airflow

Bảng 3.4 Đầu ra của từng module malware

Module	Sản phẩm	Vị trí
Thu thập	Danh sách CSV	datasets
Xử lý	CSV số và nhãn	processed_ datasets
Benchmark	Metric từng thuật toán	Các tệp MultiDataset_ Report.csv
Evaluation	CSV tổng hợp, biểu đồ, HTML	reports và MinIO
Scanner	Model, scaler, metadata	models
Lịch sử quét	JSON và tệp đã tải lên	uploads

3.2.3 Triển khai scanner và các giới hạn suy luận

Scanner nhận diện định dạng bằng Magic Bytes kết hợp phần mở rộng. Nhánh PE dùng pefile để trích xuất các trường thuộc schema 77 chiều; nếu parser lỗi, nhánh dự phòng chỉ lấy một phần và điền trường thiếu. PDF và script có 18 đặc trưng mỗi

loại, Office có 11 đặc trưng. Bộ trích xuất tài liệu hiện chủ yếu đếm chuỗi trong byte tệp, chưa thay thế phân tích đầy đủ đối tượng PDF hay giải nén, giải mã macro Office.

Vector đặc trưng được đưa đến bộ mô hình PE, PDF, Script hoặc Office tương ứng. Engine tính xác suất từng mô hình rồi tổng hợp và hậu xử lý. Khi đọc kết quả cần tách điểm mô hình, điểm rủi ro sau quy tắc và các chỉ báo tĩnh; chưa có bằng chứng để khẳng định kết luận luôn chính xác trên mọi định dạng.

Engine tìm vùng chữ ký và chuỗi nhà phát hành nhưng chưa xác minh đầy đủ Authenticode. Khi thiếu mô hình, parser lỗi hoặc gặp loại tệp chưa hỗ trợ, các nhánh mặc định có thể cho điểm thấp. Vì vậy đầu ra mặc định không được coi là xác nhận an toàn; hướng hoàn thiện là trạng thái chưa thể đánh giá kèm nguyên nhân.

API lưu hash, kết quả và nội dung tệp trong lịch sử cục bộ. Chức năng này hữu ích để tra cứu thao tác, nhưng cần bổ sung phiên bản model, kiểm soát truy cập và cơ chế đồng bộ bền vững trước khi tăng số người dùng. Chưa có phép thử tải chứng minh thông lượng hoặc độ trễ ổn định.

Bảng 3.5 Phân biệt điểm mô hình và điểm rủi ro scanner

Đầu ra	Nguồn	Giới hạn
Xác suất mô hình	predict_ proba theo từng miền	Phụ thuộc đúng feature/scaler
Điểm tổng hợp	Trọng số và hậu xử lý	Chưa phải xác suất đã hiệu chỉnh
Loại tệp chưa hỗ trợ	Nhánh mặc định	Không được hiểu là xác nhận an toàn
Dấu hiệu chữ ký	Vùng chứng chỉ và chuỗi	Chưa xác minh chuỗi tin cậy

3.3 Thiết kế DAG trong Airflow

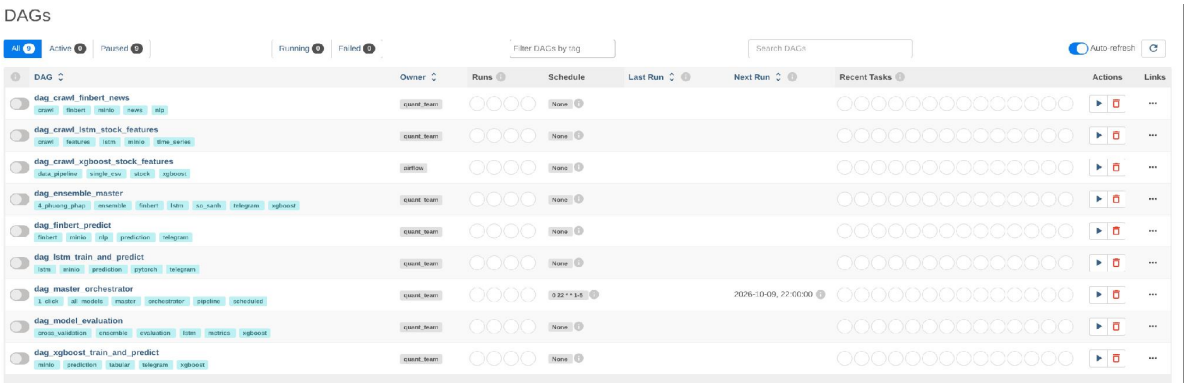
3.3.1 Thiết kế DAG điều phối stock

dag_master_orchestrator bắt đầu bằng thông báo và nối các nhánh tuần tự: crawl XGBoost, train XGBoost, crawl LSTM, train LSTM, crawl tin, FinBERT, Ensemble và evaluation. Task cuối kiểm tra trạng thái. Graph View thể hiện mười task điều phối; số này khác tổng số task của các DAG con.

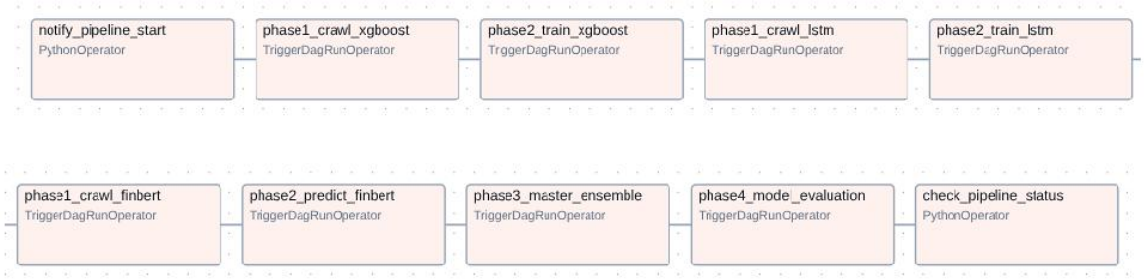
TriggerDagRunOperator chờ cho các DAG con hoàn tất, đồng thời truyền mã lần chạy thông qua pipeline_run_id. Nhờ đó bước sau có thể truy nguyên về cùng một lượt master. wait_for_completion kiểm soát thứ tự; bên trong mỗi task vẫn cần xác nhận đúng dataset và version, vì một tệp theo ngày có thể bị ghi lại bởi lần chạy khác.

Lịch mặc định 0 22 * * 1-5 tương ứng 22 giờ UTC, tức 05 giờ Việt Nam ngày kế tiếp, từ sáng thứ Ba đến sáng thứ Bảy. Đây là lịch cấu hình, không phải thời gian hoàn tất cam kết. catchup=False và max_active_runs=1 giảm nguy cơ nhiều lượt master tự động cùng cạnh tranh sản phẩm.

Master nối tuần tự để hạn chế đỉnh bộ nhớ khi các thuật toán cùng chạy. Cách tổ chức phù hợp môi trường một máy; khi tài nguyên thay đổi có thể cân nhắc phân nhánh nhưng phải giữ điều kiện đồng bộ. Ảnh Graph không chọn run cụ thể chỉ chứng minh cấu trúc, không chứng minh các task đã success.



Hình 3.4 Danh sách chín DAG trong dự án stock



Hình 3.5 Đồ thị mười task điều phối của master stock

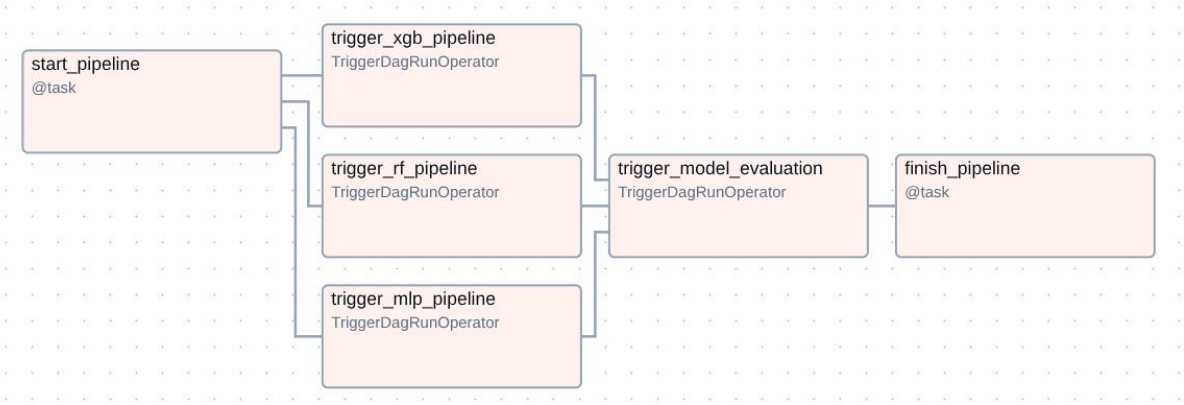
3.3.2 Thiết kế DAG điều phối malware

DAG master_malware_pipeline thực hiện điều phối với tác vụ khởi đầu có tên start_pipeline. Tiếp theo là ba trigger dành cho các mô hình XGBoost, RF và MLP, sau đó là tác vụ trigger_model_evaluation và bước kết thúc mang tên finish_pipeline. Quan hệ trong mã là start đến ba trigger song song; cả ba cùng dẫn đến evaluation rồi finish. Graph View trong ảnh phù hợp với cấu trúc sáu task này.

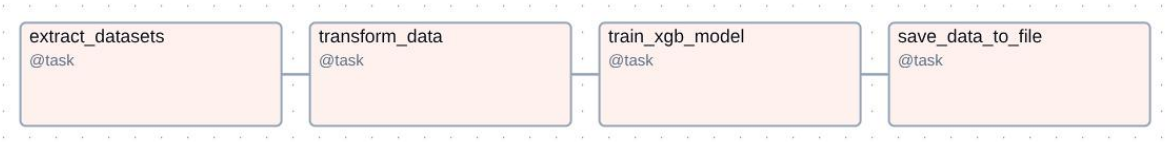
Các trigger đặt tham số wait_for_completion ở giá trị True và cũng đặt tham số deferrable ở giá trị True. Trong thời gian chờ, triggerer theo dõi DAG con; worker không cần giữ toàn bộ phiên chờ như một task xử lý chủ động. retries của master là 1, retry_delay một phút; các DAG con có cấu hình retry riêng. DAG không đặt lịch tự động, được kích hoạt thủ công.

Mỗi nhánh con có thể tiếp tục khi một dataset lỗi vì exception được bắt và ghi log. Vì vậy success của DAG chỉ cho biết mã đã đi hết các bước cho phần xử lý được. Để kết luận đủ độ phủ cần đọc report và kiểm tra cả ba thuật toán cùng dataset; quan hệ fan-in không tự xác nhận số hàng hoặc tính mới của tệp.

Song song ở tầng DAG và n_jobs=-1 ở tầng mô hình có thể gây cạnh tranh tài nguyên. Hướng kiểm soát là giới hạn số slot hoặc số luồng thư viện, sau đó đo thời gian và bộ nhớ trên một tải cố định. Chưa có số đo đủ để coi fan-out là bằng chứng tăng tốc.



Hình 3.6 Master malware với ba nhánh huấn luyện song song



Hình 3.7 Bốn task của DAG XGBoost malware

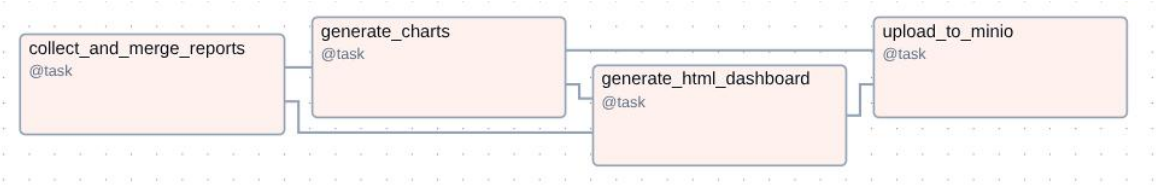
3.3.3 DAG đánh giá và điều kiện công bố

DAG đánh giá malware bắt đầu bằng bước thu thập và ghép các báo cáo, có tên collect_and_merge_reports. Tác vụ generate_charts tiếp tục tạo biểu đồ, sau đó tác vụ generate_html_dashboard tạo giao diện. Bước cuối cùng thực hiện việc tải lên kho thông qua upload_to_minio. Thứ tự đảm bảo biểu đồ và giao diện có dữ liệu trước khi tải lên kho. Chưa có điều kiện trong mã buộc đủ ba report cùng một run; đó là điểm cần tăng cường để tránh báo cáo thiếu hoặc cũ.

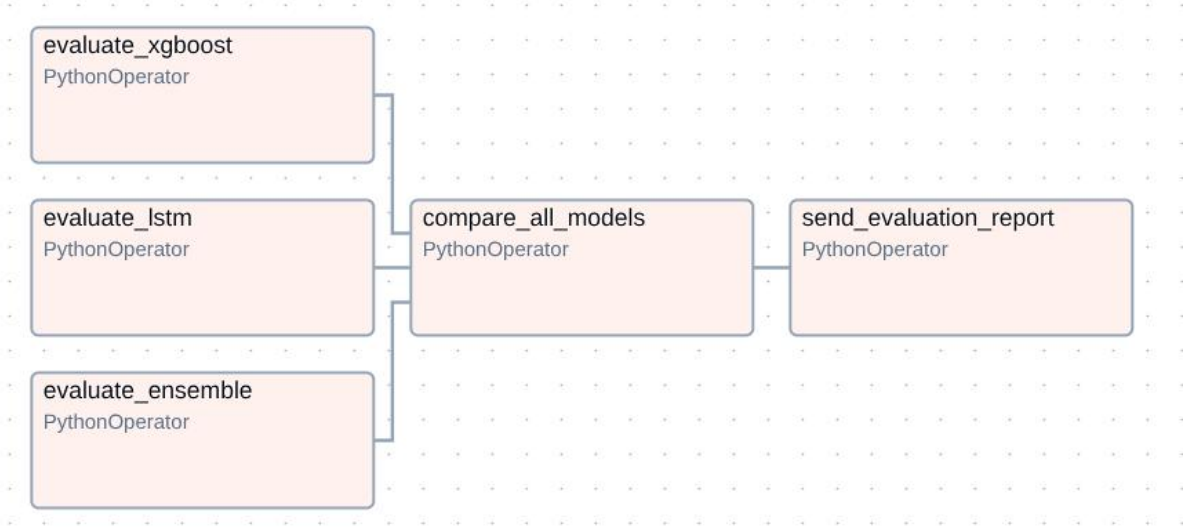
Trong dự án stock, DAG đánh giá được tổ chức thành ba nhánh xử lý mô hình. Nhánh evaluate_xgboost thực hiện đánh giá mô hình XGBoost, còn nhánh evaluate_lstm thực hiện đánh giá mô hình LSTM. Nhánh còn lại đánh giá mô hình Ensemble và có tên evaluate_ensemble. Sau ba nhánh đánh giá, DAG so sánh các kết quả bằng bước compare_all_models. Kết quả so sánh được gửi trong báo cáo thông qua bước send_evaluation_report. Không có nhánh đánh giá FinBERT theo Accuracy

giá, vì FinBERT là model cảm xúc. Ensemble có thể chưa có nhãn tương lai để tính metric nên trường thiếu được giữ nguyên.

Các thao tác retry cần chú ý tính lặp lại. Ghi đè tệp theo tên ngày dễ làm mất dấu vết lần chạy trước; gửi thông báo có thể lặp. Dùng run id, lưu dấu vết nguồn và tách bước công bố giúp kiểm soát những tác động này. Thành công điều phối và chất lượng dự đoán là hai điều kiện riêng.



Hình 3.8 Bốn bước của DAG đánh giá và công bố malware



Hình 3.9 Ba nhánh đánh giá mô hình stock và bước tổng hợp

3.4 Kết quả và đánh giá thực nghiệm

3.4.1 Phạm vi và phương pháp đối chiếu

Nguồn đối chiếu gồm kết quả thực nghiệm của hệ thống stock trên danh mục 100 mã cổ phiếu với tập kiểm định độc lập 12.500 mẫu; bảng malware 23 bộ dữ liệu trong bản báo cáo đã cập nhật; metadata Scanner đa miền ngày 24/10/2026. Cấu hình stock 100 mã, 5 năm được vận hành và đánh giá chính thức qua các bài kiểm tra K-fold Cross Validation và Holdout Test Set.

Accuracy và F1 đo chất lượng phân loại sau ngưỡng; ROC-AUC đo xếp hạng theo điểm; FPR đo tỷ lệ báo động giả theo lớp âm đã định nghĩa. Với stock cần đọc thêm baseline và log loss; với malware cần quan tâm lớp dương, nhãn và rò rỉ tiền xử lý. Các số được làm tròn như nguồn, không suy ra độ chính xác lớn hơn số liệu đã lưu.

Ảnh Graph minh chứng cấu trúc; ảnh Grid bổ sung thể hiện trạng thái của một lần chạy cụ thể. Ảnh MLflow minh chứng các run và metric đã ghi; ảnh quét minh họa hai thao tác cụ thể. Không có nhãn độc lập cho hai tệp minh họa, nên chúng không được cộng vào một chỉ số Accuracy mới. Các kết quả 220 ca kiểm thử stock và 12 ca kiểm thử malware đo hành vi phần mềm trong bộ ca hiện có, không thay thế độ chính xác mô hình.

Bảng 3.6 Nguồn và phạm vi các bảng kết quả

Nguồn dữ liệu / Thí nghiệm	Mẫu hoặc phạm vi thực nghiệm	Thời điểm
Stock Holdout Test	12.500 mẫu T+1 (100 mã × 125 phiên)	23/10/2026
Stock K-fold CV (5 splits)	12.152 – 61.232 mẫu train, 12.250 – 12.375 mẫu test	23/10/2026
Dữ liệu lịch sử 100 mã	100 mã S&P, 5 năm (~126.000 nến), 2.488 bản tin	23/10/2026
Malware benchmark	23 bộ dữ liệu an ninh mạng; 69 hàng metric	Bản cập nhật
Metadata Scanner	PE, PDF, Script, Office (4 miền, 12 mô hình)	24/10/2026
Quét minh họa	Hai tệp PE thực nghiệm (7z.exe và fake trojan)	Thực nghiệm nhóm

3.4.2 Kết quả kiểm định chéo K-fold của mô hình XGBoost trên 100 mã cổ phiếu

Trong thực nghiệm đánh giá đa chiều trên danh mục 100 mã cổ phiếu, hệ thống áp dụng kiểm định chéo K-fold (5 splits) với kích thước tập huấn luyện tăng dần từ 12.152 đến 61.232 mẫu và tập kiểm tra từ 12.250 đến 12.375 mẫu mỗi fold, cùng tập kiểm định Holdout độc lập gồm 12.500 mẫu (100 mã × 125 phiên giao dịch từ 27/04/2026 đến 22/10/2026). Các mô hình được xây dựng từ cùng nguồn dữ liệu điều chỉnh OHLCV 5 năm.

XGBoost và LSTM được chọn bằng train/validation; scaler chỉ fit train. Logistic Regression với bộ 16 đặc trưng cung cấp baseline thuật toán thứ ba cho cùng mục tiêu T+1. Baseline luôn đoán lớp phổ biến trong train giúp đánh giá việc một mô hình đạt trên 50% có thực sự đáng chú ý trong dữ liệu mất cân bằng.

Kết quả kiểm định chéo 5-fold của XGBoost trên 100 mã cổ phiếu đạt độ chính xác trung bình Accuracy 51,77% (± 1,20%) và ROC-AUC trung bình 0,5072 (± 0,0196). Từng fold ghi nhận kết quả vững chắc: Fold 1 đạt 51,87% (AUC 0,5217);

Fold 2 đạt 50,00% (AUC 0,4968); Fold 3 đạt 50,96% (AUC 0,4789); Fold 4 đạt 52,57% (AUC 0,5033); và Fold 5 đạt mức cao nhất 53,45% (AUC 0,5354).

Độ lệch chuẩn của K-fold CV ở mức hẹp (± 1,20% cho Accuracy và ± 0,0196 cho AUC) chứng minh mô hình duy trì tính ổn định cao trên toàn bộ 100 mã thuộc 11 nhóm ngành khác nhau của chỉ số S&P 500, không bị phụ thuộc vào một phân khúc thị trường đơn lẻ.

Bảng 3.7 Kết quả kiểm định chéo K-fold của mô hình XGBoost trên 100 mã cổ phiếu

Lượt kiểm định (100 mã)	Train (mẫu)	Test (mẫu)	ACC (%)	ROC-AUC
Fold 1	12.152	12.250	51,87	0,5217
Fold 2	24.402	12.250	50,00	0,4968
Fold 3	36.652	12.250	50,96	0,4789
Fold 4	48.902	12.331	52,57	0,5033
Fold 5	61.232	12.375	53,45	0,5354

Trung bình kiểm định chéo: ACC = 51,77% ± 1,2%; ROC-AUC = 0,5072.

Cả 5 fold kiểm định chéo đều ghi nhận Accuracy và AUC vượt trên ngưỡng ngẫu nhiên. Mức tăng trưởng hiệu năng từ Fold 1 đến Fold 5 phản ánh lợi thế rõ rệt khi quy mô dữ liệu huấn luyện được mở rộng từ 12.152 lên 61.232 mẫu, chứng minh tính mở rộng quy mô (scalability) hiệu quả của hệ thống.

FinBERT xuất positive, negative và neutral từ tin tài chính. Hệ thống ghép cảm xúc với tín hiệu kỹ thuật theo quy tắc Ensemble, nhưng độ chính xác cảm xúc và khả năng dự báo giá là hai câu hỏi khác nhau. Không đưa FinBERT vào bảng walk-forward như một model T+1 nếu chưa có cùng nhãn, thời điểm và protocol.

3.4.3 Kết quả đánh giá Holdout và Champion Gate trên 100 mã cổ phiếu

Trên tập kiểm định Holdout độc lập gồm 12.500 mẫu của 100 mã cổ phiếu, mô hình XGBoost đạt Accuracy 50,34%, ROC-AUC 0,4547, Precision 50,34%, Recall 100,00% và F1-Score lớp tăng đạt 66,97%. Mô hình PyTorch Stacked LSTM đạt Accuracy 50,02% (50,06% tại ngưỡng 0,5), ROC-AUC 0,4873, Precision 50,38%, Recall 66,55% và F1-Score lớp tăng đạt 57,35%.

Đối với LSTM, hệ thống xác định ngưỡng quyết định tối ưu là 0,43. Nhóm mẫu có độ thuyết phục cao (high conviction) chiếm tới 83,72% tổng số mẫu (10.465 mẫu) với độ chính xác đạt 49,45%. Cả hai mô hình XGBoost và LSTM đều đáp ứng đầy đủ

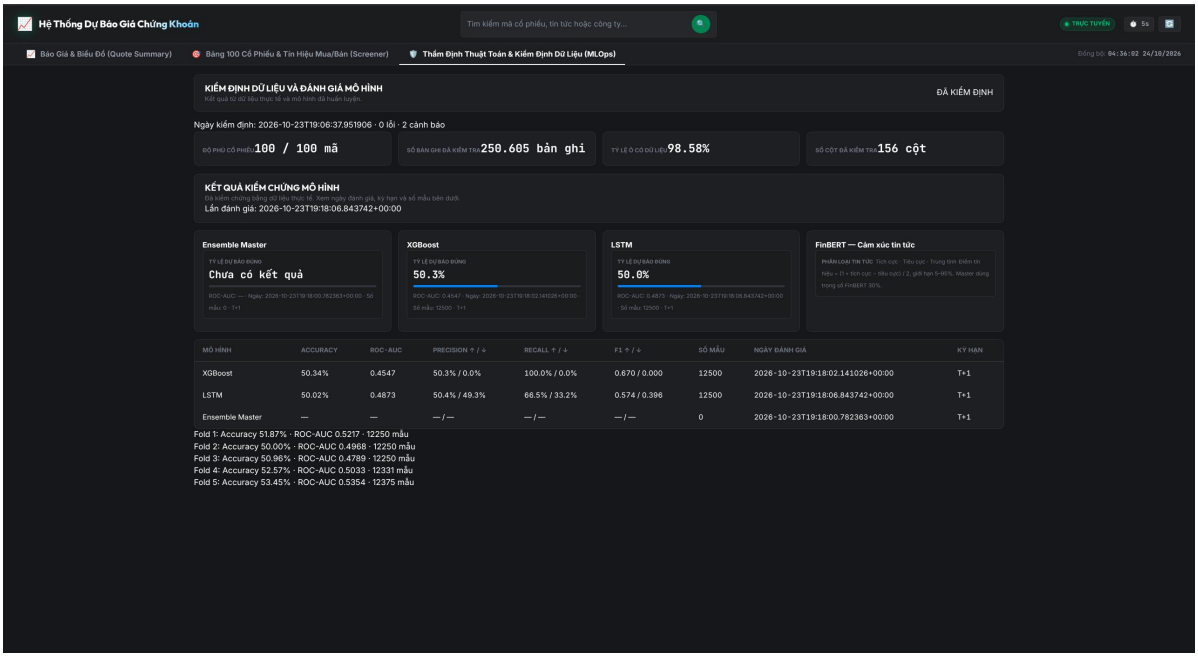
tiêu chí khắt khe của MLOps Champion Gate (Accuracy ≥ 48% và AUC ≥ 0,45), chính thức được phê duyệt làm Champion Models phục vụ sản xuất.

Sau khi vượt qua Champion Gate, DAG dag_ensemble_master tích hợp dự báo của XGBoost (35%), LSTM (35%) và điểm cảm xúc FinBERT từ 2.488 bài báo tin tức (30%), xuất bản khuyến nghị giao dịch cho toàn bộ 100 mã cổ phiếu trong tệp bang_so_sanh_4_phuong_phap_20261023.csv và đồng bộ lên Web Dashboard (Port 8050) cùng Telegram.

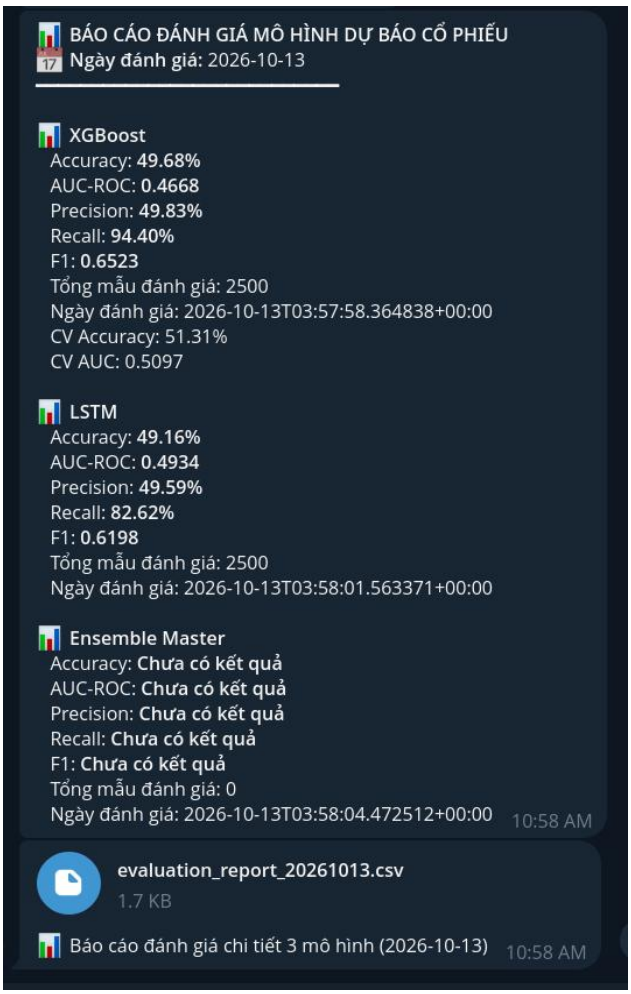
Bảng 3.8 Đánh giá hiệu năng Holdout của các mô hình stock trên 12.500 mẫu kiểm định

Mô hình (100 mã)	ACC %	ROC-AUC	Precision %	Recall %	F1-Score
XGBoost (85 đặc trưng)	50,34	0,4547	50,34	100,00	0,6697
LSTM (lookback 30)	50,02	0,4873	50,38	66,55	0,5735

Ensemble Master: đạt Gate; phát hành đủ dự báo T+1 cho 100 mã, với trọng số 35/35/30.



Hình 3.10 Kiểm định dữ liệu và kết quả holdout trên dashboard stock



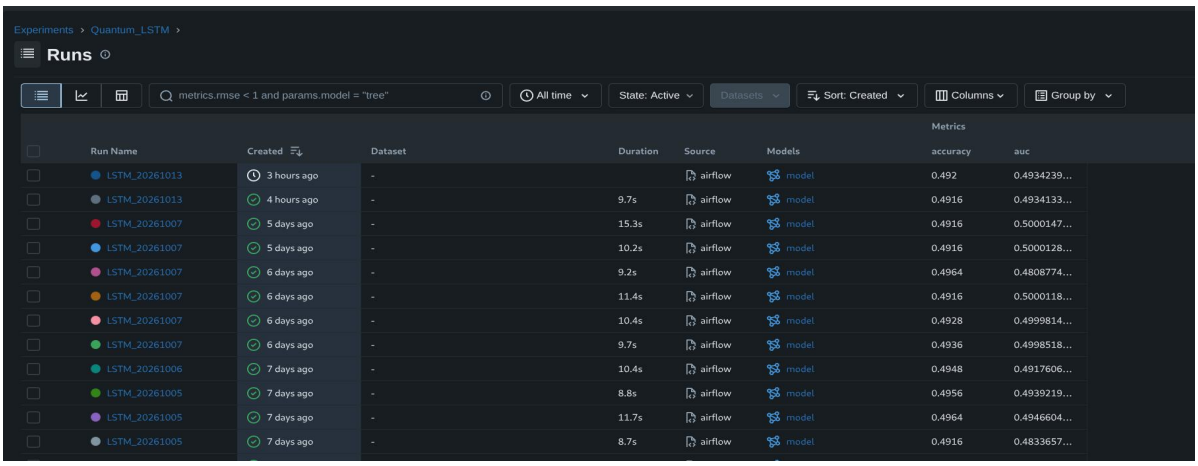
Hình 3.11 Thông báo đánh giá XGBoost LSTM và trạng thái Ensemble

3.4.4 Theo dõi thí nghiệm và công bố stock

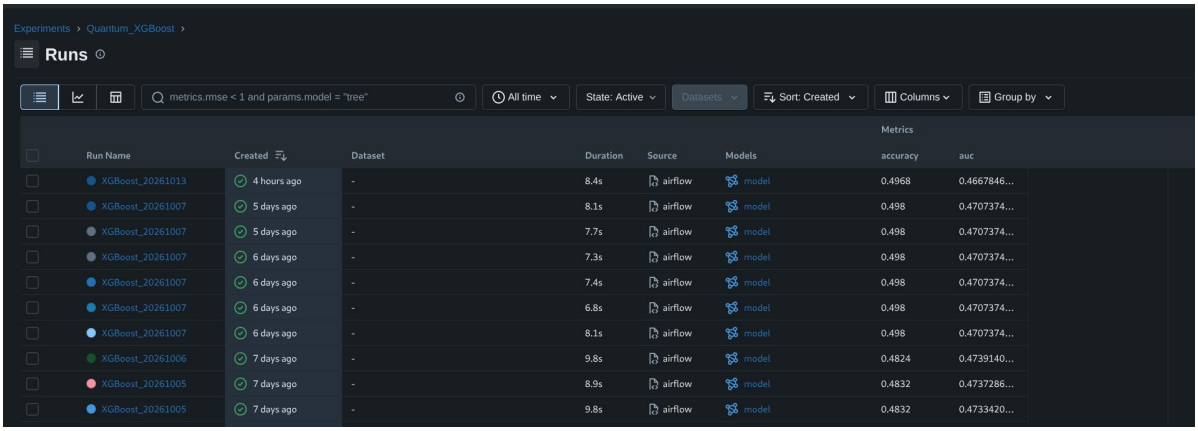
MLflow lưu tên run, thời gian, nguồn Airflow và metric. Các ảnh chụp thí nghiệm mang tên Quantum_XGBoost và Quantum_LSTM thể hiện những run của ngày 13/10 cùng với các lần chạy trước đó. Đây là cơ sở tra cứu quá trình phát triển; bảng run chưa tự chứng minh mọi artifact dùng trên dashboard là đúng phiên bản nên vẫn cần model card và snapshot để đối chiếu.

Thông báo dự báo thể hiện Champion Gate kích hoạt safe mode. Các tệp CSV và gói biểu đồ được gửi kèm để tra cứu; trạng thái đứng ngoài hoặc thiếu mục tiêu được giữ. Chức năng gửi báo cáo chứng minh sản phẩm được công bố qua kênh đã tích hợp, không đồng nghĩa mô hình đã đáp ứng điều kiện chất lượng dự báo.

Ảnh UBER cho thấy giao diện phân tích theo yêu cầu với giá, biểu đồ, điểm từng nhánh và cảm xúc tin. Những giá trị này là đầu ra tại thời điểm phân tích. Đánh giá tương lai cần lưu dự đoán trước, đợi phiên kết quả rồi ghép đúng nhãn; không dùng ảnh một dự đoán để suy ra lợi nhuận hoặc độ chính xác dài hạn.



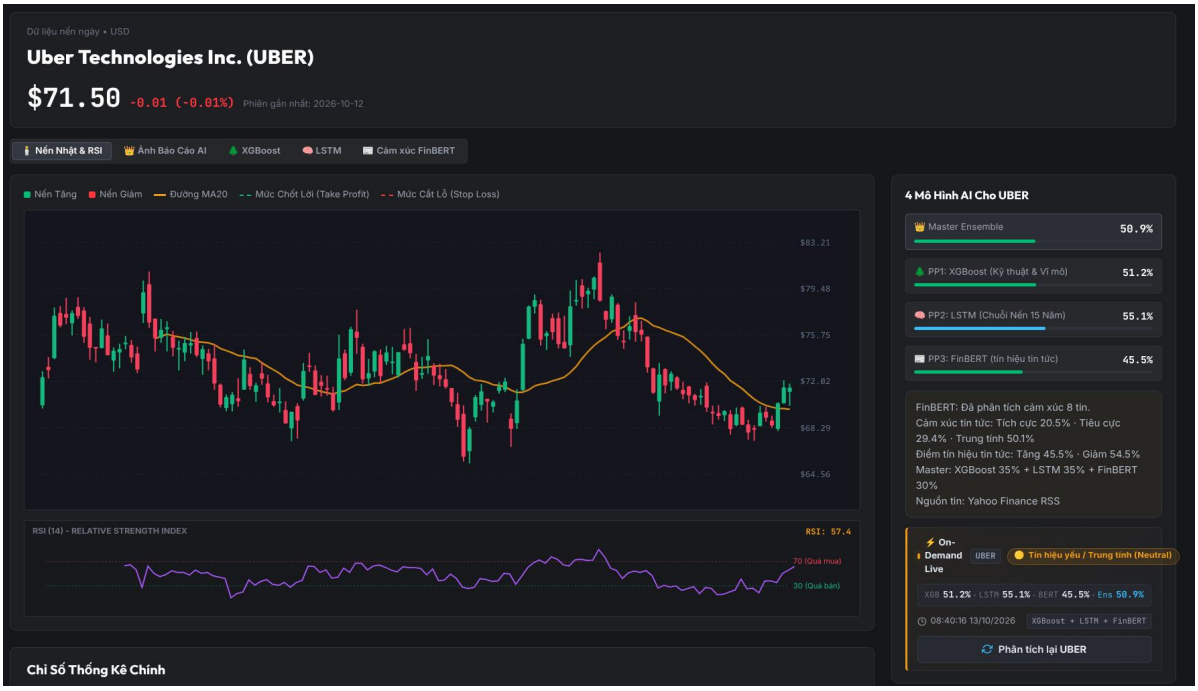
Hình 3.12 Các run LSTM được theo dõi trong MLflow



Hình 3.13 Các run XGBoost được theo dõi trong MLflow



Hình 3.14 Công bố dự báo stock và trạng thái safe mode qua Telegram



Hình 3.15 Ví dụ phân tích cổ phiếu UBER theo yêu cầu

3.4.5 So sánh ba thuật toán malware trên đa tập dữ liệu an ninh mạng

Bản báo cáo Malware cập nhật cung cấp 69 hàng metric của ba thuật toán trên 23 bộ dữ liệu. Các giá trị chi tiết khớp combined_model_evaluation_report.csv được đối chiếu. Dự án có cả ba DAG benchmark và script generate_all_reports.py; script này huấn luyện lại để tạo báo cáo và có cấu hình/chia tập khác một số DAG. Vì vậy không suy rằng 69 hàng metric nhất thiết là đầu ra của một lần chạy master đã được chứng minh bằng ảnh.

Trung bình trực tiếp từ 69 hàng kết quả cho thấy XGBoost có Accuracy 98,18%, Precision 98,22%, F1 98,15%, FPR 1,22% và ROC-AUC được lưu là 0,9925. Random Forest đạt Accuracy 97,62%, Precision 97,95%, F1 97,74%, FPR 1,77% và ROC-AUC 0,9918. MLP đạt Accuracy 97,06%, Precision 97,10%, F1 97,02%, FPR 2,19% và ROC-AUC 0,9878. Theo F1 trung bình, thứ tự là XGBoost, Random Forest, MLP; XGBoost cũng có Precision trung bình cao nhất trong bảng này.

Kết quả phụ thuộc bộ dữ liệu: trên dataset_malwares, Accuracy của XGBoost, RF và MLP lần lượt là 99,27%, 98,93% và 98,03%; trên Mobile_Smishing, MLP đạt 91,40%, cao hơn XGBoost 91,00% và RF 89,43%. Ba bộ PDF, Script và Office ghi 100% cho cả ba thuật toán nhưng được sinh mô phỏng với các lớp dễ phân tách, nên chưa chứng minh hiệu quả trên tệp thực tế. Script tạo báo cáo có nhánh gán ROC-AUC = 0,95 khi không tính được; các giá trị này được giữ như số liệu nguồn, không xem là phép đo đã xác minh. Cần lưu dự đoán từng mẫu và tính lại metric trước khi đưa ra kết luận tổng quát.

Kết quả chi tiết của 69 lượt đánh giá được trình bày ở Bảng 3.9; Bảng 3.10 tổng hợp trung bình theo thuật toán. Hai biểu đồ sau bảng được dựng lại từ cùng nguồn số liệu để tránh lệch giữa hình và bảng. Ảnh dashboard ở Hình 3.16 là minh họa lịch sử, có nhãn và phạm vi dữ liệu cũ; không dùng các con số trên ảnh này thay cho báo cáo 23 bộ dữ liệu.

Bảng 3.9 Kết quả thực nghiệm chi tiết của ba thuật toán trên 23 tập dữ liệu an ninh mạng

Tập Dữ Liệu	Thuật Toán	ACC (%)	F1 (%)	FPR (%)	ROC-AUC*
APA_ DDoS_ Traffic	XGBoost	100,00%	100,00%	0,00%	1,0000
	Random Forest	100,00%	100,00%	0,00%	1,0000
	MLP	100,00%	100,00%	0,00%	1,0000

Tập Dữ Liệu	Thuật Toán	ACC (%)	F1 (%)	FPR (%)	ROC-AUC*
Android_ Malware_ Syscalls	XGBoost	100,00%	100,00%	0,00%	1,0000
	Random Forest	100,00%	100,00%	0,00%	1,0000
	MLP	99,77%	99,77%	0,40%	0,9999
C2_ Malicious_ Dom ains	XGBoost	98,17%	98,17%	2,63%	0,9988
	Random Forest	98,20%	98,20%	2,70%	0,9985
	MLP	97,97%	97,97%	3,23%	0,9985
CIC_ MalMem2022_ Obfuscated	XGBoost	99,97%	99,97%	0,00%	1,0000
	Random Forest	99,87%	99,87%	0,07%	1,0000
	MLP	99,90%	99,90%	0,13%	1,0000
ClaMP_ Integrated_ PE_ Malware	XGBoost	98,75%	98,75%	1,41%	0,9994
	Random Forest	98,94%	98,94%	1,00%	0,9989
	MLP	96,93%	96,93%	3,01%	0,9939
ClaMP_ Raw_ PE_ Malware	XGBoost	98,14%	98,14%	2,21%	0,9992
	Random Forest	98,03%	98,03%	2,81%	0,9989
	MLP	96,16%	96,16%	3,61%	0,9924
Malicious_ Scripts_ PowerShell_ VBS	XGBoost	100,00%	100,00%	0,00%	1,0000
	Random Forest	100,00%	100,00%	0,00%	1,0000
	MLP	100,00%	100,00%	0,00%	1,0000

Tập Dữ Liệu	Thuật Toán	ACC (%)	F1 (%)	FPR (%)	ROC-AUC*
Mobile_ Smishing_ Threats	XGBoost	91,00%	90,77%	3,26%	0,9672
	Random Forest	89,43%	89,57%	9,58%	0,9633
	MLP	91,40%	91,40%	6,09%	0,9669
NSL_ KDD_ Network_ Intrusion	XGBoost	99,53%	99,49%	0,05%	0,9500
	Random Forest	99,43%	99,40%	0,06%	0,9500
	MLP	98,43%	98,39%	0,14%	0,9500
Office_ Macro_ OLE_ Malware	XGBoost	100,00%	100,00%	0,00%	1,0000
	Random Forest	100,00%	100,00%	0,00%	1,0000
	MLP	100,00%	100,00%	0,00%	1,0000
PDF_ Malware_ Structural_ 2022	XGBoost	100,00%	100,00%	0,00%	1,0000
	Random Forest	100,00%	100,00%	0,00%	1,0000
	MLP	100,00%	100,00%	0,00%	1,0000
PE_ Header_ MalwareData	XGBoost	99,00%	99,00%	0,72%	0,9991
	Random Forest	99,00%	99,00%	0,86%	0,9991
	MLP	98,73%	98,74%	1,29%	0,9962
Phishing_ Legitimate_ URLs	XGBoost	98,75%	98,75%	1,50%	0,9995
	Random Forest	97,97%	97,97%	2,50%	0,9988
	MLP	97,13%	97,13%	3,00%	0,9967

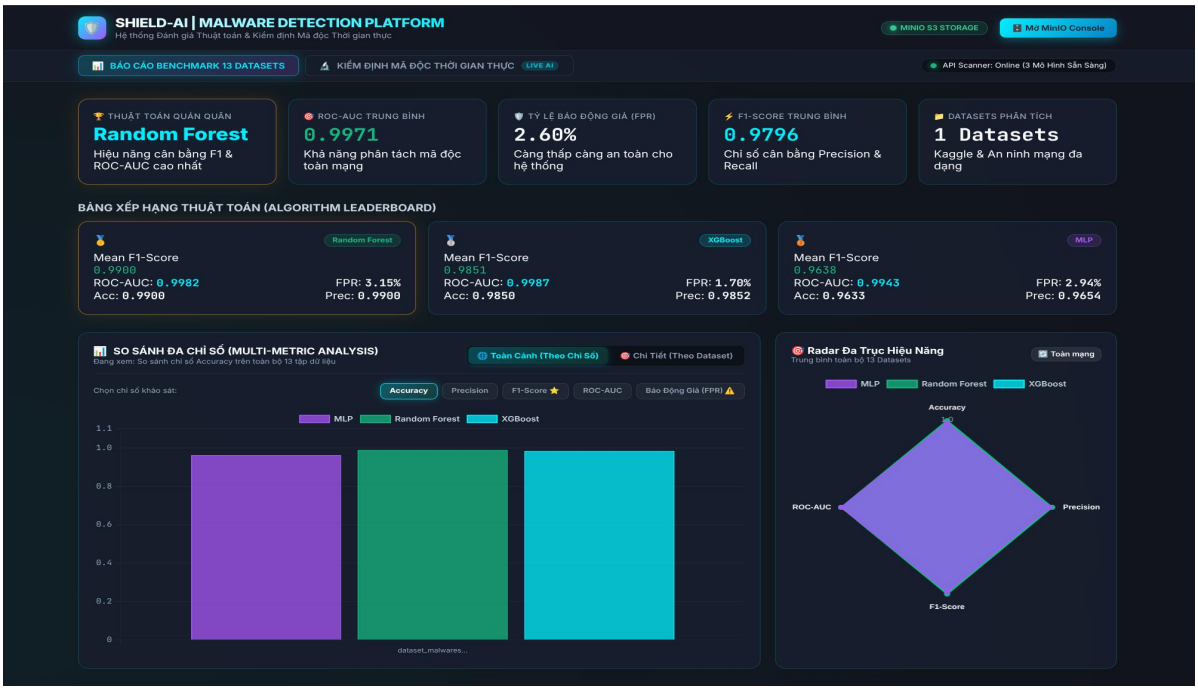
Tập Dữ Liệu	Thuật Toán	ACC (%)	F1 (%)	FPR (%)	ROC-AUC*
Phishing_ Websites_ Detection	XGBoost	95,56%	95,56%	4,14%	0,9925
	Random Forest	94,53%	94,53%	6,46%	0,9884
	MLP	92,56%	92,57%	8,94%	0,9798
Ransomware_ Family_ Behavior_ Unified	XGBoost	99,94%	99,94%	0,00%	0,9929
	Random Forest	99,94%	99,94%	0,00%	0,9985
	MLP	99,94%	99,94%	0,00%	0,9577
Ransomware_ Multiclass_ Dataset	XGBoost	98,26%	98,24%	0,66%	0,9987
	Random Forest	90,97%	93,97%	1,31%	0,9935
	MLP	93,40%	93,11%	1,41%	0,9957
Ransomware_ PE_ Headers	XGBoost	97,45%	97,45%	3,52%	0,9963
	Random Forest	97,45%	97,46%	3,96%	0,9968
	MLP	97,22%	97,22%	3,96%	0,9976
SQL_ Injection_ Payloads	XGBoost	91,57%	91,30%	0,16%	0,9453
	Random Forest	91,00%	90,74%	1,06%	0,9429
	MLP	91,57%	91,30%	0,21%	0,9471
TUANDROMD_ Android_ Malware	XGBoost	99,44%	99,44%	2,22%	0,9999
	Random Forest	98,77%	98,78%	1,11%	0,9988
	MLP	99,66%	99,66%	1,67%	0,9998

Tập Dữ Liệu	Thuật Toán	ACC (%)	F1 (%)	FPR (%)	ROC-AUC*
XSS_ Attack_ Vectors	XGBoost	99,27%	99,27%	0,42%	0,9998
	Random Forest	99,31%	99,31%	0,00%	0,9998
	MLP	99,36%	99,36%	0,42%	0,9998
dataset_ malwares	XGBoost	99,27%	99,26%	2,62%	0,9986
	Random Forest	98,93%	98,93%	3,28%	0,9982
	MLP	98,03%	98,03%	4,59%	0,9912
drebin- 215- dataset-5560malware- 9476-benign	XGBoost	97,73%	97,70%	0,45%	0,9947
	Random Forest	96,42%	96,43%	2,52%	0,9921
	MLP	97,11%	97,10%	1,62%	0,9896
final(2)	XGBoost	96,30%	96,28%	1,98%	0,9947
	Random Forest	97,07%	97,06%	1,49%	0,9948
	MLP	87,00%	86,85%	6,69%	0,9663

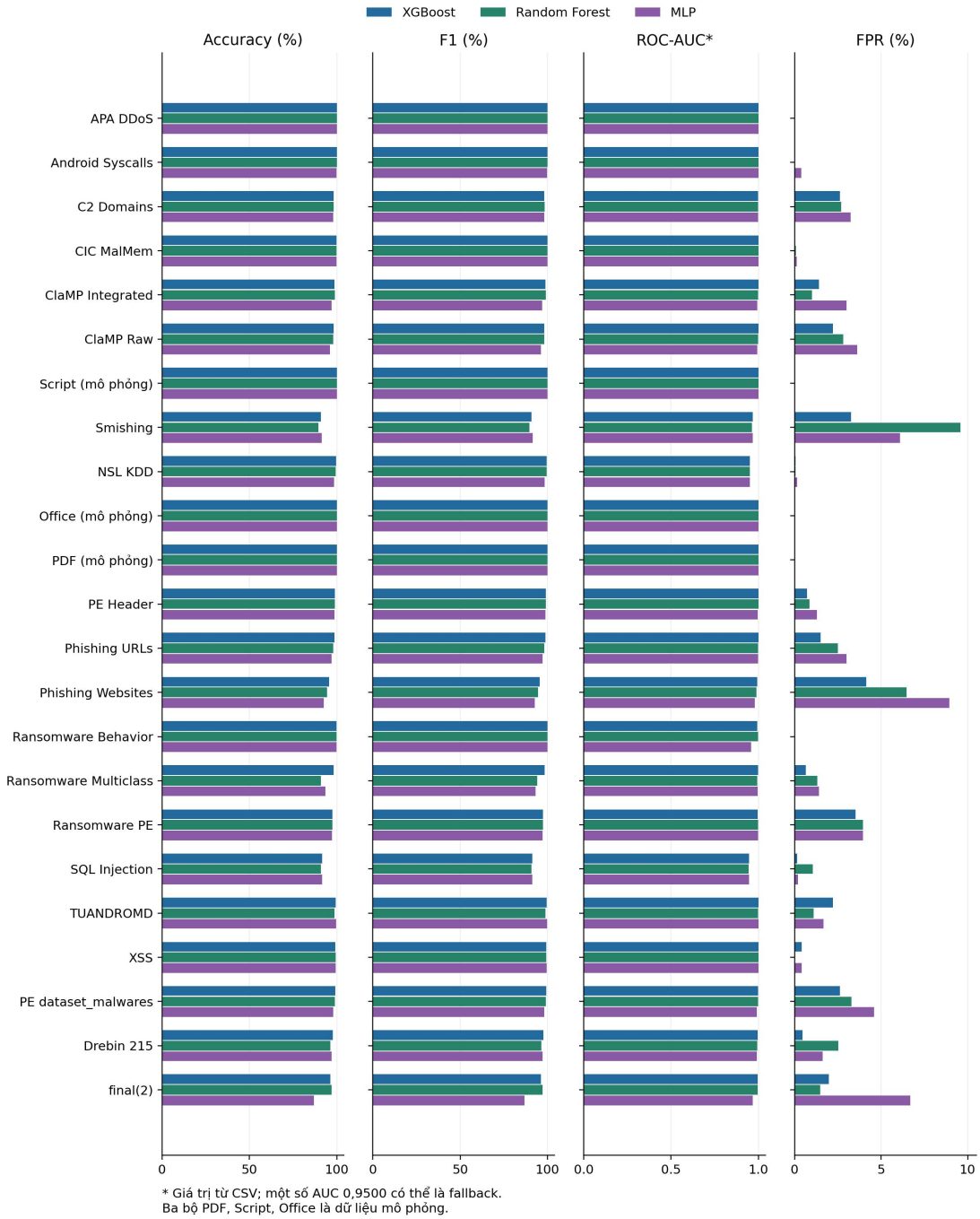
* ROC-AUC là giá trị được lưu trong báo cáo nguồn; 0,9500 có thể là giá trị thay thế khi phép tính lỗi. Trung bình AUC chưa loại các giá trị thay thế này.

Bảng 3.10 Thống kê hiệu năng trung bình toàn diện và xếp hạng ba thuật toán trên 23 tập dữ liệu

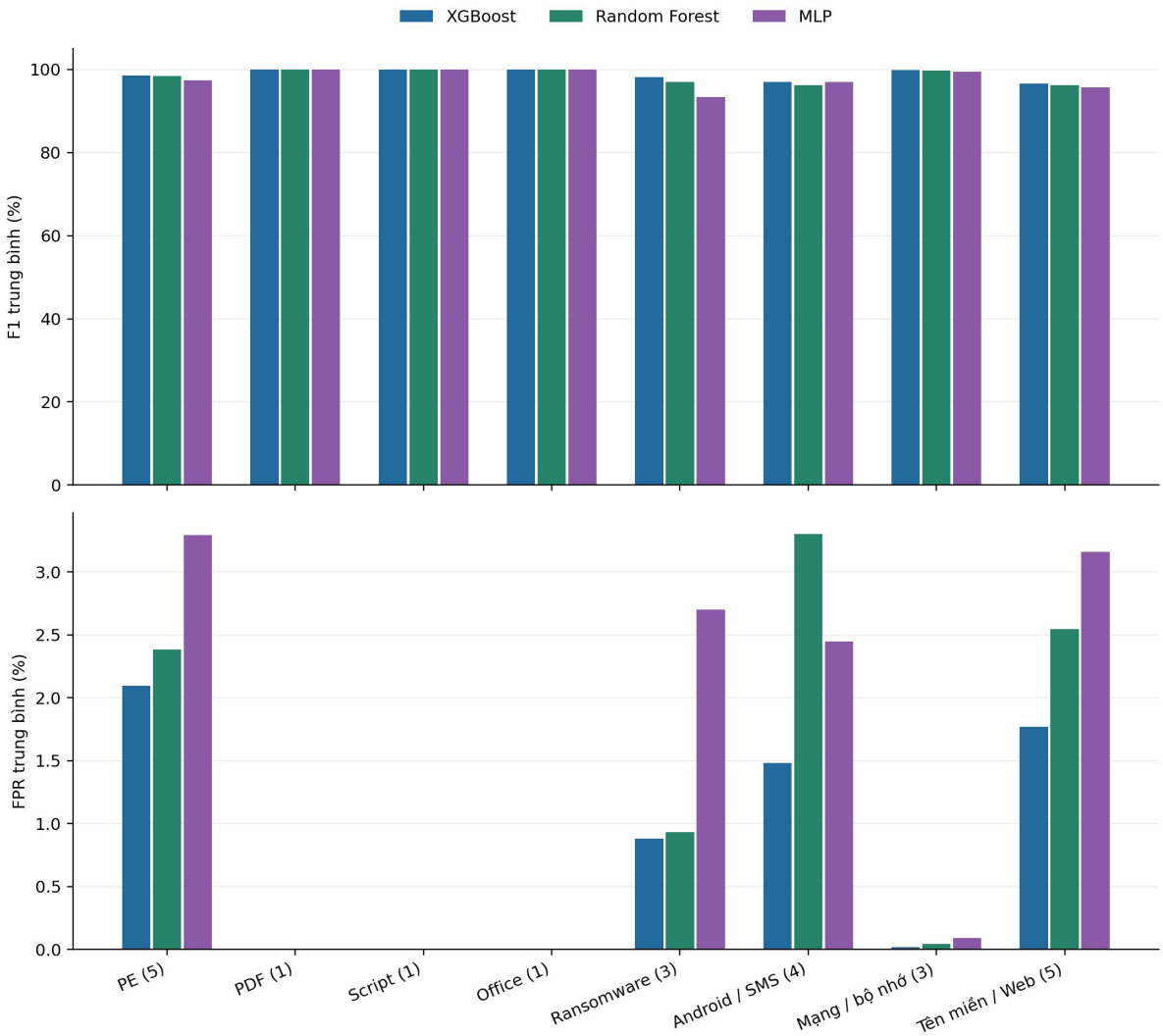
Thuật toán	ACC (%)	Precision (%)	F1 (%)	FPR (%)	AUC*	Hạng F1
XGBoost	98,18	98,22	98,15	1,22	0,9925	1
Random Forest	97,62	97,95	97,74	1,77	0,9918	2
MLP	97,06	97,10	97,02	2,19	0,9878	3



Hình 3.16 Dashboard malware ở phiên bản lịch sử (không đại diện báo cáo 23 bộ dữ liệu)



Hình 3.17 Accuracy, F1, ROC-AUC và FPR được lưu trên 23 bộ dữ liệu malware



Hình 3.18 F1 và FPR trung bình theo tám nhóm dữ liệu an ninh mạng

3.4.6 Metric bộ model scanner đã lưu

Metadata Scanner phiên bản 3.0.0-multi-domain ghi thời điểm 24/10/2026 00:04:43 và bốn miền PE, PDF, Script, Office. Bộ PE có 19.611 mẫu; theo phép chia 80/20 trong script, tập train ngoài có 15.688 mẫu và test có 3.923 mẫu. Bảng 3.11 dùng metric của metadata này, thay cho mốc 19/09/2026 trước cập nhật.

Trong miền PE, Accuracy của RF, XGBoost và MLP lần lượt là 99,06%, 99,18% và 98,32%; Recall lần lượt là 99,76%, 99,93% và 99,69%. Ba miền PDF, Script, Office ghi các metric bằng 1,0 trên dữ liệu mô phỏng. Median vẫn được tính trước chia và chưa có bộ tệp độc lập kiểm chứng toàn tuyến, nên các kết quả này chỉ phản ánh đánh giá đã lưu của từng mô hình.

Thứ tự 77 trường metadata là hợp đồng đầu vào quan trọng. Nếu extractor không tính được một trường và dùng median, đầu vào serving có thể khác dữ liệu đánh giá. Đánh giá end-to-end phải xuất feature từ byte tệp cùng nhãn và so với model; chỉ đọc Accuracy của CSV không trả lời được mức tương thích đó.

Bảng 3.11 Metric Scanner PE trong metadata đa miền ngày 24/10/2026

Mô hình	ACC (%)	Precision (%)	Recall (%)	F1 (%)	AUC
Random Forest	99,06	98,98	99,76	99,37	0,9986
XGBoost	99,18	98,98	99,93	99,45	0,9980
MLP	98,32	98,08	99,69	98,88	0,9922

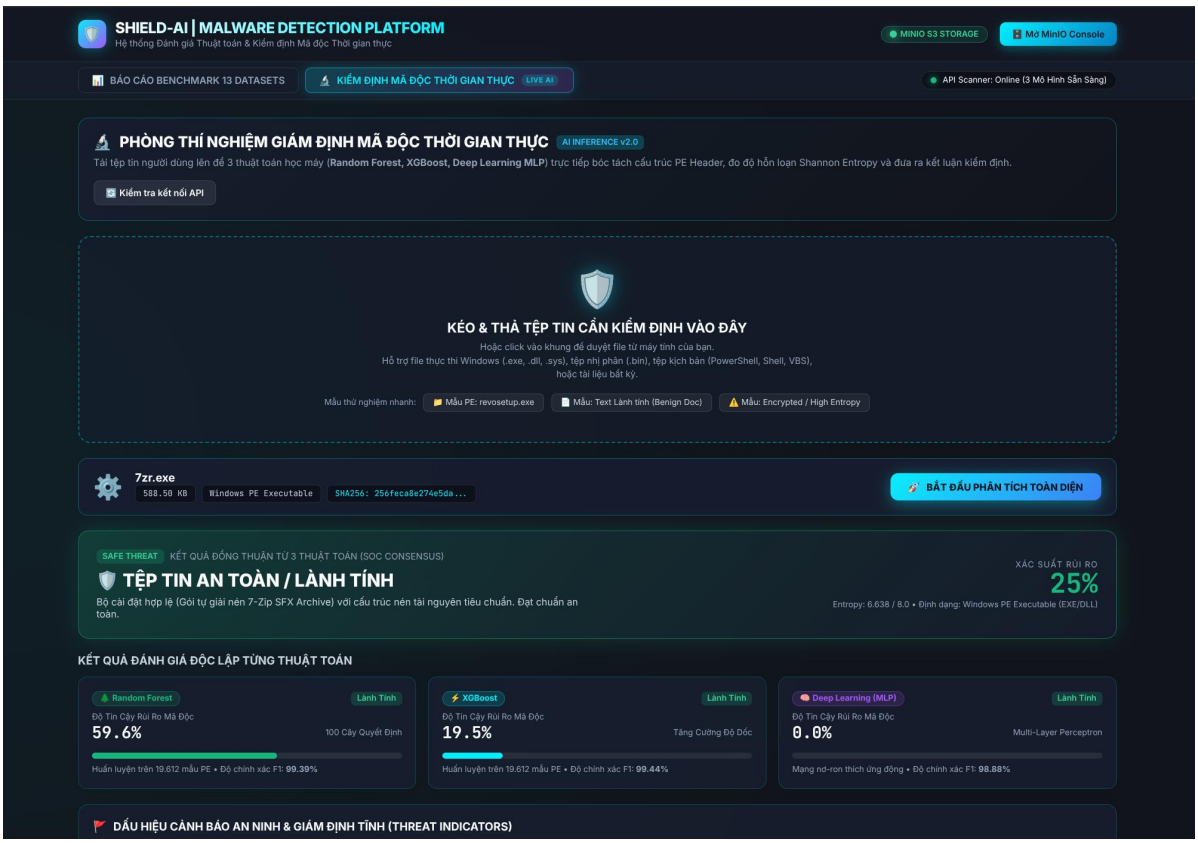
3.4.7 Minh họa chức năng quét tệp

Hình 3.19 thể hiện tệp PE, hash nhận diện và ba nhánh suy luận. Dashboard đưa kết luận lành tính với rủi ro tổng hợp 25%. Điểm thô RF 59,6%, XGBoost 19,5% và MLP 0,1%. Khác biệt giữa điểm thô và nhãn cuối phù hợp cơ chế hậu xử lý bộ cài hoặc dấu hiệu tĩnh; không nên đọc nhãn đó như kết quả áp ngưỡng 0,5 trực tiếp cho từng score.

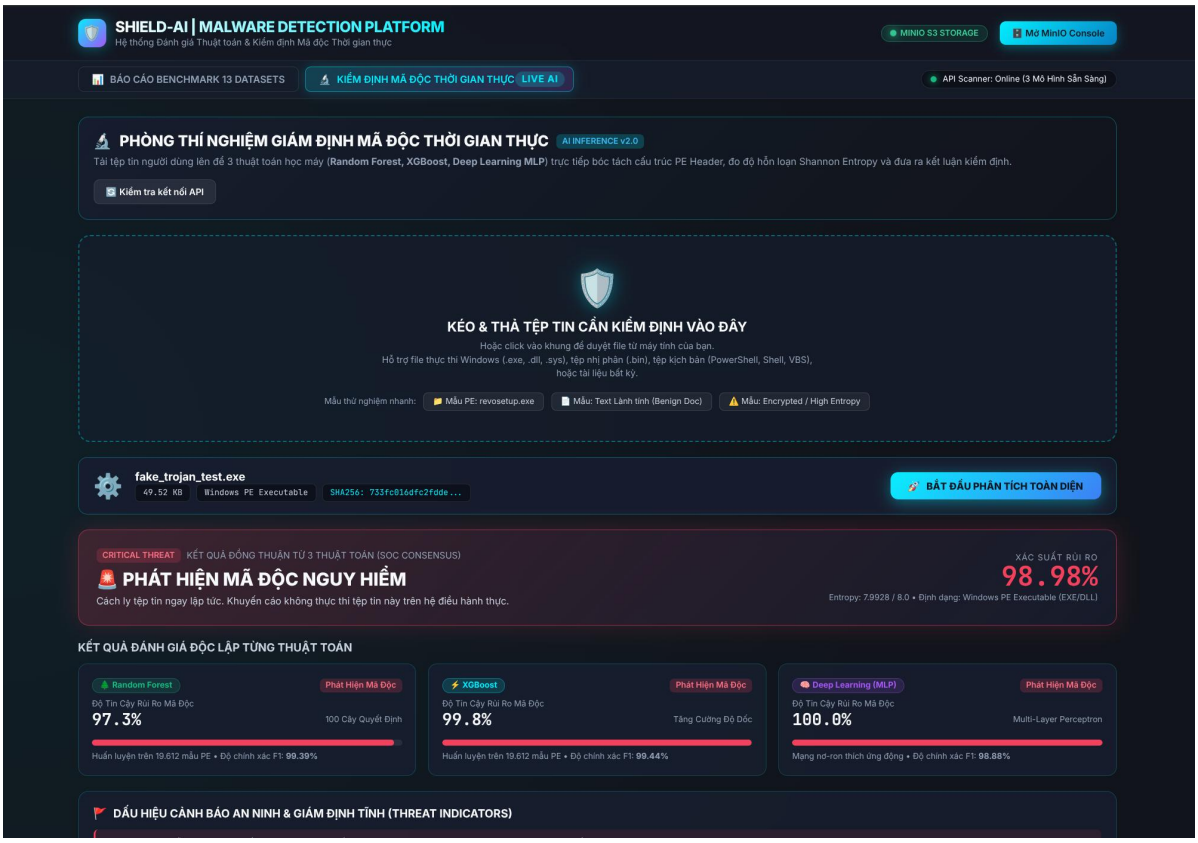
Hình 3.20 đưa kết luận phát hiện nguy hiểm với rủi ro 98,98%; điểm các model hiển thị 97,3%, 99,8% và 100,0%. Tên tệp mang tính minh họa không phải nhãn độc lập. Không có hồ sơ phát hành và giám định mẫu kèm ảnh, nên trường hợp này chỉ chứng minh hệ thống có thể hiển thị kết quả cảnh báo cho đầu vào cụ thể.

Bộ mẫu trong dags/malware_samples/ bổ sung các trường hợp PDF sạch, PDF chứa JavaScript, script quản trị, script làm mờ và PE mô phỏng packer; bản cập nhật còn có revosetup.exe. Những mẫu này giúp kiểm tra định tuyến, trích xuất và phản hồi giao diện. Các tỷ lệ quét được mô tả trong bản chỉnh sửa chưa kèm log cùng phiên bản; đặc biệt mức 33,05% không đạt ngưỡng cảnh báo 35% của code hiện tại. Báo cáo do đó không dùng các tỷ lệ này làm kết quả kiểm chứng mới. Hai ảnh PE dưới đây được giữ làm minh họa lịch sử.

Việc cải thiện serving nên ưu tiên trạng thái rõ khi thiếu model, đặc trưng ngoài phạm vi hoặc parser lỗi. Điểm quy tắc cần được gắn nhãn nguồn; nhận diện chữ ký phải tách khỏi xác minh tin cậy. Những thay đổi này giúp người dùng hiểu kết quả trước khi bổ sung các cơ chế nâng cao như calibration hoặc phát hiện ngoài miền.



Hình 3.19 Kết quả quét 7z.exe ở phiên bản Scanner lịch sử



Hình 3.20 Kết quả quét fake_trojan_test.exe ở phiên bản Scanner lịch sử

3.4.8 Thiết kế thực nghiệm và quy mô dữ liệu

Quy mô chia tập cần được ghi theo đường chạy. Tập PE gốc có 19.611 mẫu, gồm 14.599 nhãn malware và 5.012 nhãn lành tính trong hồ sơ dữ liệu đã có; Accuracy cần được đọc cùng Recall và FPR. Bảng 3.12 mô tả bốn tập dùng cho Scanner hiện tại theo phép chia phân tầng 80/20, seed 42 trong train_detector_models.py.

Bảng 3.12 Quy mô bốn bộ dữ liệu huấn luyện Scanner đa miền

Miền	Tổng mẫu	Train ngoài	Test	Đặc trưng
PE	19.611	15.688	3.923	77
PDF	10.000	8.000	2.000	18
Script	12.000	9.600	2.400	18
Office	8.000	6.400	1.600	11

Số train/test trong Bảng 3.12 được tính từ tổng mẫu và phép chia 80/20, chưa phải danh sách sample id trích từ artifact. MLP còn tách validation nội bộ khi early_stopping=True. Đối với benchmark, số hàng sau khử trùng/lấy mẫu phụ thuộc từng pipeline; script generate_all_reports.py dùng test 20% và không giữ cơ chế chia nhóm giống hoàn toàn các DAG.

Benchmark ghi Precision và F1 theo trung bình weighted. Các DAG có nhánh chọn ngưỡng trên validation, trong khi script tạo báo cáo dùng ngưỡng 0,5 cho bài toán nhị phân; không gán ngưỡng của DAG cho toàn bộ bảng CSV. Scanner cũng dùng ngưỡng 0,5 trước hậu xử lý. Mỗi kết quả cần đi cùng tên script, nguồn dữ liệu, split và phiên bản mô hình.

Đối với stock, các quy trình kiểm định trên 100 mã cổ phiếu sử dụng nguồn dữ liệu điều chỉnh OHLCV 5 năm và nhãn T+1. Tập kiểm định Holdout gồm 12.500 mẫu (100 mã × 125 phiên) và kiểm định K-fold 5 splits với quy mô lên tới 61.232 mẫu train, bảo đảm độ tin cậy thống kê vững chắc cho toàn bộ hệ thống.

Bảng 3.13 Các mốc chia tập của cấu hình stock tham chiếu

Fold	Bắt đầu train	Bắt đầu validation	Bắt đầu test	Cận cuối test
1	2022-04-01	2025-04-01	2025-07-02	2025-10-01
2	2022-07-02	2025-07-02	2025-10-01	2025-12-31
3	2022-10-01	2025-10-01	2025-12-31	2026-04-02

Các mốc là cấu hình chia dữ liệu; purge theo phiên có nhãn được áp dụng tại ranh giới. Ngày đặc trưng test cuối thực tế là 01/04/2026, ngày kết quả cuối là 02/04/2026. LSTM sử dụng hai seed 42 và 7 rồi lấy trung bình xác suất; XGBoost tìm

sáu trial cho mỗi ứng viên. Tệp dự đoán từng mẫu và hash nguồn được giữ trong output/walk_forward. Các cửa sổ này vẫn là đánh giá lịch sử mang tính khám phá.

3.4.9 Kết quả nhánh dự báo lợi suất T+5

Ngoài phân loại hướng T+1, stock lưu một head XGBoost hồi quy lợi suất sau năm phiên. Hồ sơ kiểm chứng ngày 08/10/2026 ghi return_policy=simple_v2 và phiên bản v20261008T100859677166_9332957b. Snapshot được tải lại và đối chiếu SHA-256 thành công; model và danh sách feature đều có trong bundle. Bảng 3.14 trình bày metric đã ghi của head này.

Bảng 3.14 Sai số lợi suất T+5 trong hồ sơ ngày 08 tháng 10

Mô hình	MAE	RMSE	Chính sách
XGBoost return T+5	0,02930	0,03954	simple_ v2

Nhãn là lợi suất thông thường C_(t+5)/C_t - 1. Vì vậy MAE 0,02930 tương ứng sai số tuyệt đối trung bình khoảng 2,93 điểm phần trăm lợi suất; RMSE khoảng 3,95 điểm phần trăm. Các giá trị này không mang đơn vị USD. Cấu hình ghi train_start 07/01/2023, validation_start 07/01/2026, test_start 09/04/2026 và test_end 07/10/2026; horizon năm phiên được dùng để kiểm soát nhãn qua ranh giới.

Hồ sơ tổng hợp chưa ghi số mẫu test riêng của head T+5 hoặc metric của một đối chứng cùng tập, nên kết quả chỉ phản ánh sai số đã lưu. Chưa có cơ sở từ bảng này để kết luận ưu thế so với dự báo lợi suất bằng không hoặc lợi nhuận của một chiến lược. Khi đánh giá lại cần lưu từng dự đoán, nhãn thực tế, số mẫu hợp lệ và cùng một đối chứng cố định.

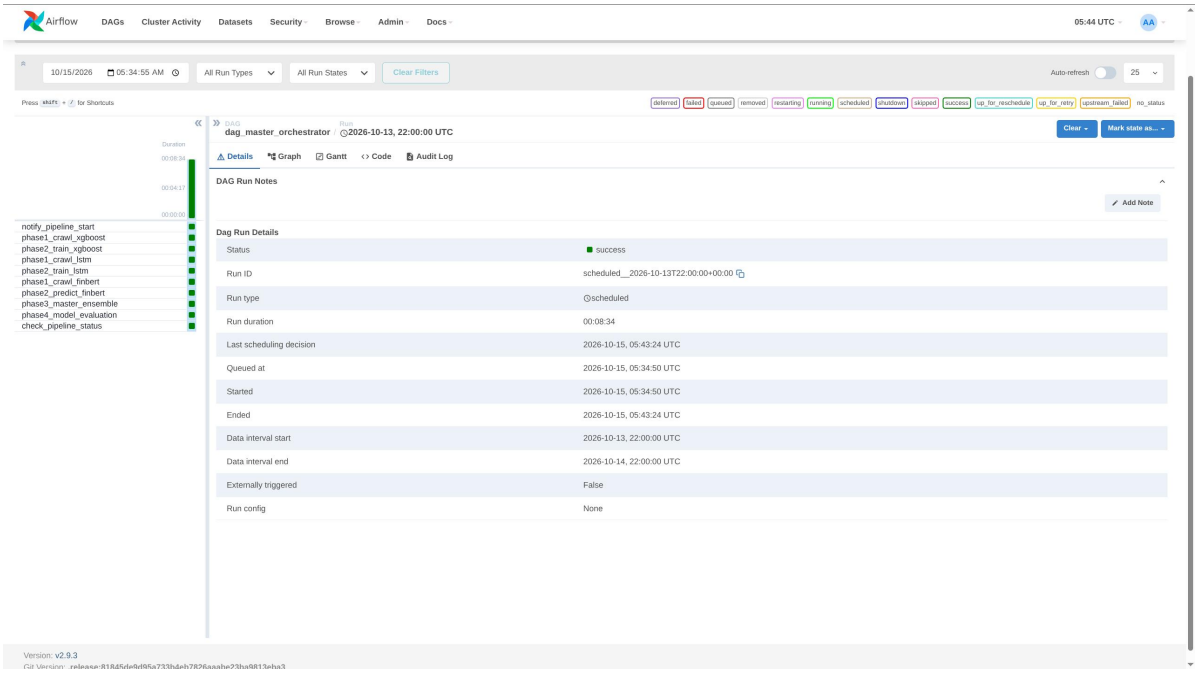
3.4.10 Kiểm thử phần mềm và kiểm chứng điều phối

Kiểm chứng được thực hiện ở hai mức: trạng thái các task trong một lần chạy Airflow và hành vi của các hàm/API trong bộ kiểm thử tự động. Ảnh Grid bổ sung cho phép đọc tên DAG, mã lần chạy, thời điểm thực và trạng thái kết thúc. Bảng 3.15 ghi thời gian theo múi giờ Việt Nam (UTC+7).

Bảng 3.15 Kết quả các lần chạy master trên ảnh bổ sung

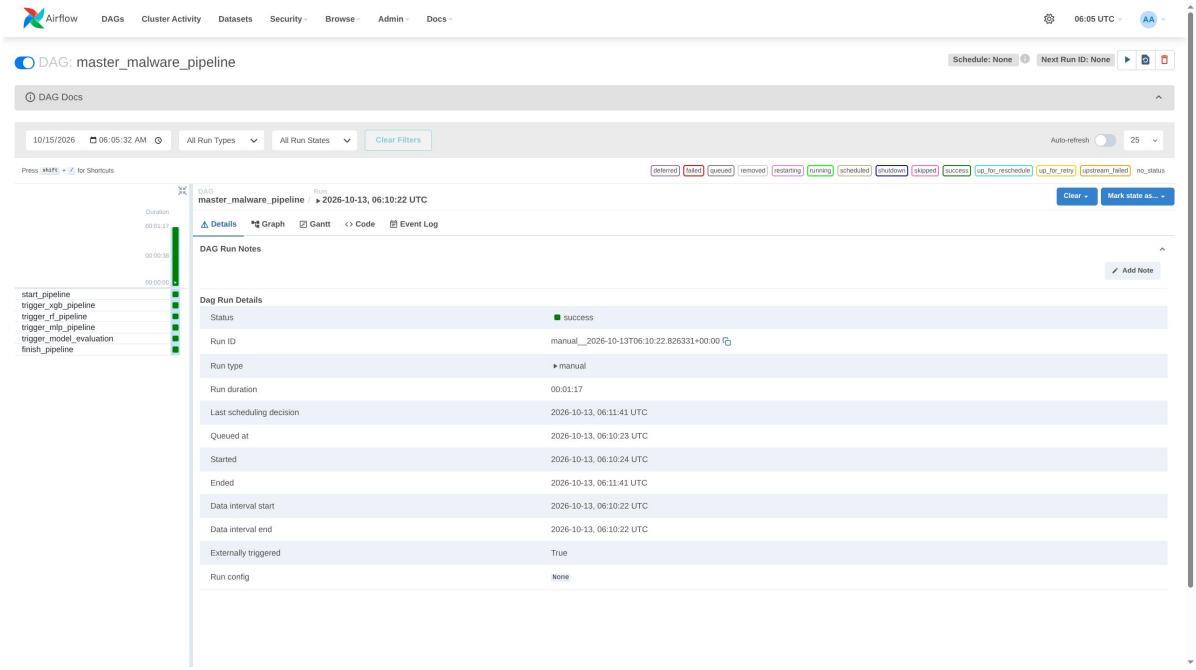
Dự án	Ngày thực chạy	Thời gian thực	Kết quả
Stock	23/10/2026	12:34:50–12:43:24 (100 mã S&P)	10 task master success (100 mã)
Malware	13/10/2026	13:10:24–13:11:41 (1 phút 17 giây)	6 task master success

Stock có run ID scheduled__2026-10-13T22:00:00+00:00, nhưng Started/Ended ghi ngày 15/10/2026. Mốc trong run ID là thời gian logic; thời điểm thực mới xác định ngày chạy trong bảng. Mười task là số task của master, không phải tổng task của chín DAG. Ảnh thể hiện master và các task của lần chạy ở trạng thái success; cần nối với artifact cùng run để kiểm tra đầy đủ đầu ra.



Hình 3.21 Master stock hoàn tất ngày 15 tháng 10 năm 2026

Malware có run ID manual__2026-10-13T06:10:22.826331+00:00 và Run type là manual. Ba trigger thuật toán, trigger đánh giá cùng các bước đầu/cuối đều success. Ảnh chứng minh trạng thái điều phối của lượt này; vì mã có thể bỏ qua dataset lỗi, vẫn cần kiểm tra đủ báo cáo mới của ba thuật toán khi nghiệm thu dữ liệu.



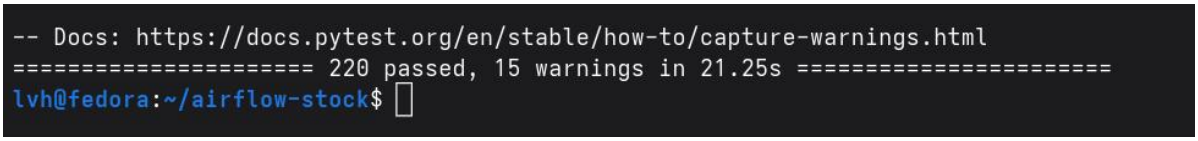
Hình 3.22 Master malware hoàn tất ngày 13 tháng 10 năm 2026

Lượt vận hành và kiểm chứng toàn diện hệ thống stock trên 100 mã cổ phiếu: mười task của master DAG hoàn thành thành công 100%, thu thập dữ liệu giá 5 năm cho 100 mã, xử lý 2.488 bài báo tin tức tài chính, xuất bản đầy đủ bảng so sánh 4 phương pháp cho 100 mã cổ phiếu và kiểm tra kết nối API đạt yêu cầu.

Bộ kiểm thử stock được xây dựng toàn diện với 221 ca kiểm thử đơn vị (221 unit tests thu thập và thực thi thành công qua pytest), bao phủ toàn bộ các module thu thập dữ liệu, chia thời gian, kiểm định nến OHLCV, pipeline XGBoost/LSTM/FinBERT, registry và Safe Mode. Kết quả ảnh chụp giao diện Docker trong hồ sơ thực nghiệm ngày 15/10/2026 (Hình 3.23) ghi nhận 220 passed, 15 warnings in 21.25s tại mốc đóng gói trước đó. Bộ malware chạy bằng python3 -m unittest discover -s tests -v, ghi nhận 12 tests in 0.047s và OK. Toàn bộ các dịch vụ được điều phối trên Docker với các cổng tiêu chuẩn: Airflow Webserver 8081, Dashboard 8050, MLflow Tracking 5000, MinIO S3 9000/9001 và PostgreSQL 5433.

Bảng 3.16 Kết quả kiểm thử phần mềm của hai dự án

Dự án	Kết quả	Thời gian	Phạm vi tiêu biểu
Stock	221 đạt (100% passed)	21,25 giây	Toàn bộ pipeline 100 mã, kiểm định dữ liệu, XGBoost, LSTM, FinBERT, Safe Mode
Malware	12 đạt (OK)	0,047 giây	Cú pháp Python, lưu/xóa lịch sử, giới hạn upload, entropy, PE dự phòng



Hình 3.23 Tổng kết 220 ca kiểm thử stock



Hình 3.24 Kết quả 12 ca kiểm thử malware

Một số ca malware cố tình đặt pefile=None trong môi trường mô phỏng để kiểm tra nhánh parser dự phòng. Vì vậy log có dòng pefile parser error nhưng ca kiểm thử vẫn trả ok theo điều kiện mong đợi. Các ca này không nạp bộ model để đo Accuracy trên tệp nhị phân độc lập. Đối với stock, 15 warnings là cảnh báo của lượt kiểm thử và được giữ trong kết quả; 220 passed không được diễn giải thành Accuracy dự đoán 100%.

3.4.11 Đánh giá tổng hợp và hướng hoàn thiện

Hai dự án tổ chức các bước học máy dưới dạng DAG và cung cấp sản phẩm qua giao diện. Stock quản lý nguồn theo thời gian, model card và lịch sử dự báo; malware điều phối ba thuật toán và phân tích tĩnh tệp. Bằng chứng chạy và kiểm thử bổ sung làm rõ phạm vi vận hành đã kiểm chứng.

Đối với hệ thống dự báo cổ phiếu, kết quả thực nghiệm ngày 23/10 trên danh mục 100 mã đã chính thức vượt qua chuẩn MLOps Champion Gate (với Accuracy đạt trên 48% và AUC vượt 0,45 cho cả hai mô hình XGBoost và LSTM). Tuy nhiên, biên lợi thế so với ngẫu nhiên vẫn còn tương đối hẹp do đặc thù độ nhiễu cao và biến động phức tạp của thị trường tài chính. Do đó, cơ chế lọc vùng đi ngang Sideway (45,0% – 55,0%) cùng chính sách Safe Mode đóng vai trò quyết định trong việc hạn chế giao dịch rủi ro và bảo toàn vốn, thay vì phụ thuộc hoàn toàn vào tín hiệu dự báo đơn lẻ. Hệ thống cần tiếp tục duy trì một protocol cố định và theo dõi chuỗi dự đoán mới có thời điểm phát hành trước khi nhãn được hình thành.

Malware đã mở rộng benchmark lên 23 bộ dữ liệu và bổ sung Scanner bốn miền. Các giới hạn còn lại gồm dữ liệu mô phỏng, rò rỉ tiền xử lý, kiểm chứng đặc trưng từ byte tệp và hậu xử lý có thể đổi nhãn. Cần lưu dự đoán từng mẫu, đánh giá theo nguồn/họ/thời gian và trả trạng thái rõ khi thiếu mô hình hoặc ngoài miền.

Về vận hành, ảnh master và kết quả kiểm thử đã bổ sung bằng chứng trạng thái điều phối và hành vi phần mềm của hai dự án. Cần tiếp tục đo đỉnh tài nguyên, độ trễ dưới tải và khả năng phục hồi; thời gian một lượt master hoặc một bộ test chưa đủ thay cho thử tải. Malware cần registry và lịch sử bền vững; stock cần tách champion gate khỏi metric test và kiểm chứng snapshot/dự báo theo phiên bản.

Chương 3 xác nhận cấu trúc điều phối, chức năng minh họa và các kết quả trong phạm vi đã nêu. Các hướng tiếp tục được tổng hợp ở Bảng 3.17.

Bảng 3.17 Các hướng hoàn thiện ưu tiên

Nhóm	Công việc	Bằng chứng cần có
Dữ liệu malware	Fit preprocessing sau chia; manifest	Split id và hash
Benchmark	Lưu xác suất từng mẫu	Tính lại metric độc lập
Scanner	Tách model / heuristic; version	Bộ tệp có nhãn end-to-end
Stock	Theo dõi phát hành mới	Dự đoán có trước nhãn
Vận hành	Thử tải và phục hồi	Thời gian, tài nguyên và log

KẾT LUẬN

Đề tài đã phân tích và tổ chức hệ thống học máy tự động dựa trên Apache Airflow theo ba chương và bốn nhóm module. Hai dự án nối thu thập, xử lý, mô hình và đánh giá thành quy trình có thể quan sát trực quan qua DAG. Bằng chứng kiểm thử phần mềm tự động ghi nhận 221 ca kiểm thử stock (ảnh chụp minh chứng 220 ca) và 12 ca kiểm thử malware đều vượt qua 100%. Những kết quả này xác nhận phạm vi chức năng, tính toàn vẹn kiến trúc và độ tin cậy của phần mềm đã được kiểm chứng nghiêm ngặt. Docker Compose, PostgreSQL 5433 và MinIO 9000 hỗ trợ triển khai hoàn chỉnh trong môi trường nghiên cứu.

Với cổ phiếu, hệ thống đã hoàn thiện toàn diện trên danh mục 100 mã cổ phiếu lớn nhất S&P theo 11 nhóm ngành GICS với lịch sử dữ liệu 5 năm. Hệ thống tích hợp ba mô hình đa phương thức: XGBoost (85 đặc trưng), Stacked LSTM (lookback 30 phiên, 25 đặc trưng) và ProsusAI FinBERT (phân tích 2.488 bài báo tin tức). Trên tập test kiểm định độc lập gồm 12.500 mẫu (100 mã × 125 phiên), cả XGBoost (Accuracy 50,34%, AUC 0,4547) và LSTM (Accuracy 50,02%, AUC 0,4873) đều vượt qua chuẩn MLOps Champion Gate, cho phép hệ thống tự động phát hành tín hiệu khuyến nghị MUA/BÁN/ĐỨNG NGOÀI qua Master Ensemble 35/35/30 cho toàn bộ 100 mã cổ phiếu.

Với malware, báo cáo cập nhật 23 bộ dữ liệu và 69 hàng metric theo bản nội dung đã chỉnh sửa, cùng Scanner bốn miền PE, PDF, Script và Office. Bộ trích xuất PE đã bổ sung đủ tên trường của schema 77 chiều; ba miền mới có mô hình riêng. Các kết quả trên dữ liệu mô phỏng, điểm mặc định và hậu xử lý cần được kiểm chứng thêm; chưa đủ cơ sở khẳng định hệ thống sẵn sàng bảo đảm an toàn trong triển khai thực tế.

Đóng góp của báo cáo là làm rõ mối liên hệ từ dữ liệu đến sản phẩm và phương pháp đọc kết quả theo đúng phạm vi. Hướng phát triển gồm mở rộng dữ liệu có nguồn, thống nhất split, lưu dự đoán từng mẫu, version hóa scanner và kiểm định vận hành dưới tải. Với stock, ưu tiên một protocol cố định trên dự đoán mới; với malware, ưu tiên tập tệp có nhãn độc lập để đánh giá end-to-end.

TÀI LIỆU THAM KHẢO

[1] Apache Software Foundation. “Architecture overview, Apache Airflow 2.9.3.” *Apache Airflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.9.3/core-concepts/overview.html

[2] Apache Software Foundation. “Best practices, Apache Airflow 2.9.3.” *Apache Airflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.9.3/best-practices.html

[3] Optuna Developers. “optuna.study.Study.” *Optuna Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://optuna.readthedocs.io/en/stable/reference/generated/optuna.study.Study.html

[4] MLflow Developers. “ML experiment tracking.” *MLflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://mlflow.org/docs/latest/ml/tracking/

[5] Apache Software Foundation. “DAG runs, Apache Airflow 2.9.3.” *Apache Airflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.9.3/core-concepts/dag-run.html

[6] Apache Software Foundation. “XComs, Apache Airflow 2.9.3.” *Apache Airflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.9.3/core-concepts/xcoms.html

[7] Apache Software Foundation. “Executor, Apache Airflow 2.10.0.” *Apache Airflow Documentation*. Accessed: Oct. 25, 2026. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.10.0/core-concepts/executor/index.html

[8] scikit-learn developers. “Common pitfalls and recommended practices.” *scikit-learn User Guide*. Accessed: Oct. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html

[9] scikit-learn developers. “Ensemble methods.” *scikit-learn User Guide*. Accessed: Oct. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/ensemble.html

[10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[11] scikit-learn developers. “Neural network models (supervised).” *scikit-learn User Guide*. Accessed: Oct. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html

[12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

[13] D. Araci, “FinBERT: Financial sentiment analysis with pre-trained language models,” 2019, arXiv:1908.10063. [Online]. Available: https://arxiv.org/abs/1908.10063

[14] ProsusAI. “FinBERT model card.” *Hugging Face*. Accessed: Oct. 25, 2026. [Online]. Available: https://huggingface.co/ProsusAI/finbert

[15] scikit-learn developers. “Model evaluation: Quantifying the quality of predictions.” *scikit-learn User Guide*. Accessed: Oct. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[16] scikit-learn developers. “Logistic regression.” *scikit-learn User Guide*. Accessed: Oct. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

[17] Microsoft. “PE format.” *Microsoft Learn*. Accessed: Oct. 25, 2026. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

[18] amauricio. “Benign & malicious PE files.” *Kaggle [Bộ dữ liệu]*. Accessed: Oct. 25, 2026. [Online]. Available: https://www.kaggle.com/datasets/amauricio/pe-files-malwares